

Bölüm 11: Çizgeler

Veri Yapıları

Temel kavramlar

Derece, yol, döngü

Komşuluk listesi ve matrisi

BFS

DFS

Ağırlıksız en kısa yol

Dijkstra

Sercan KÜLCÜ · Giresun Üniversitesi · Bilgisayar Mühendisliği

Öğrenme Çıktıları

Temel kavramlar

Düğüm, kenar, yön, ağırlık, yol ve bağlantılılık kavramlarını açıklamak.

Gösterimler

Komşuluk matrisi ile komşuluk listesini maliyetleriyle karşılaştırmak.

Gezinme ve sıralama

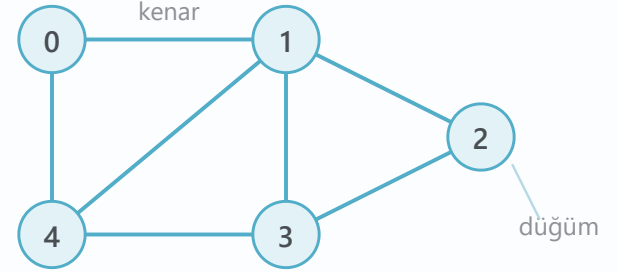
BFS, DFS ve DAG'lerde topolojik sıralamayı uygulayıp sonuçlarını yorumlamak.

En kısa yol

Ağırlıksız BFS ve Dijkstra algoritması için uygun problem türünü seçmek.

Çizge (Graph)

- D ğ mlerin (vertex / node) ve bu d ğ mleri birbirine ba layan kenarların (edge) bir araya gelmesiyle olu ur.
- Karma ık ili kileri, a ları ve yapıları temsil etmek i in kullanılır.
-  izge teorisi, d ğ mler ve kenarlar arasındaki ili kileri inceler.



Bir  izge $G = (V, E)$ ikilisidir: V d ğ mler k mesi, E kenarlar k mesi.

 rnek: $V = \{0, 1, 2, 3, 4\}$, $E = \{(0,1), (0,4), (1,2), (1,3), (1,4), (2,3), (3,4)\}$ · $n = |V| = 5$, $m = |E| = 7$

A a  ile farkı

K k yoktur, d ng  olabilir,
bir d ğ me birden  ok
yoldan ula ılabilir.

B l m 1'den hatırla

 izge "bir o a-bir ok"
ili kiyi modeller.

Notasyon

Bu b l mde n = d ğ m
sayısı, m = kenar sayısı.

Düğüm (Vertex, Node)

- Çizgenin temel yapı taşıdır.
- Şehirler, kişiler, bilgisayarlar gibi nesneleri temsil eder.
- Düğüm, bir değer (etiket, ad, konum...) saklayabilir.
- **Örnek:** haritadaki şehirler ve kavşaklar.

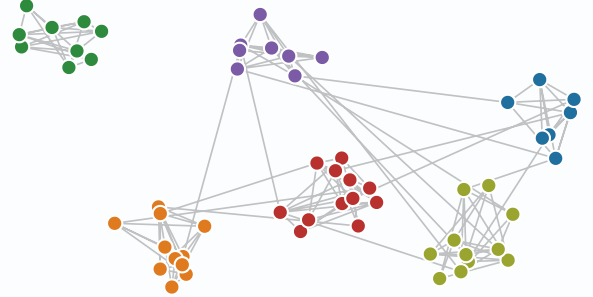


Karayolu haritası: şehirler düğüm, yollar kenar

Harita çizge olarak modellenince "İstanbul'dan Van'a en kısa yol hangisi?" sorusu bir **çizge algoritması** problemine dönüşür (bölümün sonunda: Dijkstra).

Kenar (Edge)

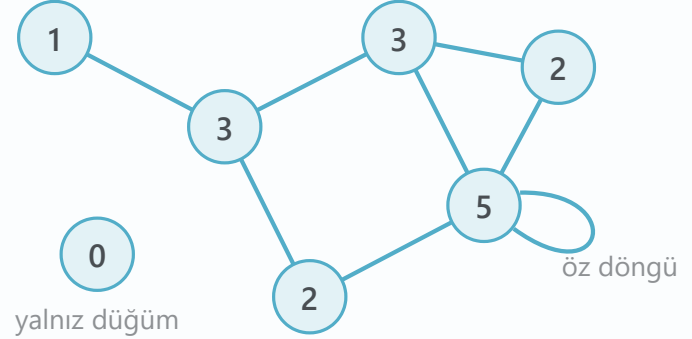
- D    mleri birbirine ba layan ba lantılardır.
- Kenarlar d    mler arasındaki ili kiyi temsil eder.
- (v, w) bir kenar ise v ve w **biti ik** (kom u, adjacent) d    mlerdir; kenar bu iki d    me **ba lıdır** (incident).
- ** rnek:**  ehirler arasındaki yollar, bilgisayar a ındaki ba lantılar, sosyal a daki arkada lıklar.



Bir sosyal a : ki iler d    m, ili kiler kenar

Derece (Degree)

- Bir düğümün derecesi, ona bağlı kenar sayısıdır: $\text{deg}(v)$.
- Derecesi 0 olan (yalnız, isolated) düğümler de olabilir.
- Öz döngü (self-loop) dereceye **2** katkı yapar.
- **Örnek:** bir kavşaktaki yolların sayısı.



Her düğümün içinde kendi derecesi yazıyor

El sıkışma lemması

Her kenar iki uca birer derece verir:

$$\sum \text{deg}(v) = 2m$$

Örnekte: $1+3+3+2+5+2+0 = 16 = 2 \cdot 8$ kenar.

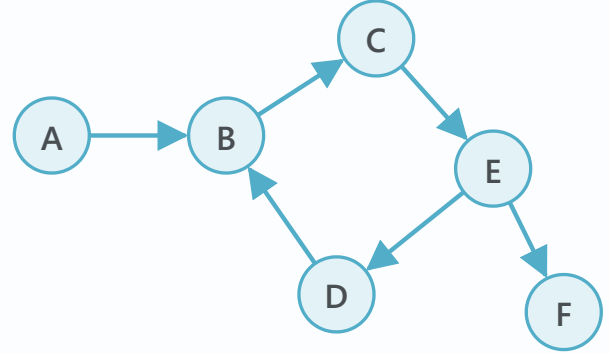
Yönlü çizgede

giriş derecesi (in-degree): gelen ok sayısı, *çıkış derecesi* (out-degree): giden ok sayısı.

$$\sum \text{giriş} = \sum \text{çıkış} = m$$

Yönlü Çizge (Digraph)

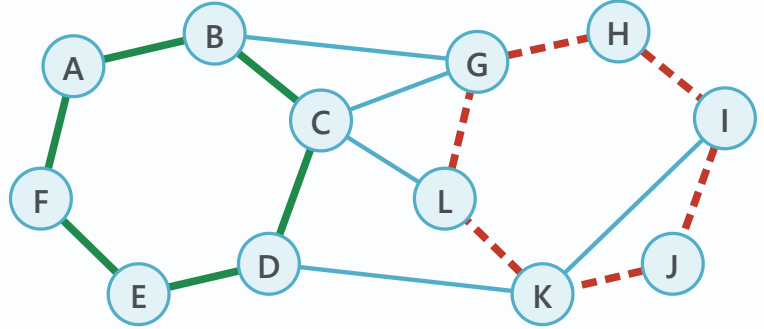
- Kenarlar bir düğümden diğerine **belirli bir yöndedir**: $(u, v) \neq (v, u)$.
- Geri dönüş yolu olmayabilir: örnekte B'den A'ya ulaşamaz.
- Yönlü çizge (directed graph), kısaca *digraph* olarak da adlandırılır.
- Örnek**: web sayfaları ve bağlantıları, ders ön koşulları, tek yönlü sokaklar.



Düğüm	A	B	C	D	E	F
giriş derecesi	0	2	1	1	1	1
çıkış derecesi	1	1	1	1	2	0

Yol (Path) ve Döngü (Cycle)

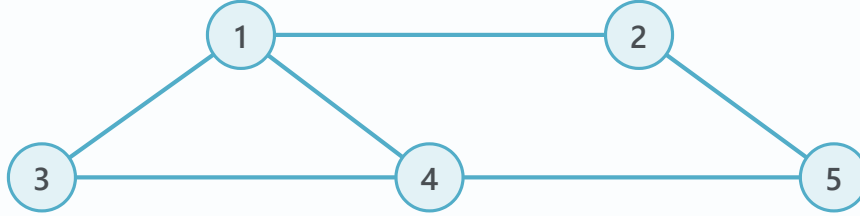
- **Yol (path):** ardışık düğümleri kenarlarla bağlanan düğüm dizisi.
- **Basit yol:** hiçbir düğümü tekrar etmeyen yol.
- **Döngü (cycle):** aynı düğümde başlayıp biten yol.
- **Basit döngü:** başlangıç düğümü dışında hiçbir düğümü tekrar etmez.
- Çizgelerde döngü olabilir; **ağaçta döngü yoktur.**



Yeşil: A-B-C-D-E-F-A · Kırmızı (kesikli): G-H-I-J-K-L-G

Örnek: uçuş rotası. İstanbul → Ankara → İzmir → İstanbul bir döngüdür; aynı şehre uğramadan dönüyorsa basit döngüdür.

Örnek Çizge

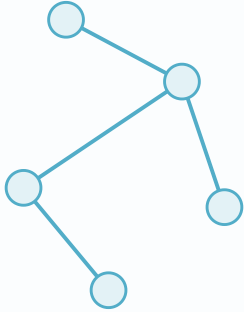


- $V = \{1, 2, 3, 4, 5\}$
- $E = \{(1,2), (1,3), (1,4), (2,5), (3,4), (4,5)\}$
- Düğüm 1 ve düğüm 2 **komşu**.
- $\deg(1) = 3$, $\deg(4) = 3$, diğerleri 2 $\rightarrow$ toplam 12
 $= 2 \cdot 6 \checkmark$

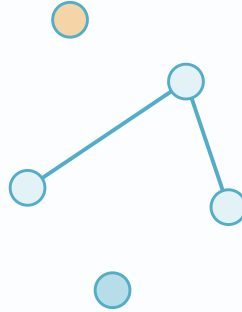
Dizi	Türü
$1 \rightarrow 2 \rightarrow 5$	basit yol
$1 \rightarrow 3 \rightarrow 4 \rightarrow 1$	basit döngü
$1 \rightarrow 3 \rightarrow 4 \rightarrow 1 \rightarrow 2$	basit olmayan yol (1 tekrar)
$1 \rightarrow 3 \rightarrow 4 \rightarrow 1 \rightarrow 4 \rightarrow 1$	basit olmayan döngü

Bağlantılı ve Tam Çizge

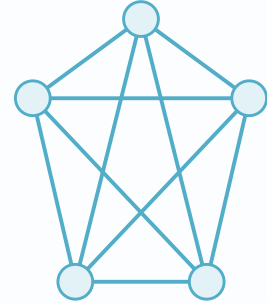
- **Bağlantılı (connected) çizge:** her iki farklı düğüm çifti arasında bir *yol* vardır.
- **Bağlantılı bileşen (connected component):** birbirine yolla ulaşabilen düğümlerin en büyük grubu.
- **Tam (complete) çizge:** her iki farklı düğüm çifti arasında bir *kenar* vardır.



Bağlantılı: 1 bileşen



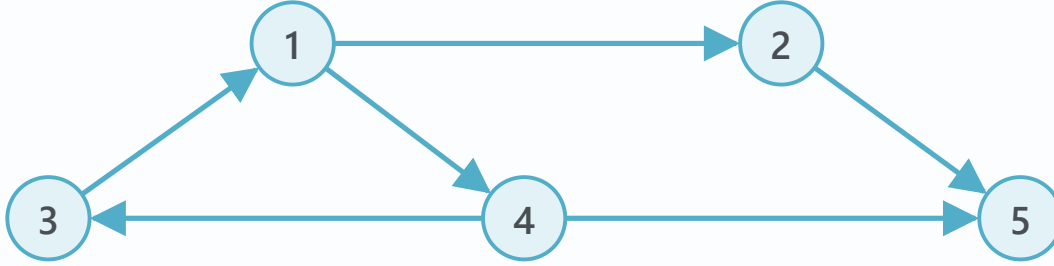
Bağlantısız: 3 bileşen



Tam çizge K_5 : 10 kenar

n düğümlü basit yönsüz çizgede $m \leq n(n-1)/2$; bu üst sınıra tam çizgede ulaşılır. Bağlantılı bir çizgede $m \geq n - 1$.

Yönlü Çizge Örneği



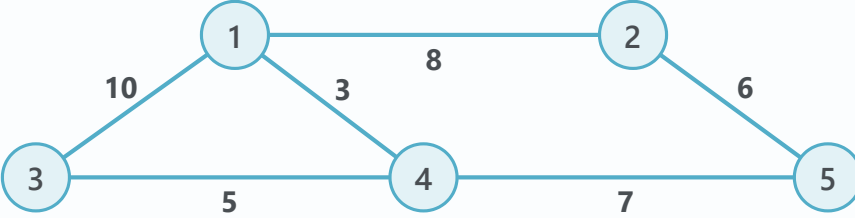
- $V = \{1, 2, 3, 4, 5\}$
- $E = \{(1,2), (1,4), (2,5), (4,5), (3,1), (4,3)\}$
- Düğüm 2, düğüm 1'in **komşusu** ($1 \rightarrow 2$ kenarı var).
- Düğüm 1, düğüm 2'nin komşusu **değil!**

Dizi	Türü
$1 \rightarrow 2 \rightarrow 5$	yönlü yol
$1 \rightarrow 4 \rightarrow 3 \rightarrow 1$	yönlü döngü
$5 \rightarrow \dots$	5'ten çıkan kenar yok

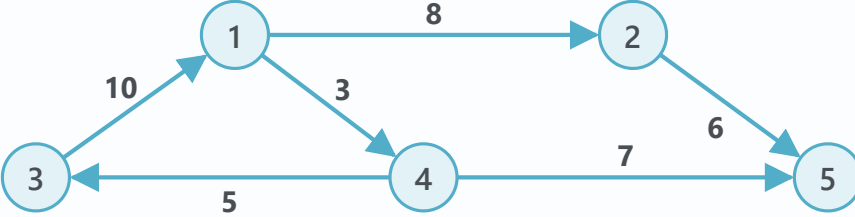
Yönlü kenar (u, v) sıralı bir çifttir: u kuyruk (başlangıç), v baş (hedef) düğümdür.

Ağırlıklı Çizge

- Her kenara bir sayı (**ağırlık**, weight) atanır: uzaklık, süre, maliyet, kapasite...
- Bir yolun uzunluğu = üzerindeki kenar ağırlıklarının toplamı. Yönsüz veya yönlü olabilir.



Ağırlıklı yönsüz: $3 \rightarrow 4 \rightarrow 1$ yolu $5 + 3 = 8$



Ağırlıklı yönlü: $1 \rightarrow 4 \rightarrow 3$ yolu $3 + 5 = 8$

Doğrudan kenar her zaman en kısa yol değildir: yönsüz çizgede 1–3 kenarı 10, ama $1 \rightarrow 4 \rightarrow 3$ yolu yalnızca 8.

Çizge Soyut Veri Tipi (Graph ADT)

Goodrich'teki `Graph` arayüzünün temel işlemleri (V: düğüm, E: kenar):

İşlem	Açıklama
<code>numVertices()</code> , <code>numEdges()</code>	Düğüm sayısı n, kenar sayısı m
<code>vertices()</code> , <code>edges()</code>	Tüm düğümler / tüm kenarlar üzerinde gezinme
<code>getEdge(u, v)</code>	u'dan v'ye kenar varsa onu, yoksa null döndürür
<code>endVertices(e)</code> , <code>opposite(v, e)</code>	Kenarın iki ucu / e kenarının v dışındaki ucu
<code>outDegree(v)</code> , <code>inDegree(v)</code>	Çıkış / giriş derecesi (yönsüzde ikisi eşit)
<code>outgoingEdges(v)</code> , <code>incomingEdges(v)</code>	v'den çıkan / v'ye gelen kenarlar
<code>insertVertex(x)</code> , <code>insertEdge(u, v, x)</code>	Yeni düğüm / yeni kenar ekler
<code>removeVertex(v)</code> , <code>removeEdge(e)</code>	Düğümü (bağlı kenarlarıyla) / kenarı siler

Bu işlemlerin maliyeti, çizgenin bellekte nasıl temsil edildiğine bağlıdır → sonraki alt başlık: Çizge Gösterimleri.

Çizge Nasıl Saklanır? Üç Temel Gösterim

Kenar listesi

Tüm kenarlar tek bir listede tutulur.

$[(A, B), (A, E), \dots]$

Komşuluk listesi

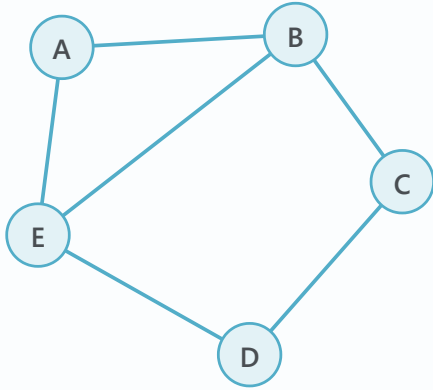
Her düğüm için komşularının listesi tutulur.

$V[i] = \{j, k, l\}$

Komşuluk matrisi

$n \times n$ iki boyutlu dizi; hücre, kenarın varlığını gösterir.

$M[i][j] = 1$



Kenar listesi (edge list) en basit gösterimdir:

(A, B)

(A, E)

(B, C)

(B, E)

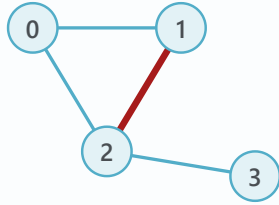
(C, D)

(D, E)

- Alan $O(n + m)$; kenar eklemek $O(1)$.
- Ama "u ile v komşu mu?" ya da "u'nun komşuları kim?" için tüm listeyi taramak gerekir: **$O(m)$** .

Komşuluk Matrisi (Adjacency Matrix)

- $n \times n$ boyutunda iki boyutlu bir dizidir: `matris[i][j]` = 1, düğüm i'den j'ye kenar varsa; yoksa 0.
- **Ağırlıklı çizgede** `matris[i][j]` kenarın ağırlığını tutar; kenar yoksa ∞ (veya özel bir değer).
- **Yönsüz çizgede** matris simetriktir: `matris[i][j] == matris[j][i]`.
- Alan karmaşıklığı $O(n^2)$; düğüm sayısı arttıkça bellek hızla büyür.



	0	1	2	3
0	0	1	1	0
1	1	0	1	0
2	1	1	0	1
3	0	0	1	0

← `m[1][2]`: tek hücre, $O(1)$

← 2'nin komşuları: tüm satır, $O(n)$

simetrik: `m[i][j] = m[j][i]`

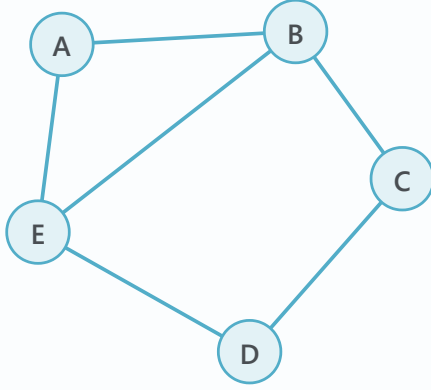
Kolay

- + İki düğüm arasında kenar var mı? $O(1)$
- + Kenar ekleme / silme $O(1)$

Zor

- Bir düğümün komşularını bulmak: tüm satır, $O(n)$
- Seyrek çizgede hücrelerin çoğu boş (0)

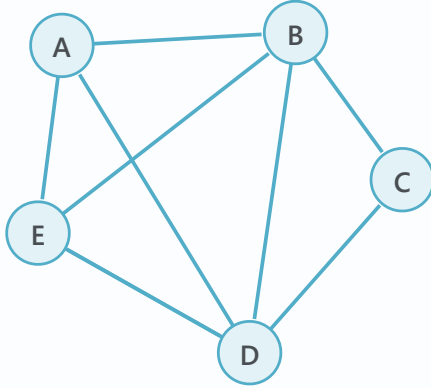
Komşuluk Matrisi: Üç Örnek



	A	B	C	D	E
A	0	1	0	0	1
B	1	0	1	0	1
C	0	1	0	1	0
D	0	0	1	0	1
E	1	1	0	1	0

Yönsüz çizge: her kenar iki hücreye yazılır, matris köşegene göre **simetrik**. Satır toplamı = derece (B: 3).

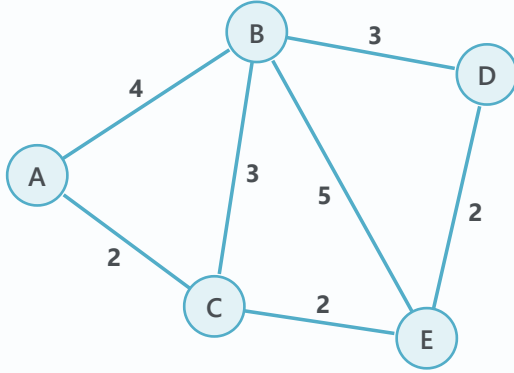
Komşuluk Matrisi: Üç Örnek



	A	B	C	D	E
A	0	1	0	0	1
B	0	0	1	1	0
C	0	0	0	0	0
D	1	0	1	0	1
E	0	1	0	1	0

Yönlü çizge: satır = kaynak, sütun = hedef. Simetrik değil; satır toplamı çıkış, sütun toplamı giriş derecesi.

Komşuluk Matrisi: Üç Örnek

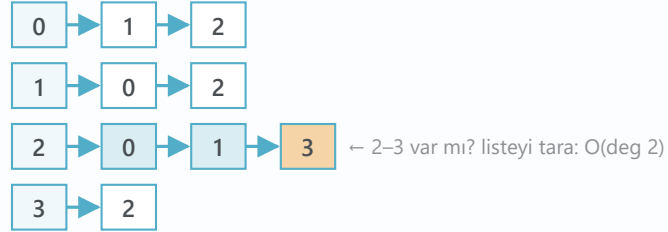
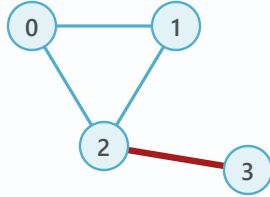


	A	B	C	D	E
A	0	4	∞	∞	∞
B	∞	0	3	3	∞
C	2	∞	0	∞	2
D	∞	∞	∞	0	∞
E	∞	5	∞	2	0

Ağırlıklı çizge: hücrede ağırlık, kenar yoksa ∞ (köşegende 0). Boş hücreye 0 yazmak, 0 ağırlıklı bir kenarla karıştırılabilir.

Komşuluk Listesi (Adjacency List)

- n düğüm için n adet liste (çoğunlukla bağlı liste veya dinamik dizi) tutulur.
- `liste[i]`, düğüm i 'ye komşu olan düğümleri (ağırlıklıysa ağırlıklarıyla) içerir.
- **Yönsüz çizgede** her kenar (v, w) iki listeye de eklenir: w , v 'nin listesine; v , w 'nin listesine.
- Alan karmaşıklığı **$O(n + m)$** ; seyrek çizgeler için idealdir.



← 2-3 var mı? listeyi tara: $O(\deg 2)$

toplam kayıt $2m = 8 \rightarrow$ alan $O(n + m)$

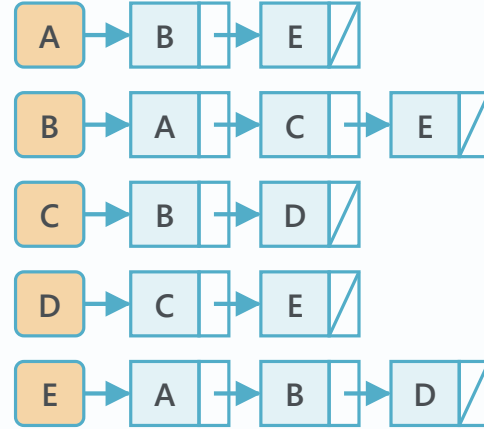
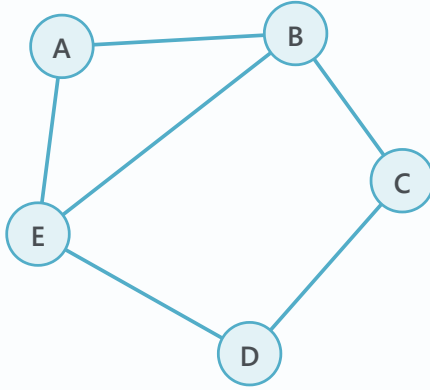
Kolay

- + Bir düğümün komşularını gezmek:
 $O(\deg(v))$
- + Kenar ekleme $O(1)$

Zor

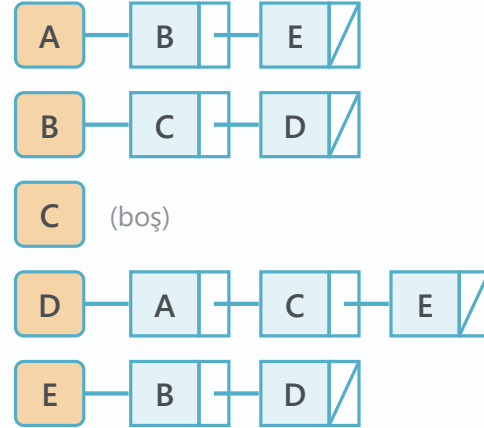
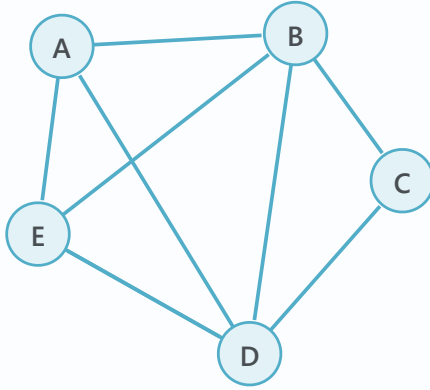
- u ile v arasında kenar var mı? Listeyi taramak gerekir: **$O(\deg(u))$**

Komşuluk Listesi: Üç Örnek



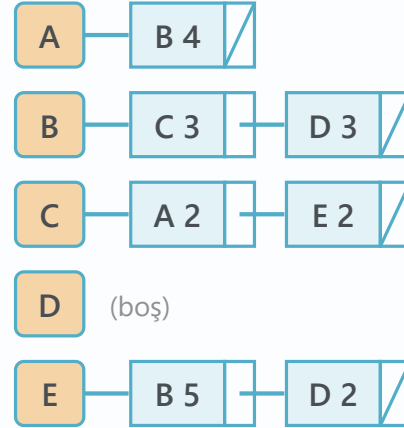
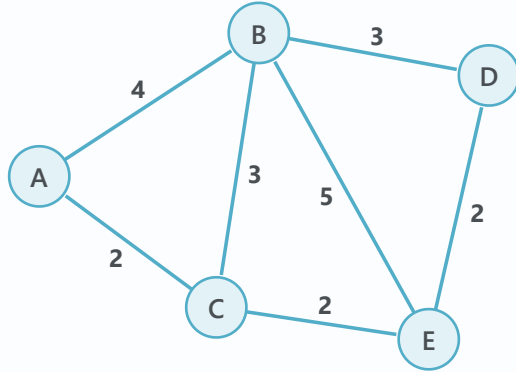
Yönsüz: 6 kenar → listelerde 12 kayıt (her kenar iki kez). Liste uzunluğu = derece.

Komşuluk Listesi: Üç Örnek



Yönlü: yalnızca çıkan kenarlar listelenir; 9 kenar → 9 kayıt. C'den çıkan kenar yok, listesi boş.

Komşuluk Listesi: Üç Örnek



Ağırlıklı: her kayıt (komşu, ağırlık) çifti tutar; ör. B'nin listesi: (C, 3), (D, 3).

Gösterimlerin Bellek ve İşlem Maliyeti

İşlem (n düğüm, m kenar)	Kenar listesi	Komşuluk listesi	Komşuluk matrisi
Bellek	$O(n + m)$	$O(n + m)$	$O(n^2)$
u-v kenarı var mı? <code>getEdge</code>	$O(m)$	yönsüz: $O(\min(\deg u, \deg v))$ yönlü u→v: $O(\text{outDeg } u)$	$O(1)$
v'nin komşularını gez	$O(m)$	$O(\deg v)$	$O(n)$

Seyrek çizge ($m \approx n$)

Yol ağıları, sosyal ağlar → *komşuluk listesi*. $n = 10^6$ düğümde matris 10^{12} hücre ister!

Yoğun çizge ($m \approx n^2$)

Sık "kenar var mı?" sorgusu → *komşuluk matrisi*.

Java ile Komşuluk Listesi

```
class Cizge {
    private final int n; // düğümler: 0..n-1
    private final List<List<Integer>> komsu = new ArrayList<>();
    Cizge(int n) {
        this.n = n;
        for (int i = 0; i < n; i++)
            komsu.add(new ArrayList<>());
    }
    void kenarEkle(int u, int v) {
        komsu.get(u).add(v);
        komsu.get(v).add(u); // yönlü çizgede bu satır yok
    }
    List<Integer> komsular(int u) { return komsu.get(u); } // O(1)
    int derece(int u) { return komsu.get(u).size(); }
}
```

Matris karşılığı

boolean[][] m = new boolean[n][n];
 kenar ekle: m[u][v] = m[v][u] = true;
 sorgu m[u][v] O(1).

Ağırlıklı çizge

Liste ögesi int[]{v, w} veya küçük bir
 Kenar sınıfı; matriste int[][] , kenar yoksa
 Integer.MAX_VALUE .

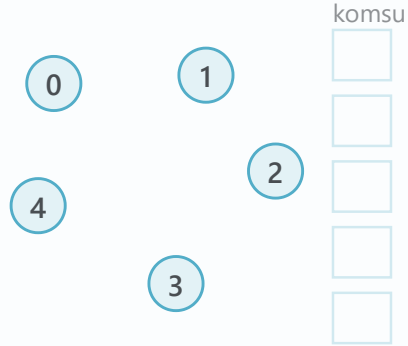
Kod İzleme: Komşuluk Listesi Kurma

```
1 Cizge(int n) {  
    ▶ this.n = n;  
3   for (int i = 0; i < n; i++)  
4       komsu.add(new ArrayList<>());  
5 }  
6 void kenarEkle(int u, int v) {  
7     komsu.get(u).add(v);  
8     komsu.get(v).add(u); // yönlüde yok  
9 }  
10 // main:  
11 Cizge g = new Cizge(5);  
12 int[][] E = {{0,1}, {0,4}, {1,2},  
13              {1,4}, {2,3}, {3,4}};  
14 for (int[] e : E) g.kenarEkle(e[0], e[1]);
```

ÇAĞRI YIĞINI

Cizge(5)

main()



`new Cizge(5)` : yapıcı çağrılır; **n = 5** atanır.

`komsu` henüz boş bir dış liste.

Adım 1 / 19

DEĞİŞKENLER

`n = 5`

Kod İzleme: Komşuluk Listesi Kurma

```

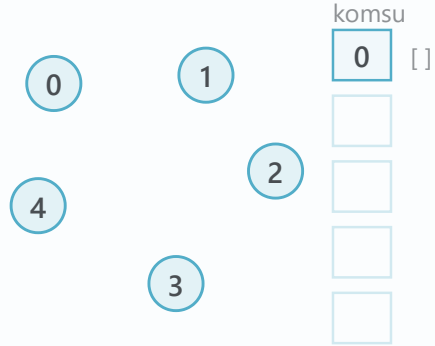
1 Cizge(int n) {
2   this.n = n;
3   for (int i = 0; i < n; i++)
4     komsu.add(new ArrayList<>());
5 }
6 void kenarEkle(int u, int v) {
7   komsu.get(u).add(v);
8   komsu.get(v).add(u); // yönlüde yok
9 }
10 // main:
11 Cizge g = new Cizge(5);
12 int[][] E = {{0,1}, {0,4}, {1,2},
13              {1,4}, {2,3}, {3,4}};
14 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```

ÇAĞRI YIĞINI

Cizge(5)

main()



i = 0: 0 numaralı düğüm için **boş bir iç liste** eklenir.

Adım 2 / 19

DEĞİŞKENLER

n = 5

i = 0

Kod İzleme: Komşuluk Listesi Kurma

```

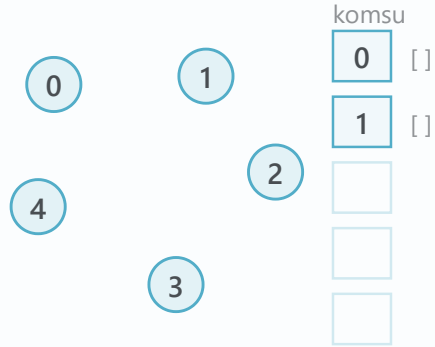
1 Cizge(int n) {
2   this.n = n;
3   for (int i = 0; i < n; i++)
4     komsu.add(new ArrayList<>());
5 }
6 void kenarEkle(int u, int v) {
7   komsu.get(u).add(v);
8   komsu.get(v).add(u); // yönlüde yok
9 }
10 // main:
11 Cizge g = new Cizge(5);
12 int[][] E = {{0,1}, {0,4}, {1,2},
13              {1,4}, {2,3}, {3,4}};
14 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```

ÇAĞRI YIĞINI

Cizge(5)

main()



i = 1: 1 numaralı düğüm için **boş bir iç liste** eklenir.

Adım 3 / 19

DEĞİŞKENLER

n = 5

i = 1

Kod İzleme: Komşuluk Listesi Kurma

```

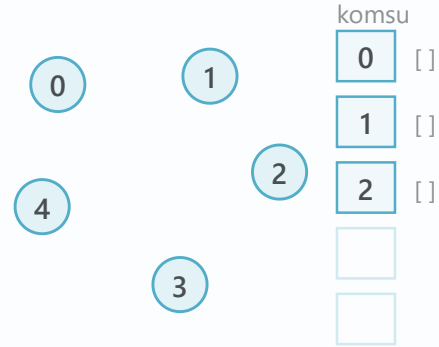
1 Cizge(int n) {
2   this.n = n;
3   for (int i = 0; i < n; i++)
4     komsu.add(new ArrayList<>());
5 }
6 void kenarEkle(int u, int v) {
7   komsu.get(u).add(v);
8   komsu.get(v).add(u); // yönlüde yok
9 }
10 // main:
11 Cizge g = new Cizge(5);
12 int[][] E = {{0,1}, {0,4}, {1,2},
13              {1,4}, {2,3}, {3,4}};
14 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```

ÇAĞRI YIĞINI

Cizge(5)

main()



i = 2: 2 numaralı düğüm için **boş bir iç liste** eklenir.

Adım 4 / 19

DEĞİŞKENLER

n = 5

i = 2

Kod İzleme: Komşuluk Listesi Kurma

```

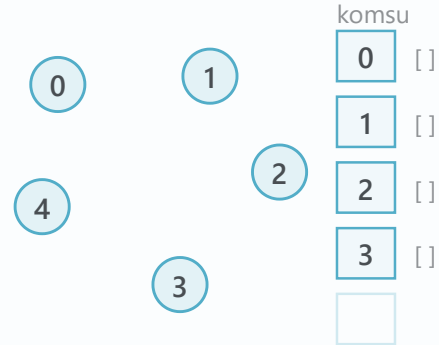
1 Cizge(int n) {
2   this.n = n;
3   ▶ for (int i = 0; i < n; i++)
4     ▶ komsu.add(new ArrayList<>());
5 }
6 void kenarEkle(int u, int v) {
7   komsu.get(u).add(v);
8   komsu.get(v).add(u); // yönlüde yok
9 }
10 // main:
11 Cizge g = new Cizge(5);
12 int[][] E = {{0,1}, {0,4}, {1,2},
13              {1,4}, {2,3}, {3,4}};
14 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```

ÇAĞRI YIĞINI

Cizge(5)

main()



i = 3: 3 numaralı düğüm için **boş bir iç liste** eklenir.

Adım 5 / 19

DEĞİŞKENLER

n = 5

i = 3

Kod İzleme: Komşuluk Listesi Kurma

```

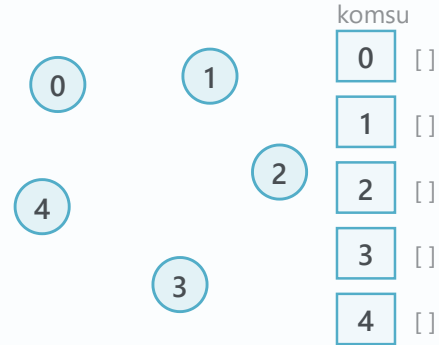
1 Cizge(int n) {
2   this.n = n;
3   ▶ for (int i = 0; i < n; i++)
4     ▶ komsu.add(new ArrayList<>());
5 }
6 void kenarEkle(int u, int v) {
7   komsu.get(u).add(v);
8   komsu.get(v).add(u); // yönlüde yok
9 }
10 // main:
11 Cizge g = new Cizge(5);
12 int[][] E = {{0,1}, {0,4}, {1,2},
13             {1,4}, {2,3}, {3,4}};
14 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```

ÇAĞRI YIĞINI

Cizge(5)

main()



i = 4: 4 numaralı düğüm için **boş bir iç liste** eklenir. Artık her düğümün (boş) komşu listesi var.

Adım 6 / 19

DEĞİŞKENLER

n = 5

i = 4

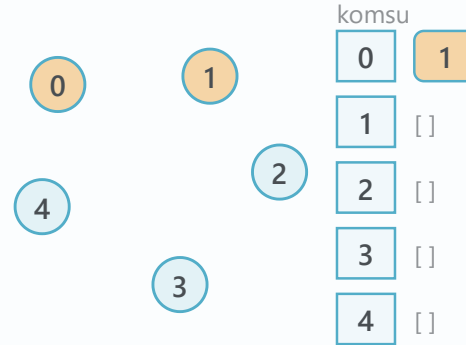
Kod İzleme: Komşuluk Listesi Kurma

```
1 Cizge(int n) {  
2     this.n = n;  
3     for (int i = 0; i < n; i++)  
4         komsu.add(new ArrayList<>());  
5 }  
6 void kenarEkle(int u, int v) {  
7     komsu.get(u).add(v);  
8     komsu.get(v).add(u); // yönlüde yok  
9 }  
10 // main:  
11 Cizge g = new Cizge(5);  
12 int[][] E = {{0,1}, {0,4}, {1,2},  
13             {1,4}, {2,3}, {3,4}};  
14 for (int[] e : E) g.kenarEkle(e[0], e[1]);
```

ÇAĞRI YIĞINI

kenarEkle(0, 1)

main()



Kenar 0–1: **1**, 0'ın listesinin sonuna eklenir.

Adım 7 / 19

DEĞİŞKENLER

u = 0

v = 1

Kod İzleme: Komşuluk Listesi Kurma

```

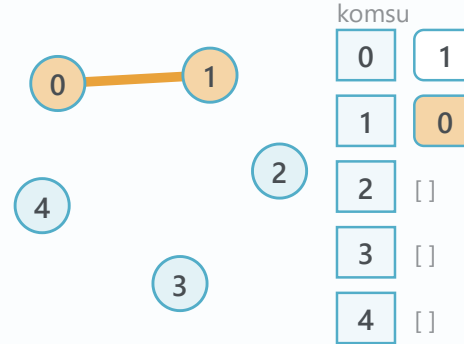
1 Cizge(int n) {
2   this.n = n;
3   for (int i = 0; i < n; i++)
4     komsu.add(new ArrayList<>());
5 }
6 void kenarEkle(int u, int v) {
7   komsu.get(u).add(v);
8   komsu.get(v).add(u); // yönlüde yok
9 }
10 // main:
11 Cizge g = new Cizge(5);
12 int[][] E = {{0,1}, {0,4}, {1,2},
13              {1,4}, {2,3}, {3,4}};
14 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```

ÇAĞRI YIĞINI

kenarEkle(0, 1)

main()



Yönsüz çizge: aynı kenar ters yönde de eklenir → 0, 1'in listesine.

Adım 8 / 19

DEĞİŞKENLER

u = 0

v = 1

Kod İzleme: Komşuluk Listesi Kurma

```

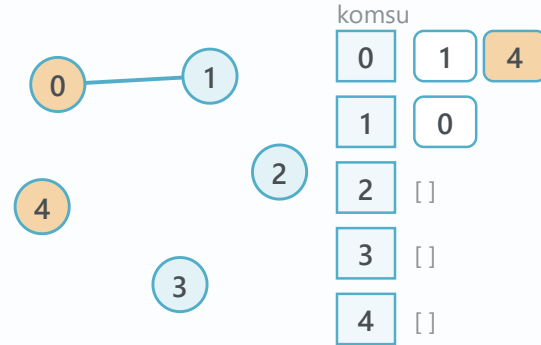
1 Cizge(int n) {
2   this.n = n;
3   for (int i = 0; i < n; i++)
4     komsu.add(new ArrayList<>());
5 }
6 void kenarEkle(int u, int v) {
7   ▶ komsu.get(u).add(v);
8   komsu.get(v).add(u); // yönlüde yok
9 }
10 // main:
11 Cizge g = new Cizge(5);
12 int[][] E = {{0,1}, {0,4}, {1,2},
13              {1,4}, {2,3}, {3,4}};
14 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```

ÇAĞRI YİĞİNİ

kenarEkle(0, 4)

main()



Kenar 0–4: **4**, 0'ın listesinin sonuna eklenir.

Adım 9 / 19

DEĞİŞKENLER

u = 0

v = 4

Kod İzleme: Komşuluk Listesi Kurma

```

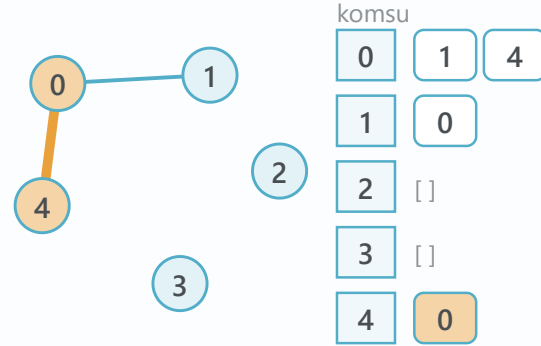
1 Cizge(int n) {
2   this.n = n;
3   for (int i = 0; i < n; i++)
4     komsu.add(new ArrayList<>());
5 }
6 void kenarEkle(int u, int v) {
7   komsu.get(u).add(v);
8   komsu.get(v).add(u); // yönlüde yok
9 }
10 // main:
11 Cizge g = new Cizge(5);
12 int[][] E = {{0,1}, {0,4}, {1,2},
13              {1,4}, {2,3}, {3,4}};
14 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```

ÇAĞRI YIĞINI

kenarEkle(0, 4)

main()



Yönsüz çizge: aynı kenar ters yönde de eklenir → 0, 4'ün listesine.

Adım 10 / 19

DEĞİŞKENLER

u = 0

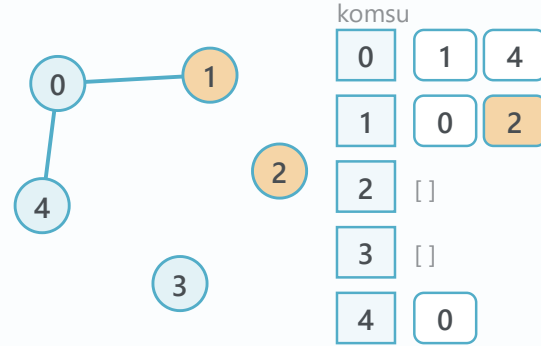
v = 4

Kod İzleme: Komşuluk Listesi Kurma

```

1 Cizge(int n) {
2   this.n = n;
3   for (int i = 0; i < n; i++)
4     komsu.add(new ArrayList<>());
5 }
6 void kenarEkle(int u, int v) {
7   ▶ komsu.get(u).add(v);
8   komsu.get(v).add(u); // yönlüde yok
9 }
10 // main:
11 Cizge g = new Cizge(5);
12 int[][] E = {{0,1}, {0,4}, {1,2},
13             {1,4}, {2,3}, {3,4}};
14 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```



Kenar 1-2: **2**, 1'in listesinin sonuna eklenir.

Adım 11 / 19

DEĞİŞKENLER

u = 1

v = 2

ÇAĞRI YIĞINI

kenarEkle(1, 2)

main()

Kod İzleme: Komşuluk Listesi Kurma

```

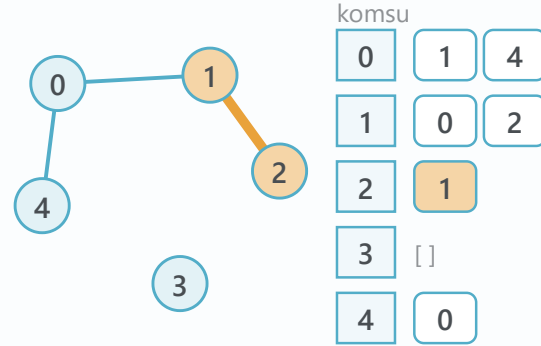
1 Cizge(int n) {
2   this.n = n;
3   for (int i = 0; i < n; i++)
4     komsu.add(new ArrayList<>());
5 }
6 void kenarEkle(int u, int v) {
7   komsu.get(u).add(v);
8   komsu.get(v).add(u); // yönlüde yok
9 }
10 // main:
11 Cizge g = new Cizge(5);
12 int[][] E = {{0,1}, {0,4}, {1,2},
13              {1,4}, {2,3}, {3,4}};
14 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```

ÇAĞRI YIĞINI

kenarEkle(1, 2)

main()



Yönsüz çizge: aynı kenar ters yönde de eklenir → 1, 2'nin listesine.

Adım 12 / 19

DEĞİŞKENLER

u = 1

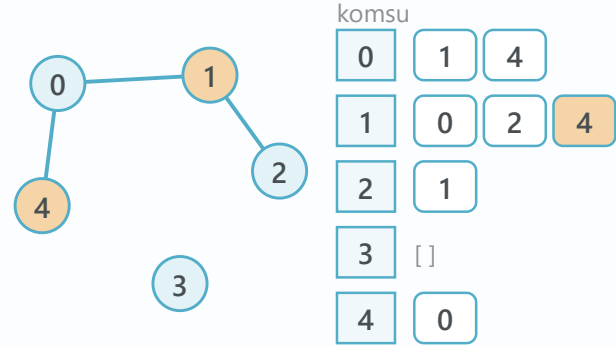
v = 2

Kod İzleme: Komşuluk Listesi Kurma

```

1 Cizge(int n) {
2   this.n = n;
3   for (int i = 0; i < n; i++)
4     komsu.add(new ArrayList<>());
5 }
6 void kenarEkle(int u, int v) {
7   ▶ komsu.get(u).add(v);
8   komsu.get(v).add(u); // yönlüde yok
9 }
10 // main:
11 Cizge g = new Cizge(5);
12 int[][] E = {{0,1}, {0,4}, {1,2},
13              {1,4}, {2,3}, {3,4}};
14 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```



Kenar 1–4: **4**, 1'in listesinin sonuna eklenir.

Adım 13 / 19

ÇAĞRI YIĞINI

kenarEkle(1, 4)

main()

DEĞİŞKENLER

u = 1

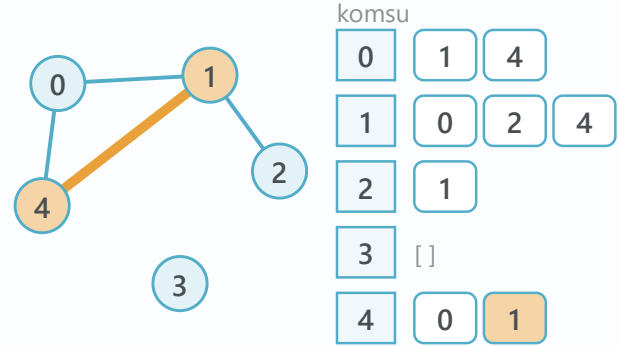
v = 4

Kod İzleme: Komşuluk Listesi Kurma

```

1 Cizge(int n) {
2   this.n = n;
3   for (int i = 0; i < n; i++)
4     komsu.add(new ArrayList<>());
5 }
6 void kenarEkle(int u, int v) {
7   komsu.get(u).add(v);
8   komsu.get(v).add(u); // yönlüde yok
9 }
10 // main:
11 Cizge g = new Cizge(5);
12 int[][] E = {{0,1}, {0,4}, {1,2},
13             {1,4}, {2,3}, {3,4}};
14 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```



Yönsüz çizge: aynı kenar ters yönde de eklenir → **1**, 4'ün listesine.

Adım 14 / 19

DEĞİŞKENLER

u = 1

v = 4

ÇAĞRI YIĞINI

kenarEkle(1, 4)

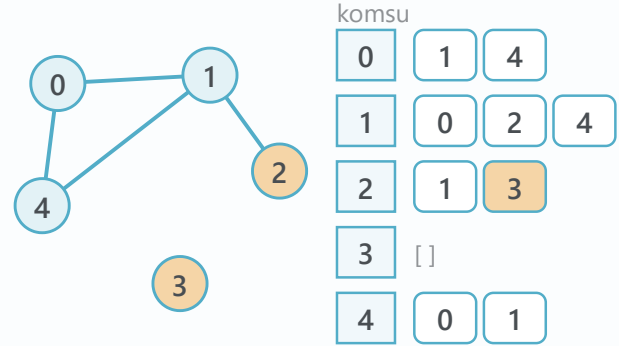
main()

Kod İzleme: Komşuluk Listesi Kurma

```

1 Cizge(int n) {
2   this.n = n;
3   for (int i = 0; i < n; i++)
4     komsu.add(new ArrayList<>());
5 }
6 void kenarEkle(int u, int v) {
7   ▶ komsu.get(u).add(v);
8   komsu.get(v).add(u); // yönlüde yok
9 }
10 // main:
11 Cizge g = new Cizge(5);
12 int[][] E = {{0,1}, {0,4}, {1,2},
13              {1,4}, {2,3}, {3,4}};
14 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```



Kenar 2–3: **3**, 2'nin listesinin sonuna eklenir.

Adım 15 / 19

ÇAĞRI YIĞINI

kenarEkle(2, 3)

main()

DEĞİŞKENLER

u = 2

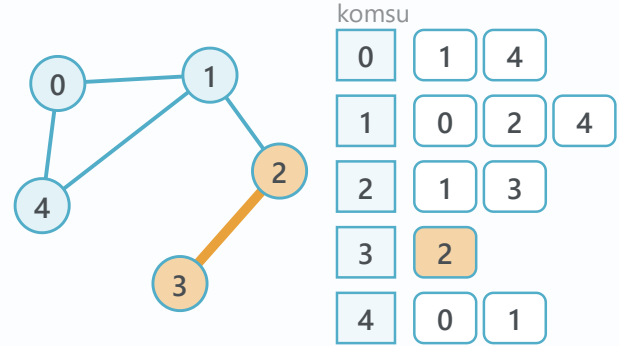
v = 3

Kod İzleme: Komşuluk Listesi Kurma

```

1 Cizge(int n) {
2   this.n = n;
3   for (int i = 0; i < n; i++)
4     komsu.add(new ArrayList<>());
5 }
6 void kenarEkle(int u, int v) {
7   komsu.get(u).add(v);
8   komsu.get(v).add(u); // yönlüde yok
9 }
10 // main:
11 Cizge g = new Cizge(5);
12 int[][] E = {{0,1}, {0,4}, {1,2},
13              {1,4}, {2,3}, {3,4}};
14 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```



Yönsüz çizge: aynı kenar ters yönde de eklenir → 2, 3'ün listesine.

Adım 16 / 19

DEĞİŞKENLER

u = 2

v = 3

ÇAĞRI YIĞINI

kenarEkle(2, 3)

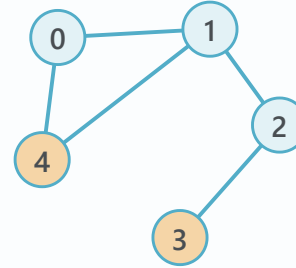
main()

Kod İzleme: Komşuluk Listesi Kurma

```

1 Cizge(int n) {
2   this.n = n;
3   for (int i = 0; i < n; i++)
4     komsu.add(new ArrayList<>());
5 }
6 void kenarEkle(int u, int v) {
7   ▶ komsu.get(u).add(v);
8   komsu.get(v).add(u); // yönlüde yok
9 }
10 // main:
11 Cizge g = new Cizge(5);
12 int[][] E = {{0,1}, {0,4}, {1,2},
13              {1,4}, {2,3}, {3,4}};
14 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```



komsu

0	1	4	
1	0	2	4
2	1	3	
3	2	4	
4	0	1	

Kenar 3–4: **4**, 3'ün listesinin sonuna eklenir.

Adım 17 / 19

DEĞİŞKENLER

u = 3

v = 4

ÇAĞRI YIĞINI

kenarEkle(3, 4)

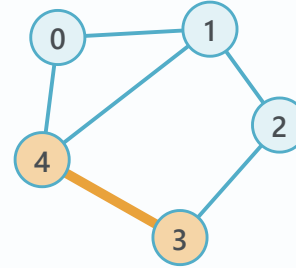
main()

Kod İzleme: Komşuluk Listesi Kurma

```

1 Cizge(int n) {
2   this.n = n;
3   for (int i = 0; i < n; i++)
4     komsu.add(new ArrayList<>());
5 }
6 void kenarEkle(int u, int v) {
7   komsu.get(u).add(v);
8   komsu.get(v).add(u); // yönlüde yok
9 }
10 // main:
11 Cizge g = new Cizge(5);
12 int[][] E = {{0,1}, {0,4}, {1,2},
13              {1,4}, {2,3}, {3,4}};
14 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```



komsu

0	1	4	
1	0	2	4
2	1	3	
3	2	4	
4	0	1	3

Yönsüz çizge: aynı kenar ters yönde de eklenir → **3**, 4'ün listesine.

Adım 18 / 19

DEĞİŞKENLER

u = 3

v = 4

ÇAĞRI YIĞINI

kenarEkle(3, 4)

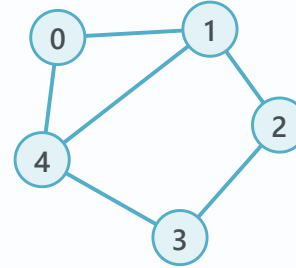
main()

Kod İzleme: Komşuluk Listesi Kurma

```

1 Cizge(int n) {
2     this.n = n;
3     for (int i = 0; i < n; i++)
4         komsu.add(new ArrayList<>());
5 }
6 void kenarEkle(int u, int v) {
7     komsu.get(u).add(v);
8     komsu.get(v).add(u); // yönlüde yok
9 }
10 // main:
11 Cizge g = new Cizge(5);
12 int[][] E = {{0,1}, {0,4}, {1,2},
13              {1,4}, {2,3}, {3,4}};
14 ▶ for (int[] e : E) g.kenarEkle(e[0], e[1]);

```



komsu

0	1	4	
1	0	2	4
2	1	3	
3	2	4	
4	0	1	3

Bitti: 6 kenar → listelerde **2•6 = 12** kayıt. Her **kenarEkle** $O(1)$; bellek $O(n + m)$. $derece(1) =$ liste uzunluğu = 3.

Adım 19 / 19

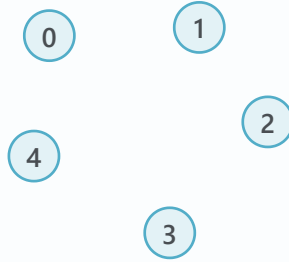
ÇAĞRI YİĞİNİ

main()

DEĞİŞKENLER

—

Kod İzleme: Komşuluk Matrisi Kurma



	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0

$m[\text{satır}][\text{sütun}]$ (1 = true)

5×5 **boolean** matris oluşturulur; Java tüm hücreleri **false** (0) ile başlatır. Bellek $O(n^2) = 25$ hücre.

Adım 1 / 14

```

▶ boolean[][] m = new boolean[n][n];
2 void kenarEkle(int u, int v) {
3   m[u][v] = true;
4   m[v][u] = true; // yönlü çizgede yok
5 }
6 // main: aynı 6 kenar (n = 5)
7 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```

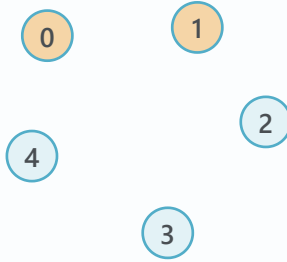
DEĞİŞKENLER

$n = 5$

ÇAĞRI YIĞINI

main()

Kod İzleme: Komşuluk Matrisi Kurma



	0	1	2	3	4
0	0	1	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0

m[satır][sütun] (1 = true)

Kenar 0–1: **m[0][1] = true** (satır 0, sütun 1).

Adım 2 / 14

```
1 boolean[][] m = new boolean[n][n];
2 void kenarEkle(int u, int v) {
3     m[u][v] = true;
4     m[v][u] = true; // yönlü çizgede yok
5 }
6 // main: aynı 6 kenar (n = 5)
7 for (int[] e : E) g.kenarEkle(e[0], e[1]);
```

DEĞİŞKENLER

u = 0

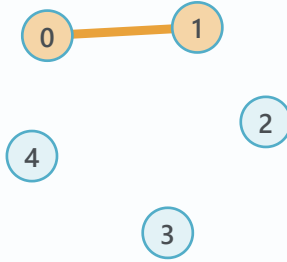
v = 1

ÇAĞRI YIĞINI

kenarEkle(0, 1)

main()

Kod İzleme: Komşuluk Matrisi Kurma



	0	1	2	3	4
0	0	1	0	0	0
1	1	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0

$m[\text{satur}][\text{sütun}]$ (1 = true)

Simetrik hücre: **$m[1][0] = \text{true}$** . Yönsüz çizgenin matrisi köşegene göre simetriktir.

Adım 3 / 14

```

1 boolean[][] m = new boolean[n][n];
2 void kenarEkle(int u, int v) {
3   m[u][v] = true;
4   ▶ m[v][u] = true; // yönlü çizgede yok
5 }
6 // main: aynı 6 kenar (n = 5)
7 for (int[] e : E) g.kenarEkle(e[0], e[1]);
  
```

DEĞİŞKENLER

$u = 0$

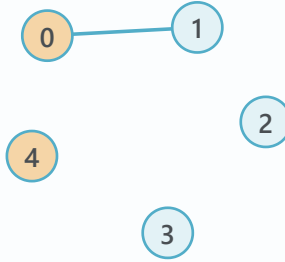
$v = 1$

ÇAĞRI YIĞINI

kenarEkle(0, 1)

main()

Kod İzleme: Komşuluk Matrisi Kurma



	0	1	2	3	4
0	0	1	0	0	1
1	1	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0

$m[\text{satır}][\text{sütun}]$ (1 = true)

Kenar 0–4: **$m[0][4] = \text{true}$** (satır 0, sütun 4).

Adım 4 / 14

```

1 boolean[][] m = new boolean[n][n];
2 void kenarEkle(int u, int v) {
3     ▶ m[u][v] = true;
4     m[v][u] = true; // yönlü çizgede yok
5 }
6 // main: aynı 6 kenar (n = 5)
7 for (int[] e : E) g.kenarEkle(e[0], e[1]);
  
```

DEĞİŞKENLER

$u = 0$

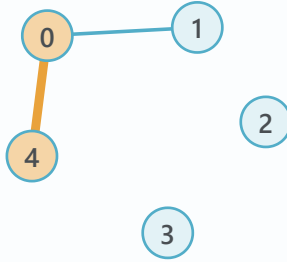
$v = 4$

ÇAĞRI YIĞINI

kenarEkle(0, 4)

main()

Kod İzleme: Komşuluk Matrisi Kurma



	0	1	2	3	4
0	0	1	0	0	1
1	1	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	1	0	0	0	0

$m[\text{satır}][\text{sütun}]$ (1 = true)

Simetrik hücre: **$m[4][0] = \text{true}$** . Yönsüz çizgenin matrisi köşegene göre simetriktir.

Adım 5 / 14

```

1 boolean[][] m = new boolean[n][n];
2 void kenarEkle(int u, int v) {
3     m[u][v] = true;
4     ► m[v][u] = true; // yönlü çizgede yok
5 }
6 // main: aynı 6 kenar (n = 5)
7 for (int[] e : E) g.kenarEkle(e[0], e[1]);
  
```

DEĞİŞKENLER

$u = 0$

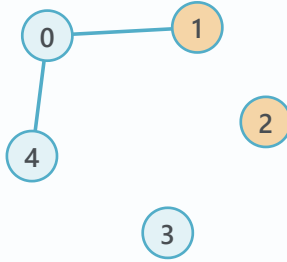
$v = 4$

ÇAĞRI YIĞINI

kenarEkle(0, 4)

main()

Kod İzleme: Komşuluk Matrisi Kurma



	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	1	0	0	0	0

$m[\text{satır}][\text{sütun}]$ (1 = true)

Kenar 1-2: **$m[1][2] = \text{true}$** (sıra 1, sütun 2).

Adım 6 / 14

```

1 boolean[][] m = new boolean[n][n];
2 void kenarEkle(int u, int v) {
3     m[u][v] = true;
4     m[v][u] = true; // yönlü çizgede yok
5 }
6 // main: aynı 6 kenar (n = 5)
7 for (int[] e : E) g.kenarEkle(e[0], e[1]);
  
```

DEĞİŞKENLER

u = 1

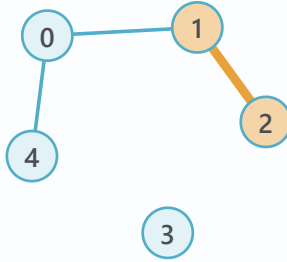
v = 2

ÇAĞRI YIĞINI

kenarEkle(1, 2)

main()

Kod İzleme: Komşuluk Matrisi Kurma



	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	0	0
2	0	1	0	0	0
3	0	0	0	0	0
4	1	0	0	0	0

$m[\text{satur}][\text{sütun}]$ (1 = true)

Simetrik hücre: **$m[2][1] = \text{true}$** . Yönsüz çizgenin matrisi köşegene göre simetriktir.

Adım 7 / 14

```

1 boolean[][] m = new boolean[n][n];
2 void kenarEkle(int u, int v) {
3     m[u][v] = true;
4     m[v][u] = true; // yönlü çizgede yok
5 }
6 // main: aynı 6 kenar (n = 5)
7 for (int[] e : E) g.kenarEkle(e[0], e[1]);
  
```

DEĞİŞKENLER

u = 1

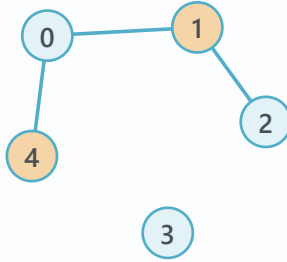
v = 2

ÇAĞRI YIĞINI

kenarEkle(1, 2)

main()

Kod İzleme: Komşuluk Matrisi Kurma



	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	0	1
2	0	1	0	0	0
3	0	0	0	0	0
4	1	0	0	0	0

$m[\text{satır}][\text{sütun}]$ (1 = true)

Kenar 1–4: $m[1][4] = \text{true}$ (sıra 1, sütun 4).

Adım 8 / 14

```
1 boolean[][] m = new boolean[n][n];
2 void kenarEkle(int u, int v) {
3     m[u][v] = true;
4     m[v][u] = true; // yönlü çizgede yok
5 }
6 // main: aynı 6 kenar (n = 5)
7 for (int[] e : E) g.kenarEkle(e[0], e[1]);
```

DEĞİŞKENLER

$u = 1$

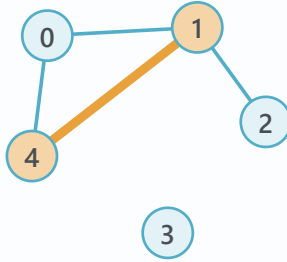
$v = 4$

ÇAĞRI YIĞINI

`kenarEkle(1, 4)`

`main()`

Kod İzleme: Komşuluk Matrisi Kurma



	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	0	1
2	0	1	0	0	0
3	0	0	0	0	0
4	1	1	0	0	0

$m[\text{satur}][\text{sütun}]$ (1 = true)

Simetrik hücre: **$m[4][1] = \text{true}$** . Yönsüz çizgenin matrisi köşegene göre simetriktir.

Adım 9 / 14

```

1 boolean[][] m = new boolean[n][n];
2 void kenarEkle(int u, int v) {
3     m[u][v] = true;
4     m[v][u] = true; // yönlü çizgede yok
5 }
6 // main: aynı 6 kenar (n = 5)
7 for (int[] e : E) g.kenarEkle(e[0], e[1]);
  
```

DEĞİŞKENLER

u = 1

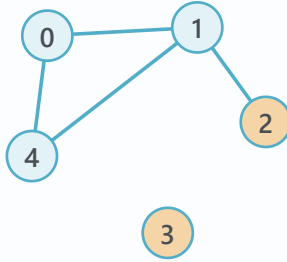
v = 4

ÇAĞRI YIĞINI

kenarEkle(1, 4)

main()

Kod İzleme: Komşuluk Matrisi Kurma



	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	0	1
2	0	1	0	1	0
3	0	0	0	0	0
4	1	1	0	0	0

$m[\text{satır}][\text{sütun}]$ (1 = true)

Kenar 2-3: **$m[2][3] = \text{true}$** (satır 2, sütun 3).

Adım 10 / 14

```

1 boolean[][] m = new boolean[n][n];
2 void kenarEkle(int u, int v) {
3     ▶ m[u][v] = true;
4     m[v][u] = true; // yönlü çizgede yok
5 }
6 // main: aynı 6 kenar (n = 5)
7 for (int[] e : E) g.kenarEkle(e[0], e[1]);
  
```

DEĞİŞKENLER

u = 2

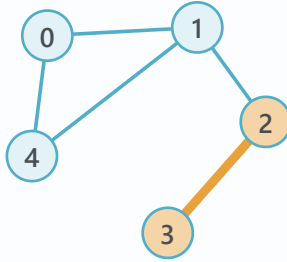
v = 3

ÇAĞRI YIĞINI

kenarEkle(2, 3)

main()

Kod İzleme: Komşuluk Matrisi Kurma



	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	0	1
2	0	1	0	1	0
3	0	0	1	0	0
4	1	1	0	0	0

$m[\text{satur}][\text{sütun}]$ (1 = true)

Simetrik hücre: **$m[3][2] = \text{true}$** . Yönsüz çizgenin matrisi köşegene göre simetriktir.

Adım 11 / 14

```

1 boolean[][] m = new boolean[n][n];
2 void kenarEkle(int u, int v) {
3     m[u][v] = true;
4     m[v][u] = true; // yönlü çizgede yok
5 }
6 // main: aynı 6 kenar (n = 5)
7 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```

DEĞİŞKENLER

u = 2

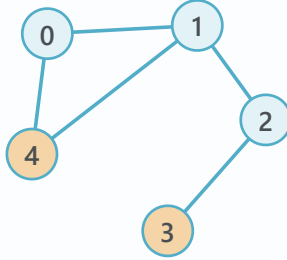
v = 3

ÇAĞRI YIĞINI

kenarEkle(2, 3)

main()

Kod İzleme: Komşuluk Matrisi Kurma



	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	0	1
2	0	1	0	1	0
3	0	0	1	0	1
4	1	1	0	0	0

$m[\text{satır}][\text{sütun}]$ (1 = true)

Kenar 3–4: $m[3][4] = \text{true}$ (satır 3, sütun 4).

Adım 12 / 14

```

1 boolean[][] m = new boolean[n][n];
2 void kenarEkle(int u, int v) {
3     m[u][v] = true;
4     m[v][u] = true; // yönlü çizgede yok
5 }
6 // main: aynı 6 kenar (n = 5)
7 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```

DEĞİŞKENLER

$u = 3$

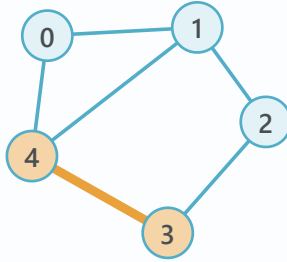
$v = 4$

ÇAĞRI YIĞINI

kenarEkle(3, 4)

main()

Kod İzleme: Komşuluk Matrisi Kurma



	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	0	1
2	0	1	0	1	0
3	0	0	1	0	1
4	1	1	0	1	0

$m[\text{satur}][\text{sutun}]$ (1 = true)

Simetrik hücre: **$m[4][3] = \text{true}$** . Yönsüz çizgenin matrisi köşegene göre simetriktir.

Adım 13 / 14

```

1 boolean[][] m = new boolean[n][n];
2 void kenarEkle(int u, int v) {
3     m[u][v] = true;
4     ▶ m[v][u] = true; // yönlü çizgede yok
5 }
6 // main: aynı 6 kenar (n = 5)
7 for (int[] e : E) g.kenarEkle(e[0], e[1]);

```

DEĞİŞKENLER

$u = 3$

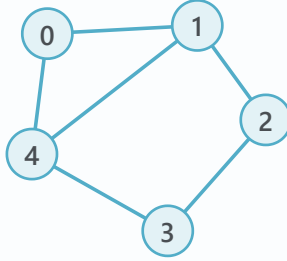
$v = 4$

ÇAĞRI YIĞINI

kenarEkle(3, 4)

main()

Kod İzleme: Komşuluk Matrisi Kurma



	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	0	1
2	0	1	0	1	0
3	0	0	1	0	1
4	1	1	0	1	0

$m[\text{satır}][\text{sütun}]$ (1 = true)

Bitti: 12 hücre true, 13'ü boşa false. "2-4 kenarı var mı?" → $m[2][4]$ tek erişim, **O(1)**; ama 2'nin komşularını bulmak için tüm satır taranır: $O(n)$.

Adım 14 / 14

```

1 boolean[][] m = new boolean[n][n];
2 void kenarEkle(int u, int v) {
3     m[u][v] = true;
4     m[v][u] = true; // yönlü çizgede yok
5 }
6 // main: aynı 6 kenar (n = 5)
▶ for (int[] e : E) g.kenarEkle(e[0], e[1]);

```

DEĞİŞKENLER

—

ÇAĞRI YİĞİNİ

main()

Gezinme Algoritmaları

Gezinme (traversal): bir başlangıç düğümünden ulaşılabilen **tüm düğümleri, her birini bir kez** sistematik olarak ziyaret etmek.

Genişlik Öncelikli Arama (BFS)

Breadth-First Search. Önce tüm yakın komşular, sonra onların komşuları... Seviye seviye ilerler.

yardımcı yapı: kuyruk (FIFO)

Derinlik Öncelikli Arama (DFS)

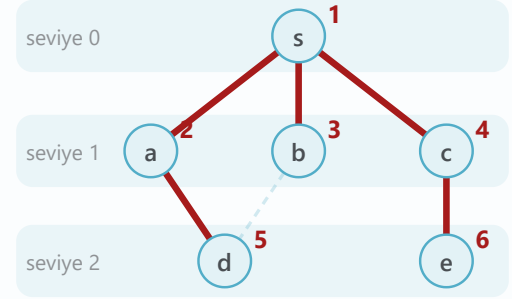
Depth-First Search. Bir yol boyunca gidebildiği kadar derine iner, çıkmazda geri döner.

yardımcı yapı: yığın (LIFO) / özyineleme

Ağaçtan farkı: çizgede döngü olabilir, aynı düğüme birden çok yoldan gelinebilir → **ziyaret edildi** işareti (ör. `boolean[] ziyaret`) şarttır; yoksa algoritma sonsuza kadar döner.

Genişlik Öncelikli Arama (BFS)

- Döğümleri **seviye seviye** keşfeder: önce başlangıca 1 kenar uzaklıktakiler, sonra 2 kenar uzaklıktakiler...
- **Kuyruk** yapısını kullanır: ilk keşfedilen ilk işlenir.
- Her döğüm yalnızca bir kez ziyaret edilir.



Numara = ziyaret sırası; kalın = BFS ağacı

Artıları

- + Ağırlıksız çizgede en kısa yolu (en az kenar) bulur.
- + Yakın döğümleri hızlı keşfeder.

Eksisi

- Bellek kullanımı yüksek olabilir: bir seviyenin tamamı kuyrukta bekler.

Örnek: sosyal ağda "arkadaşımın arkadaşları" (2. seviye), bir web tarayıcının (crawler) siteyi katman katman gezmesi.

BFS Algoritması

Sözde kod

```
BFS(başlangıç):  
    kuyruk.ekle(başlangıç)  
    başlangıç.ziyaretEdildi()  
    while kuyruk boş değil:  
        w = kuyruk.çıkar()  
        for w'nin ziyaret edilmemiş  
            her komşusu u:  
                u.ziyaretEdildi()  
                kuyruk.ekle(u)
```

Düğüm kuyruğa **girerken** işaretlenir; böylece aynı düğüm kuyruğa iki kez girmez.

Java

```
void bfs(int s) {  
    boolean[] ziyaret = new boolean[n];  
    Queue<Integer> q = new ArrayDeque<>();  
    ziyaret[s] = true; q.add(s);  
    while (!q.isEmpty()) {  
        int w = q.poll();  
        System.out.print(w + " ");  
        for (int u : komsu.get(w))  
            if (!ziyaret[u]) {  
                ziyaret[u] = true;  
                q.add(u);  
            }  
    }  
}
```

Kod İzleme: bfs(A)

```

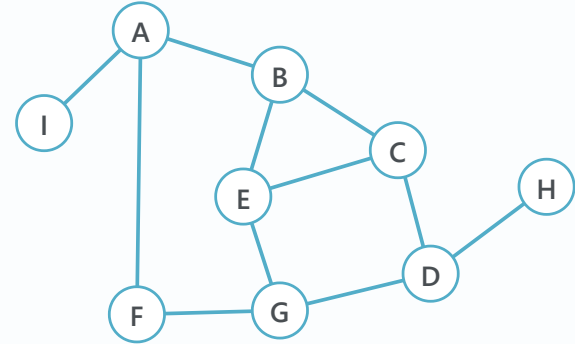
1 void bfs(int s) {
2     boolean[] ziyaret = new boolean[n];
3     Queue<Integer> q = new ArrayDeque<>();
4     ziyaret[s] = true; q.add(s);
5     while (!q.isEmpty()) {
6         int w = q.poll();
7         System.out.print(w + " ");
8         for (int u : komsu.get(w))
9             if (!ziyaret[u]) {
10                 ziyaret[u] = true;
11                 q.add(u);
12             }
13     }
14 }

```

DEĞİŞKENLER

s = A

ÇIKTI



q (ön → arka)

(boş)

--	--	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
F	F	F	F	F	F	F	F	F

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

s = A (A=0, B=1, ..., I=8; komşular alfabetik).

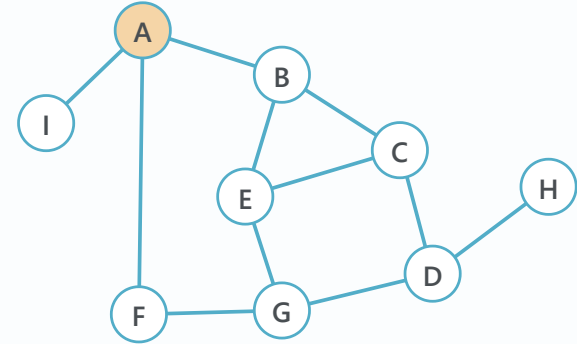
ziyaret tümü false, kuyruk boş.

Kod İzleme: bfs(A)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ► ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     int w = q.poll();
7     System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

```



q (ön → arka)



ziyaret[]

A	B	C	D	E	F	G	H	I
T	F	F	F	F	F	F	F	F

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

A kuyruğa girerken işaretlenir: **ziyaret[A] = true** , q = [A].

DEĞİŞKENLER

s = A

ÇIKTI

Adım 2 / 15

Kod İzleme: bfs(A)

```

1 void bfs(int s) {
2     boolean[] ziyaret = new boolean[n];
3     Queue<Integer> q = new ArrayDeque<>();
4     ziyaret[s] = true; q.add(s);
5     while (!q.isEmpty()) {
6         ▶ int w = q.poll();
7         ▶ System.out.print(w + " ");
8         for (int u : komsu.get(w))
9             if (!ziyaret[u]) {
10                 ziyaret[u] = true;
11                 q.add(u);
12             }
13     }
14 }

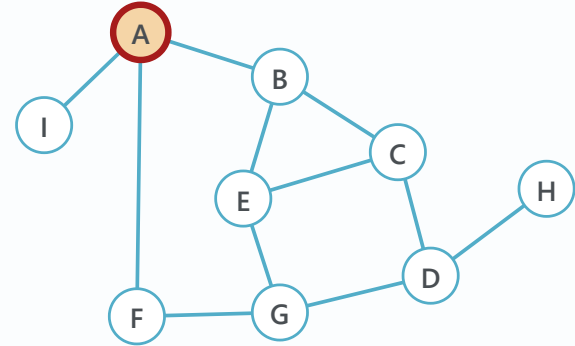
```

DEĞİŞKENLER

w = A

ÇIKTI

A



q (ön → arka)

(boş)

--	--	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	F	F	F	F	F	F	F	F

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

Kuyruğun önündeki **A** çıkarılır (poll) ve yazdırılır. Şimdi komşuları B, F, I taranır.

Kod İzleme: bfs(A)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     int w = q.poll();
7     System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

```

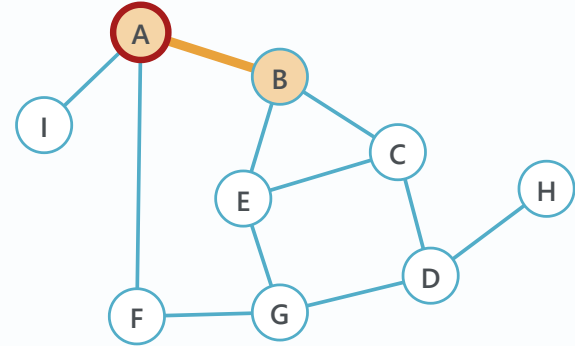
DEĞİŞKENLER

w = A

u = B

ÇIKTI

A



q (ön → arka)



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	F	F	F	F	F	F	F

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

u = B ziyaret edilmemiş → işaretlenir ve kuyruğun arkasına eklenir (seviye 1).

Kod İzleme: bfs(A)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     int w = q.poll();
7     System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

```

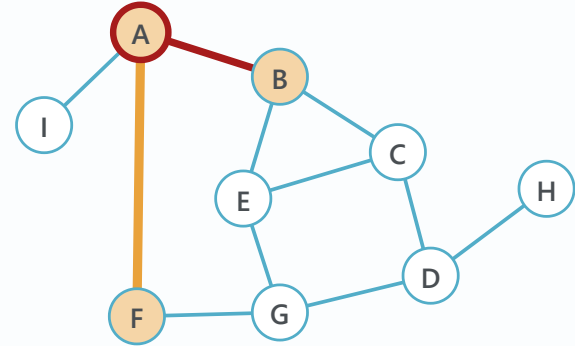
DEĞİŞKENLER

w = A

u = F

ÇIKTI

A



q (ön → arka)

B	F							
---	---	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	F	F	F	T	F	F	F

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

u = F ziyaret edilmemiş → işaretlenir ve kuyruğun arkasına eklenir (seviye 1).

Kod İzleme: bfs(A)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     int w = q.poll();
7     System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

```

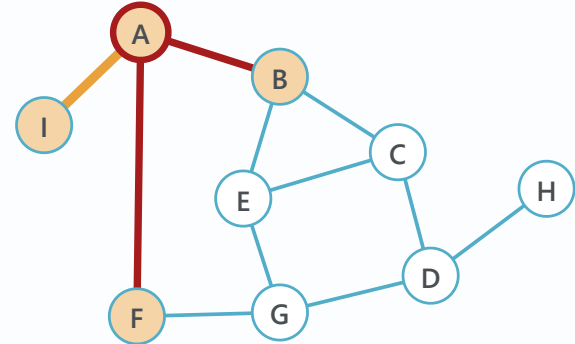
DEĞİŞKENLER

w = A

u = I

ÇIKTI

A



q (ön → arka)

B	F	I					
---	---	---	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	F	F	F	T	F	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

u = I ziyaret edilmemiş → işaretlenir ve kuyruğun arkasına eklenir (seviye 1).

Kod İzleme: bfs(A)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     ▶ int w = q.poll();
7     ▶ System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

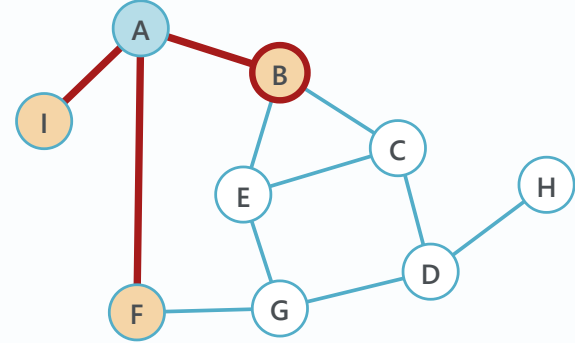
```

DEĞİŞKENLER

w = B

ÇIKTI

A B



q (ön → arka)

F	I							
---	---	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	F	F	F	T	F	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

Kuyruğun önündeki **B** çıkarılır (poll) ve yazdırılır. Şimdi komşuları A, C, E taranır.

Kod İzleme: bfs(A)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     int w = q.poll();
7     System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

```

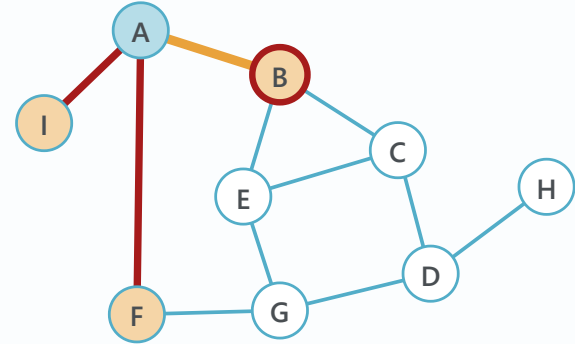
DEĞİŞKENLER

w = B

u = A

ÇIKTI

A B



q (ön → arka)

F	I							
---	---	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	F	F	F	T	F	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

u = A: zaten ziyaret edilmiş → atlanır. Ziyaret işareti olmasa kuyruğa tekrar girerdi.

Kod İzleme: bfs(A)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     int w = q.poll();
7     System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

```

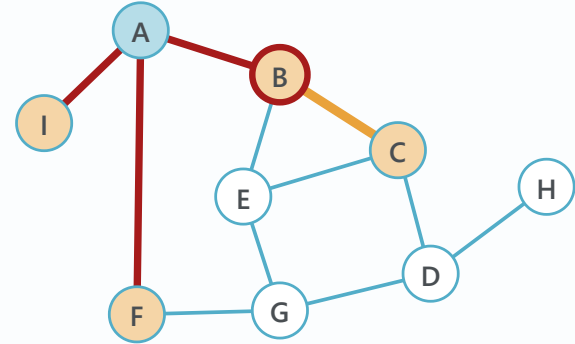
DEĞİŞKENLER

w = B

u = C

ÇIKTI

A B



q (ön → arka)

F	I	C						
---	---	---	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	F	F	T	F	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

u = C ziyaret edilmemiş → işaretlenir ve kuyruğun arkasına eklenir (seviye 2).

Kod İzleme: bfs(A)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     int w = q.poll();
7     System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

```

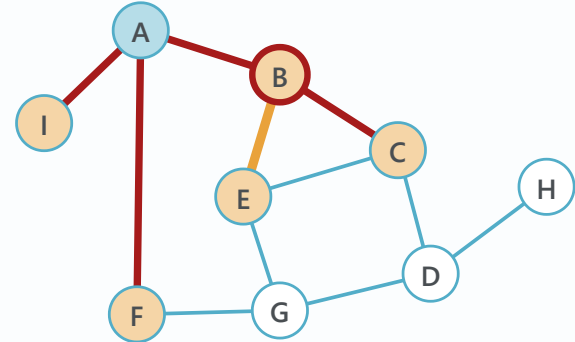
DEĞİŞKENLER

w = B

u = E

ÇIKTI

A B



q (ön → arka)

F	I	C	E				
---	---	---	---	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	F	T	T	F	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

u = E ziyaret edilmemiş → işaretlenir ve kuyruğun arkasına eklenir (seviye 2).

Kod İzleme: bfs(A)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     ▶ int w = q.poll();
7     ▶ System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

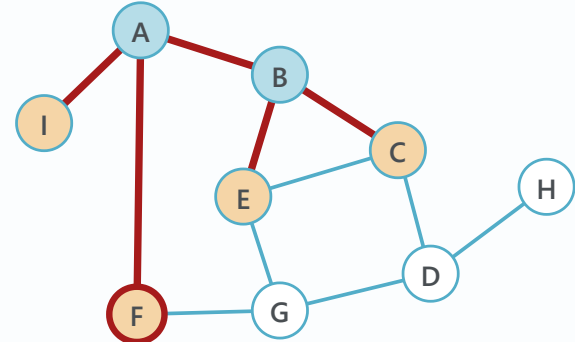
```

DEĞİŞKENLER

w = F

ÇIKTI

A B F



q (ön → arka)

I	C	E						
---	---	---	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	F	T	T	F	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

Kuyruğun önündeki **F** çıkarılır (poll) ve yazdırılır.
Şimdi komşuları A, G taranır.

Adım 11 / 15

Kod İzleme: bfs(A)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     int w = q.poll();
7     System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

```

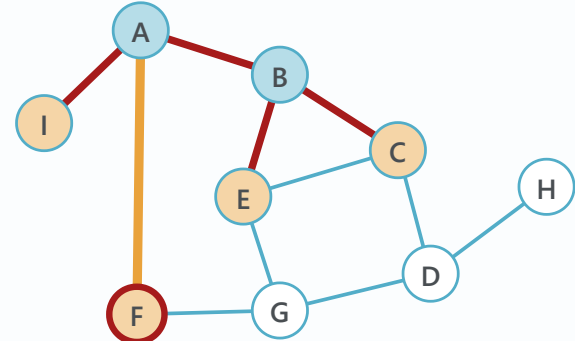
DEĞİŞKENLER

w = F

u = A

ÇIKTI

A B F



q (ön → arka)

I	C	E						
---	---	---	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	F	T	T	F	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

u = A: zaten ziyaret edilmiş → atlanır.

Kod İzleme: bfs(A)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     int w = q.poll();
7     System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

```

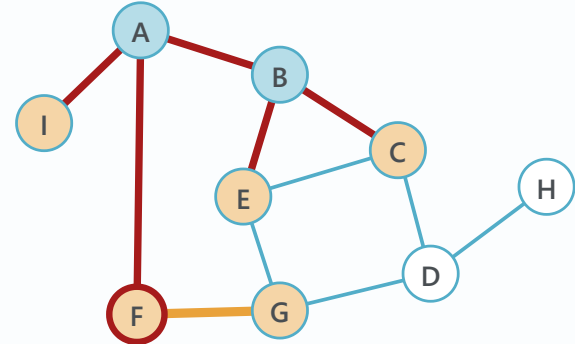
DEĞİŞKENLER

w = F

u = G

ÇIKTI

A B F



q (ön → arka)

I	C	E	G				
---	---	---	---	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	F	T	T	T	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

u = G ziyaret edilmemiş → işaretlenir ve kuyruğun arkasına eklenir (seviye 2).

Kod İzleme: bfs(A)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     ▶ int w = q.poll();
7     ▶ System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

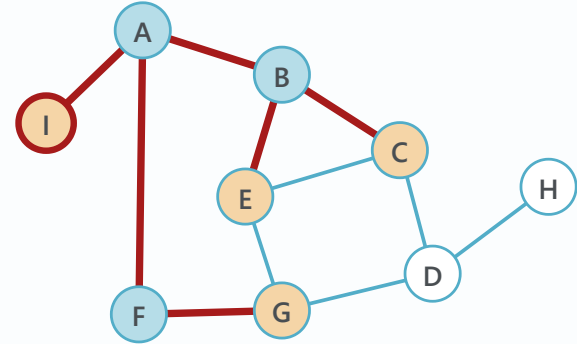
```

DEĞİŞKENLER

w = I

ÇIKTI

A B F I



q (ön → arka)

C	E	G						
---	---	---	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	F	T	T	T	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

Kuyruğun önündeki **I** çıkarılır (poll) ve yazdırılır.
Şimdi komşusu A taranır.

Kod İzleme: bfs(A)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     int w = q.poll();
7     System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

```

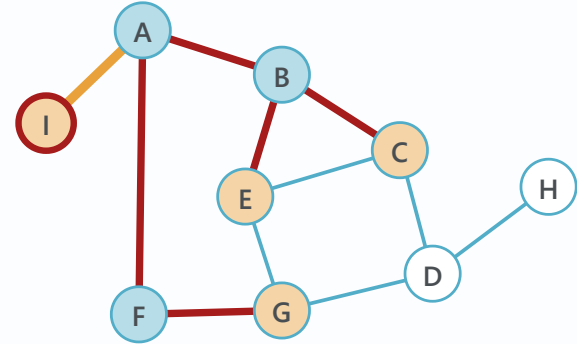
DEĞİŞKENLER

w = I

u = A

ÇIKTI

A B F I



q (ön → arka)

C	E	G						
---	---	---	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	F	T	T	T	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

u = A: zaten ziyaret edilmiş → atlanır.

Kod İzleme: bfs(A) (devam)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     ▶ int w = q.poll();
7     ▶ System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

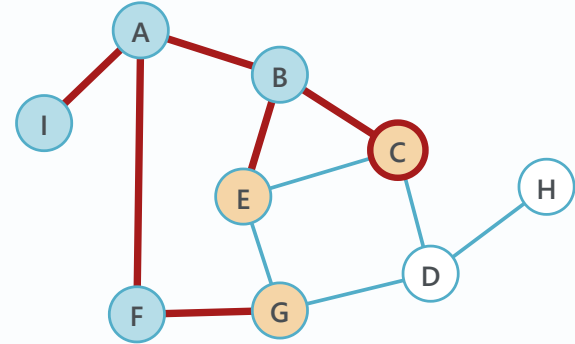
```

DEĞİŞKENLER

w = C

ÇIKTI

A B F I C



q (ön → arka)

E	G							
---	---	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	F	T	T	T	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

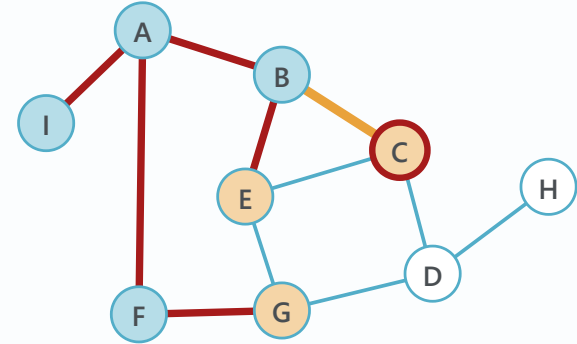
Kuyruğun önündeki **C** çıkarılır (poll) ve yazdırılır. Şimdi komşuları B, D, E taranır.

Kod İzleme: bfs(A) (devam)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     int w = q.poll();
7     System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

```



q (ön → arka)

E	G							
---	---	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	F	T	T	T	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

u = B: zaten ziyaret edilmiş → atlanır.

DEĞİŞKENLER

w = C

u = B

ÇIKTI

A B F I C

Kod İzleme: bfs(A) (devam)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     int w = q.poll();
7     System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

```

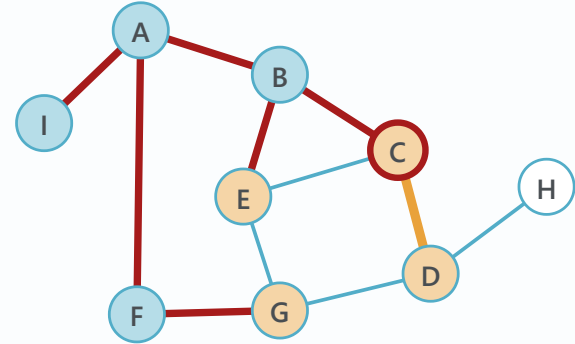
DEĞİŞKENLER

w = C

u = D

ÇIKTI

A B F I C



q (ön → arka)

E	G	D						
---	---	---	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

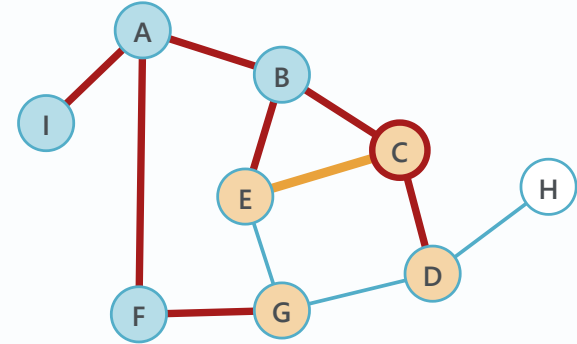
u = D ziyaret edilmemiş → işaretlenir ve kuyruğun arkasına eklenir (seviye 3).

Kod İzleme: bfs(A) (devam)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     int w = q.poll();
7     System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

```



q (ön → arka)

E	G	D						
---	---	---	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

u = E: zaten ziyaret edilmiş → atlanır.

DEĞİŞKENLER

w = C

u = E

ÇIKTI

A B F I C

Kod İzleme: bfs(A) (devam)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     ▶ int w = q.poll();
7     ▶ System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

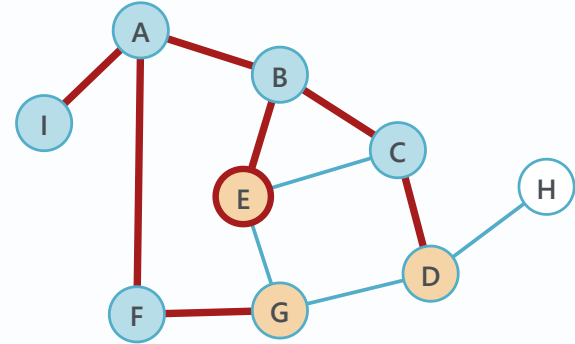
```

DEĞİŞKENLER

w = E

ÇIKTI

A B F I C E



q (ön → arka)

G	D							
---	---	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

Kuyruğun önündeki **E** çıkarılır (poll) ve yazdırılır. Şimdi komşuları B, C, G taranır.

Kod İzleme: bfs(A) (devam)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     int w = q.poll();
7     System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       ▶ if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

```

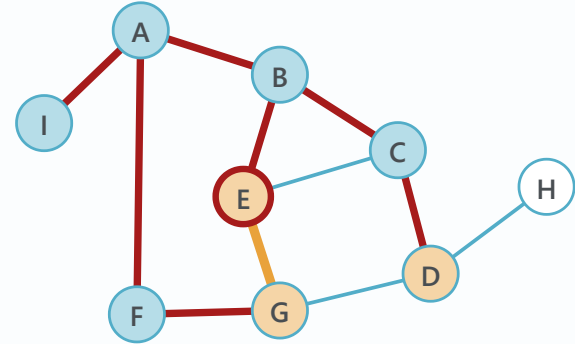
DEĞİŞKENLER

w = E

u = G

ÇIKTI

A B F I C E



q (ön → arka)

G	D							
---	---	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

u = B, C, G: hepsi zaten ziyaret edilmiş → atlanır.

Kod İzleme: bfs(A) (devam)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     ▶ int w = q.poll();
7     ▶ System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

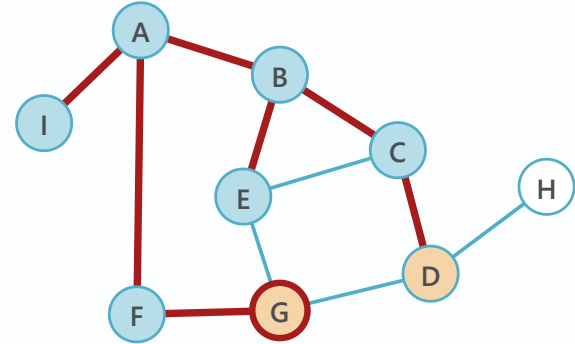
```

DEĞİŞKENLER

w = G

ÇIKTI

A B F I C E G



q (ön → arka)

D								
---	--	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

Kuyruğun önündeki **G** çıkarılır (poll) ve yazdırılır. Şimdi komşuları D, E, F taranır.

Kod İzleme: bfs(A) (devam)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     int w = q.poll();
7     System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       ▶ if (!ziyaret[u]) {
10         ziyaret[u] = true;
11         q.add(u);
12       }
13   }
14 }

```

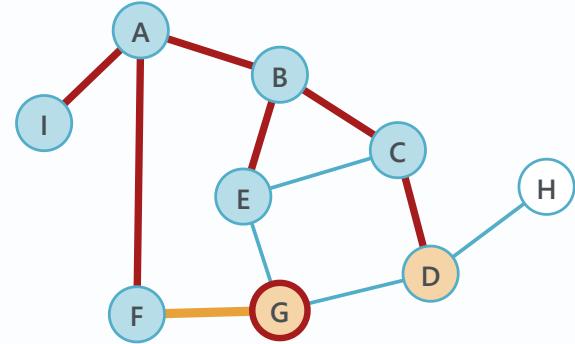
DEĞİŞKENLER

w = G

u = F

ÇIKTI

A B F I C E G



q (ön → arka)

D								
---	--	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

u = D, E, F: hepsi zaten ziyaret edilmiş → atlanır.

Kod İzleme: bfs(A) (devam)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     ▶ int w = q.poll();
7     ▶ System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

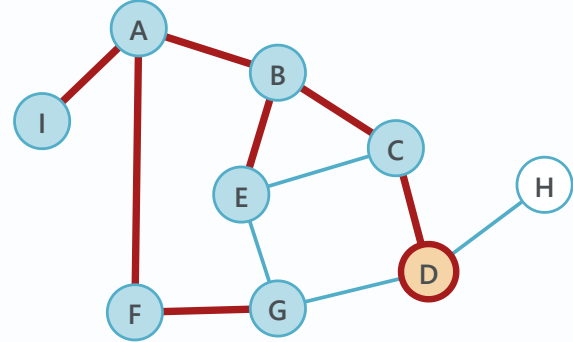
```

DEĞİŞKENLER

w = D

ÇIKTI

A B F I C E G D



q (ön → arka)

(boş)

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

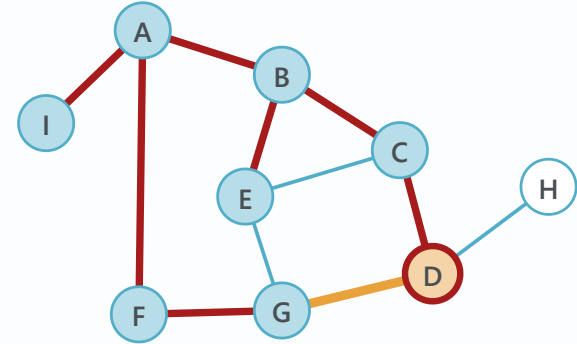
Kuyruğun önündeki **D** çıkarılır (poll) ve yazdırılır. Şimdi komşuları C, G, H taranır.

Kod İzleme: bfs(A) (devam)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     int w = q.poll();
7     System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       ▶ if (!ziyaret[u]) {
10         ziyaret[u] = true;
11         q.add(u);
12       }
13   }
14 }

```



q (ön → arka)

(boş)

--	--	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	F	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

u = C, G: hepsi zaten ziyaret edilmiş → atlanır.

DEĞİŞKENLER

w = D

u = G

ÇIKTI

A B F I C E G D

Kod İzleme: bfs(A) (devam)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     int w = q.poll();
7     System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

```

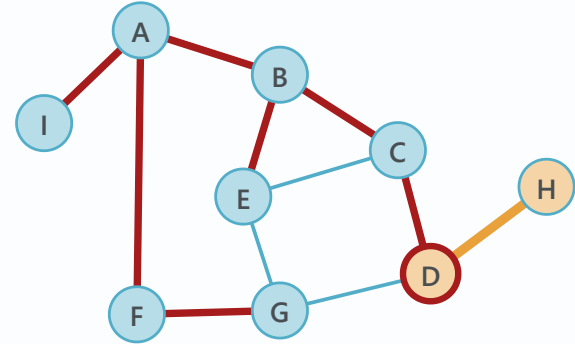
DEĞİŞKENLER

w = D

u = H

ÇIKTI

A B F I C E G D



q (ön → arka)



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

u = H ziyaret edilmemiş → işaretlenir ve kuyruğun arkasına eklenir (seviye 4).

Kod İzleme: bfs(A) (devam)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     ▶ int w = q.poll();
7     ▶ System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13    }
14  }

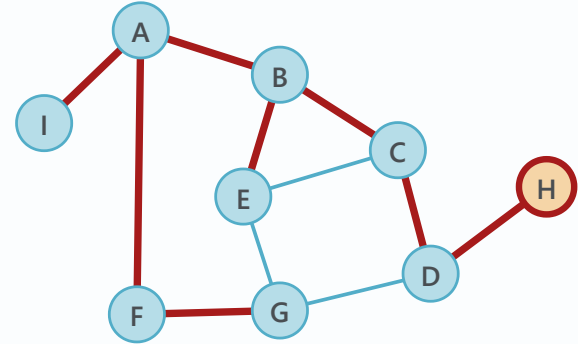
```

DEĞİŞKENLER

w = H

ÇIKTI

A B F I C E G D H



q (ön → arka)

(boş)								
-------	--	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

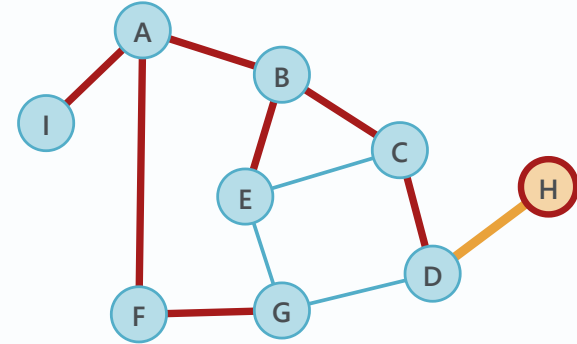
Kuyruğun önündeki **H** çıkarılır (poll) ve yazdırılır. Şimdi komşusu D taranır.

Kod İzleme: bfs(A) (devam)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   while (!q.isEmpty()) {
6     int w = q.poll();
7     System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       ▶ if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

```



q (ön → arka)

(boş)

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

u = D: zaten ziyaret edilmiş → atlanır.

DEĞİŞKENLER

w = H

u = D

ÇIKTI

A B F I C E G D H

Kod İzleme: bfs(A) (devam)

```

1 void bfs(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Queue<Integer> q = new ArrayDeque<>();
4   ziyaret[s] = true; q.add(s);
5   ► while (!q.isEmpty()) {
6     int w = q.poll();
7     System.out.print(w + " ");
8     for (int u : komsu.get(w))
9       if (!ziyaret[u]) {
10        ziyaret[u] = true;
11        q.add(u);
12      }
13   }
14 }

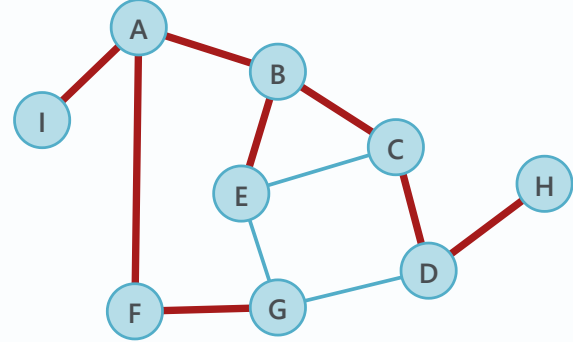
```

DEĞİŞKENLER

—

ÇIKTI

A B F I C E G D H



q (ön → arka)

(boş)

ziyaret[]

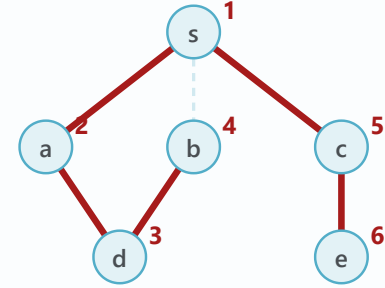
A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	T

○ görülmedi ● kuyrukta ● işleniyor ● işlendi

Kuyruk boş → biter. Sıra **A B F I C E G D H**: seviye
seviye. Her düğüm bir kez kuyruğa girer, her liste
bir kez taranır → **O(n + m)**.

Derinlik Öncelikli Arama (DFS)

- Düğümleri **bir yol boyunca** keşfeder: gidebildiği kadar derine iner.
- Çıkmaza gelince (ziyaret edilmemiş komşu kalmayınca) **geri döner** (backtracking) ve başka dalı dener.
- **Yığın** (stack) yapısı veya özyinelemeli çağrılar kullanılır.



s→a→d→b derine; çıkmazda geri, sonra c→e

Artıları

- + Bellek kullanımı genellikle düşüktür (yalnızca mevcut yol yığında).
- + Bir yolun sonuna kadar gitme yeteneği sağlar.

Eksisi

- En kısa yolu garanti etmez.

Kullanım: labirent çözme, döngü tespiti, bağlantılı bileşenleri bulma ve yönlü döngüsüz çizgelerde topolojik sıralama.

DFS Algoritması

Özyinelemeli

```
void dfs(int v, boolean[] ziyaret) {  
    ziyaret[v] = true;  
    System.out.print(v + " ");  
    for (int u : komşu.get(v))  
        if (!ziyaret[u])  
            dfs(u, ziyaret);  
}
```

Özyineleme, yığının bizim yerimize **çağrı yığını** (Bölüm 4) ile tutar.

Açık yığınla

```
void dfsYigin(int s) {  
    boolean[] ziyaret = new boolean[n];  
    Deque<Integer> yigin = new ArrayDeque<>();  
    yigin.push(s); ziyaret[s] = true;  
    while (!yigin.isEmpty()) {  
        int v = yigin.peek();  
        Integer yeni = null;  
        for (int u : komşu.get(v))  
            if (!ziyaret[u]) { yeni = u; break; }  
        if (yeni == null) yigin.pop(); // geri  
        else { ziyaret[yeni] = true;  
            yigin.push(yeni); }  
    }  
}
```

Kod İzleme: dfs(A) (özyinelemeli)

```

1 void dfs(int v, boolean[] ziyaret) {
  ► ziyaret[v] = true;
  ► System.out.print(v + " ");
  4 for (int u : komsu.get(v))
  5   if (!ziyaret[u])
  6     dfs(u, ziyaret);
  7 }

```

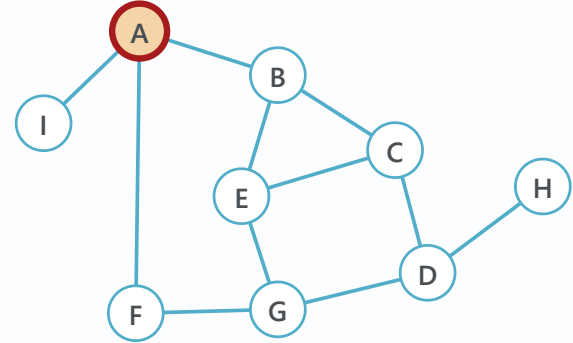
ÇAĞRI YIĞINI

dfs(A)

main()

ÇIKTI

A



ziyaret[]

A	B	C	D	E	F	G	H	I
T	F	F	F	F	F	F	F	F

○ görülmedi ● yığında ● üstte ● çıktı

dfs(A, ziyaret) çağrıldı (A=0, B=1, ..., I=8; komşular alfabetik). A işaretlenir ve yazdırılır.

Adım 1 / 15

DEĞİŞKENLER

v = A

Kod İzleme: dfs(A) (özyinelemeli)

```

1 void dfs(int v, boolean[] ziyaret) {
  ► ziyaret[v] = true;
  ► System.out.print(v + " ");
  4 for (int u : komsu.get(v))
  5   if (!ziyaret[u])
  6     dfs(u, ziyaret);
  7 }

```

ÇAĞRI YIĞINI

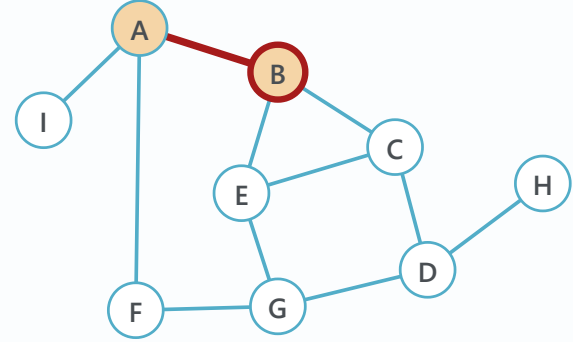
dfs(B)

dfs(A)

main()

ÇIKTI

A B



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	F	F	F	F	F	F	F

○ görülmedi ● yığında ● üstte ● çıktı

A'nın komşusu B ziyaret edilmemiş → **dfs(B)**
çağrılır: yeni çerçeve açılır, B işaretlenip yazdırılır.

Adım 2 / 15

DEĞİŞKENLER

v = B

Kod İzleme: dfs(A) (özyinelemeli)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

ÇAĞRI YIĞINI

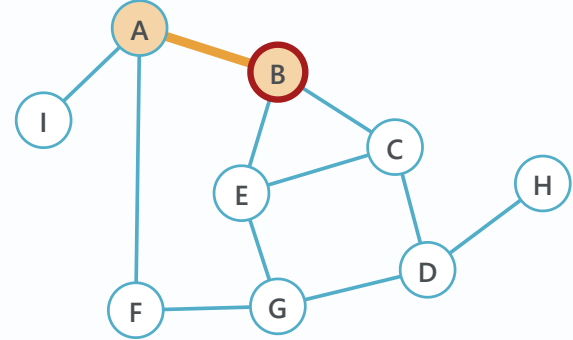
dfs(B)

dfs(A)

main()

ÇIKTI

A B



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	F	F	F	F	F	F	F

○ görülmedi ● yığında ● üstte ● çıktı

B'nin komşusu A zaten ziyaret edilmiş → atla.

Adım 3 / 15

DEĞİŞKENLER

v = B

u = A

Kod İzleme: dfs(A) (özyinelemeli)

```

1 void dfs(int v, boolean[] ziyaret) {
  ► ziyaret[v] = true;
  ► System.out.print(v + " ");
  4 for (int u : komsu.get(v))
  5   if (!ziyaret[u])
  6     dfs(u, ziyaret);
  7 }

```

ÇAĞRI YIĞINI

dfs(C)

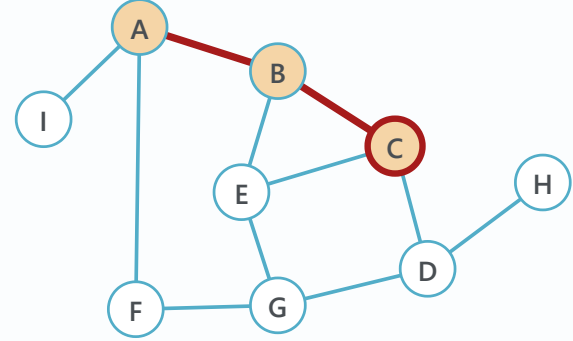
dfs(B)

dfs(A)

main()

ÇIKTI

A B C



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	F	F	F	F	F	F

○ görülmedi ● yığında ● üstte ● çıktı

B'nin komşusu C ziyaret edilmemiş → **dfs(C)**
çağrılır: yeni çerçeve açılır, C işaretlenip yazdırılır.

Adım 4 / 15

DEĞİŞKENLER

v = C

Kod İzleme: dfs(A) (özyinelemeli)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

ÇAĞRI YIĞINI

dfs(C)

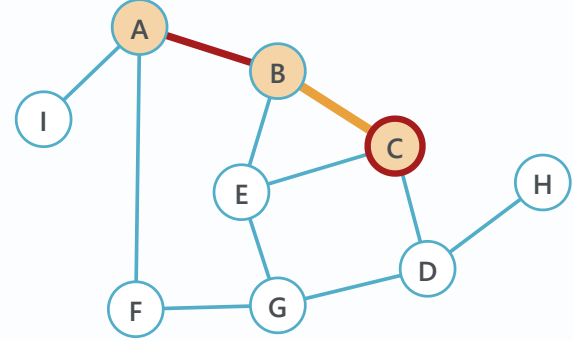
dfs(B)

dfs(A)

main()

ÇIKTI

A B C



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	F	F	F	F	F	F

○ görülmedi ● yığında ● üstte ● çıktı

C'nin komşusu B zaten ziyaret edilmiş → atla.

Adım 5 / 15

DEĞİŞKENLER

v = C

u = B

Kod İzleme: dfs(A) (özyinelemeli)

```

1 void dfs(int v, boolean[] ziyaret) {
  ► ziyaret[v] = true;
  ► System.out.print(v + " ");
  4 for (int u : komsu.get(v))
  5   if (!ziyaret[u])
  6     dfs(u, ziyaret);
  7 }

```

ÇAĞRI YIĞINI

dfs(D)

dfs(C)

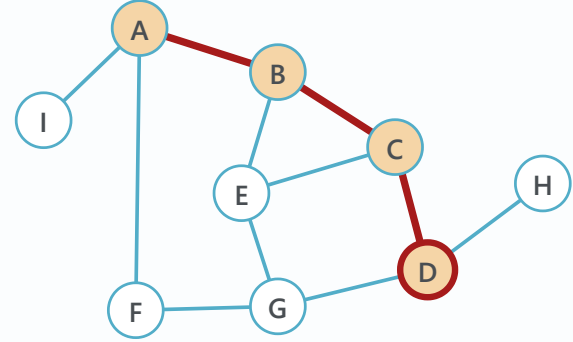
dfs(B)

dfs(A)

main()

ÇIKTI

A B C D



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	F	F	F	F	F

○ görülmedi ● yığında ● üstte ● çıktı

C'nin komşusu D ziyaret edilmemiş → **dfs(D)**
çağrılır: yeni çerçeve açılır, D işaretlenip yazdırılır.

Adım 6 / 15

DEĞİŞKENLER

v = D

Kod İzleme: dfs(A) (özyinelemeli)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

ÇAĞRI YIĞINI

dfs(D)

dfs(C)

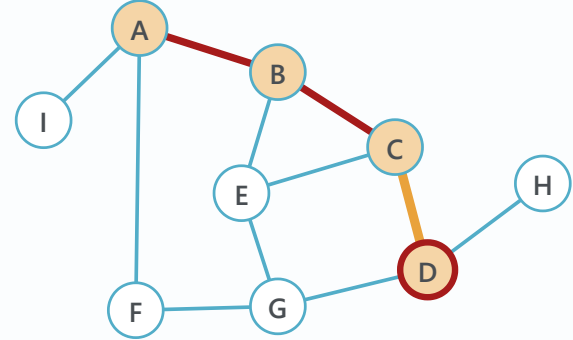
dfs(B)

dfs(A)

main()

ÇIKTI

A B C D



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	F	F	F	F	F

○ görülmedi ● yığında ● üstte ● çıktı

D'nin komşusu C zaten ziyaret edilmiş → atla.

Adım 7 / 15

DEĞİŞKENLER

v = D

u = C

Kod İzleme: dfs(A) (özyinelemeli)

```

1 void dfs(int v, boolean[] ziyaret) {
  ► ziyaret[v] = true;
  ► System.out.print(v + " ");
  4 for (int u : komsu.get(v))
  5   if (!ziyaret[u])
  6     dfs(u, ziyaret);
  7 }

```

ÇAĞRI YIĞINI

dfs(G)

dfs(D)

dfs(C)

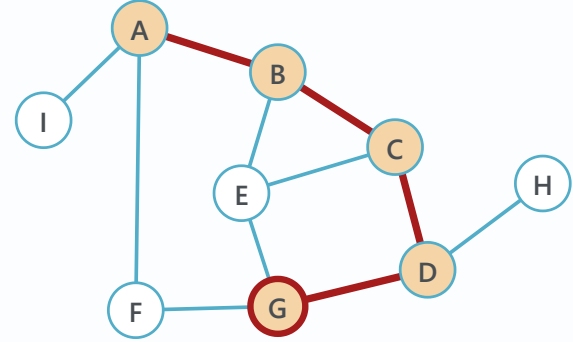
dfs(B)

dfs(A)

main()

ÇIKTI

A B C D G



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	F	F	T	F	F

○ görülmedi ● yığında ● üstte ● çıktı

D'nin komşusu G ziyaret edilmemiş → **dfs(G)**
çağrılır: yeni çerçeve açılır, G işaretlenip yazdırılır.

Adım 8 / 15

DEĞİŞKENLER

v = G

Kod İzleme: dfs(A) (özyinelemeli)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

ÇAĞRI YIĞINI

dfs(G)

dfs(D)

dfs(C)

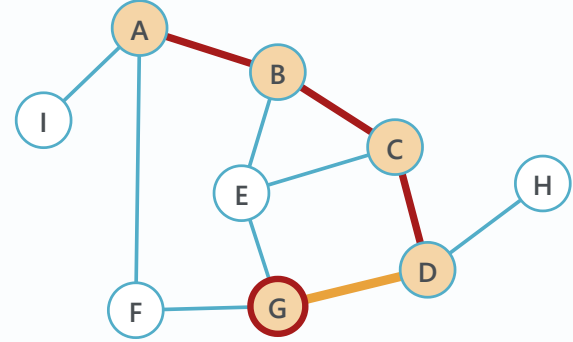
dfs(B)

dfs(A)

main()

ÇIKTI

A B C D G



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	F	F	T	F	F

○ görülmedi ● yığında ● üstte ● çıktı

G'nin komşusu D zaten ziyaret edilmiş → atla.

Adım 9 / 15

DEĞİŞKENLER

v = G

u = D

Kod İzleme: dfs(A) (özyinelemeli)

```

1 void dfs(int v, boolean[] ziyaret) {
  ► ziyaret[v] = true;
  ► System.out.print(v + " ");
  4 for (int u : komsu.get(v))
  5   if (!ziyaret[u])
  6     dfs(u, ziyaret);
  7 }

```

ÇAĞRI YIĞINI

dfs(E)

dfs(G)

dfs(D)

dfs(C)

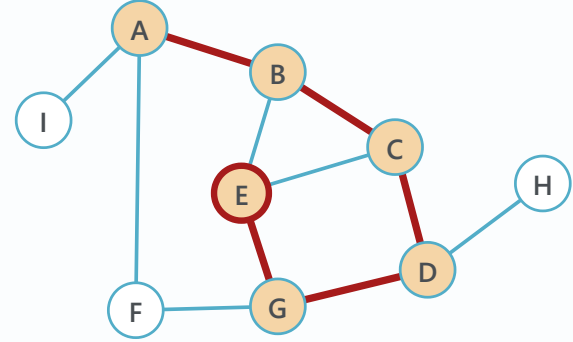
dfs(B)

dfs(A)

main()

ÇIKTI

A B C D G E



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	F	T	F	F

○ görülmedi ● yığında ● üstte ● çıktı

G'nin komşusu E ziyaret edilmemiş → **dfs(E)**
çağrılır: yeni çerçeve açılır, E işaretlenip yazdırılır.

Adım 10 / 15

DEĞİŞKENLER

v = E

Kod İzleme: dfs(A) (özyinelemeli)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

ÇAĞRI YIĞINI

dfs(E)

dfs(G)

dfs(D)

dfs(C)

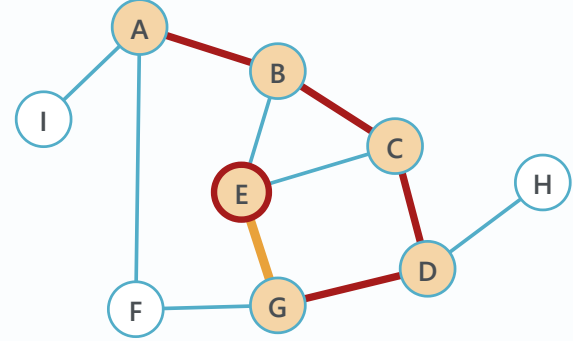
dfs(B)

dfs(A)

main()

ÇIKTI

A B C D G E



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	F	T	F	F

○ görülmedi ● yığında ● üstte ● çıktı

E'nin komşuları B, C, G zaten ziyaret edilmiş → atla.

Adım 11 / 15

DEĞİŞKENLER

v = E

u = G

Kod İzleme: dfs(A) (özyinelemeli)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

ÇAĞRI YIĞINI

dfs(E)

dfs(G)

dfs(D)

dfs(C)

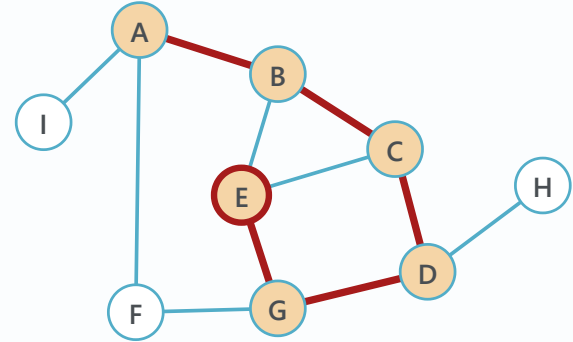
dfs(B)

dfs(A)

main()

ÇIKTI

A B C D G E



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	F	T	F	F

○ görülmedi ● yığında ● üstte ● çıktı

E'nin gidilecek komşusu kalmadı → **dfs(E)** biter, çerçeve yığından çıkar (geri dönüş).

Adım 12 / 15

DEĞİŞKENLER

v = E

Kod İzleme: dfs(A) (özyinelemeli)

```

1 void dfs(int v, boolean[] ziyaret) {
  ► ziyaret[v] = true;
  ► System.out.print(v + " ");
  4 for (int u : komsu.get(v))
  5   if (!ziyaret[u])
  6     dfs(u, ziyaret);
  7 }

```

ÇAĞRI YIĞINI

dfs(F)

dfs(G)

dfs(D)

dfs(C)

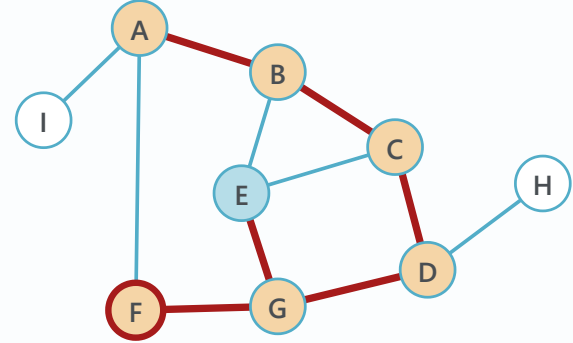
dfs(B)

dfs(A)

main()

ÇIKTI

A B C D G E F



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	F	F

○ görülmedi ● yığında ● üstte ● çıktı

G'nin komşusu F ziyaret edilmemiş → **dfs(F)**
çağrılır: yeni çerçeve açılır, F işaretlenip yazdırılır.

Adım 13 / 15

DEĞİŞKENLER

v = F

Kod İzleme: dfs(A) (özyinelemeli)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

ÇAĞRI YIĞINI

dfs(F)

dfs(G)

dfs(D)

dfs(C)

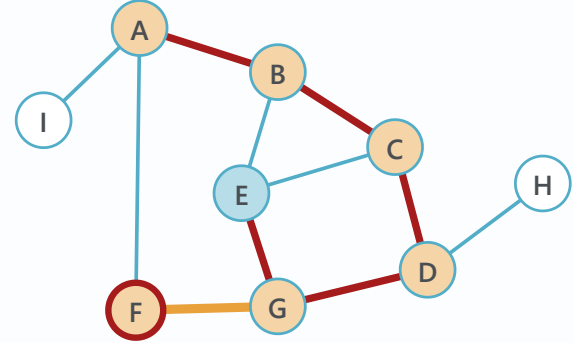
dfs(B)

dfs(A)

main()

ÇIKTI

A B C D G E F



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	F	F

○ görülmedi ● yığında ● üstte ● çıktı

F'nin komşuları A, G zaten ziyaret edilmiş → atla.

Adım 14 / 15

DEĞİŞKENLER

v = F

u = G

Kod İzleme: dfs(A) (özyinelemeli)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

ÇAĞRI YIĞINI

dfs(F)

dfs(G)

dfs(D)

dfs(C)

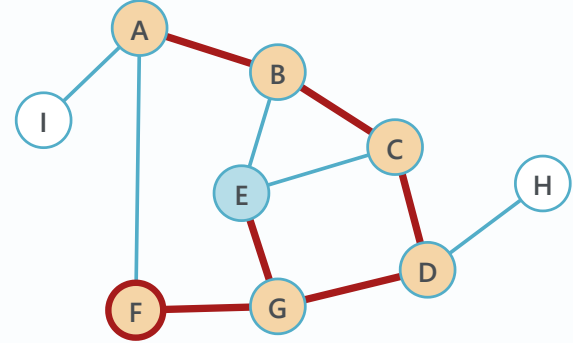
dfs(B)

dfs(A)

main()

ÇIKTI

A B C D G E F



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	F	F

○ görülmedi ● yığında ● üstte ● çıktı

F'nin gidilecek komşusu kalmadı → **dfs(F)** biter, çerçeve yığından çıkar (geri dönüş).

Adım 15 / 15

DEĞİŞKENLER

v = F

Kod İzleme: dfs(A) (özyinelemeli) (devam)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

ÇAĞRI YIĞINI

dfs(G)

dfs(D)

dfs(C)

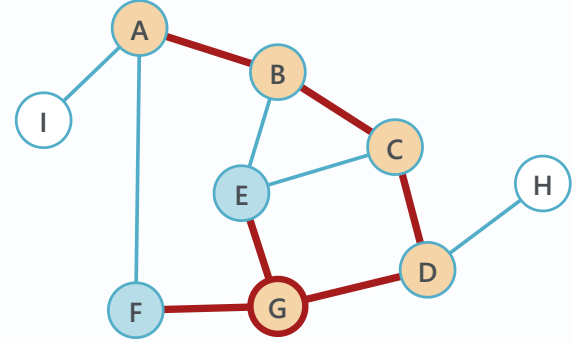
dfs(B)

dfs(A)

main()

ÇIKTI

A B C D G E F



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	F	F

○ görülmedi ● yığında ● üstte ● çıktı

G'nin gidilecek komşusu kalmadı → **dfs(G)**
biter, çerçeve yığından çıkar (geri dönüş).

Adım 1 / 14

DEĞİŞKENLER

v = G

Kod İzleme: dfs(A) (özyinelemeli) (devam)

```

1 void dfs(int v, boolean[] ziyaret) {
  ► ziyaret[v] = true;
  ► System.out.print(v + " ");
  4 for (int u : komsu.get(v))
  5   if (!ziyaret[u])
  6     dfs(u, ziyaret);
  7 }

```

ÇAĞRI YIĞINI

dfs(H)

dfs(D)

dfs(C)

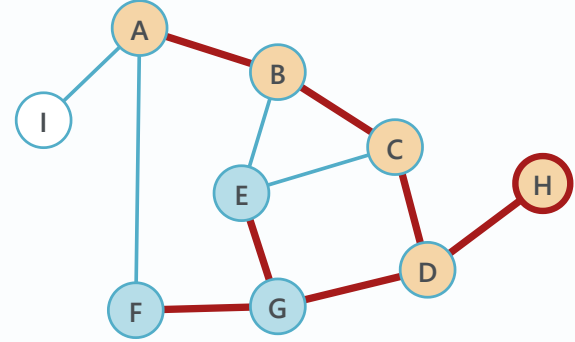
dfs(B)

dfs(A)

main()

ÇIKTI

A B C D G E F H



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	F

○ görülmedi ● yığında ● üstte ● çıktı

D'nin komşusu H ziyaret edilmemiş → **dfs(H)**
çağrılır: yeni çerçeve açılır, H işaretlenip yazdırılır.

Adım 2 / 14

DEĞİŞKENLER

v = H

Kod İzleme: dfs(A) (özyinelemeli) (devam)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

ÇAĞRI YIĞINI

dfs(H)

dfs(D)

dfs(C)

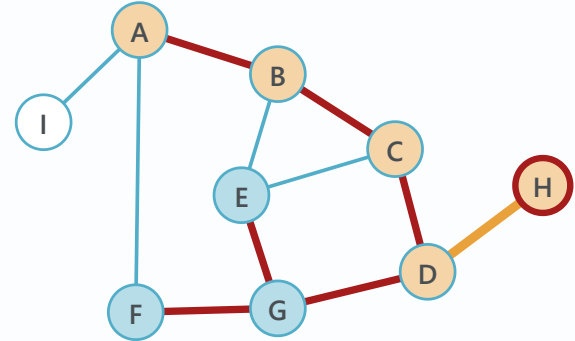
dfs(B)

dfs(A)

main()

ÇIKTI

A B C D G E F H



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	F

○ görülmedi ● yığında ● üstte ● çıktı

H'nin komşusu D zaten ziyaret edilmiş → atla.

Adım 3 / 14

DEĞİŞKENLER

v = H

u = D

Kod İzleme: dfs(A) (özyinelemeli) (devam)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

ÇAĞRI YIĞINI

dfs(H)

dfs(D)

dfs(C)

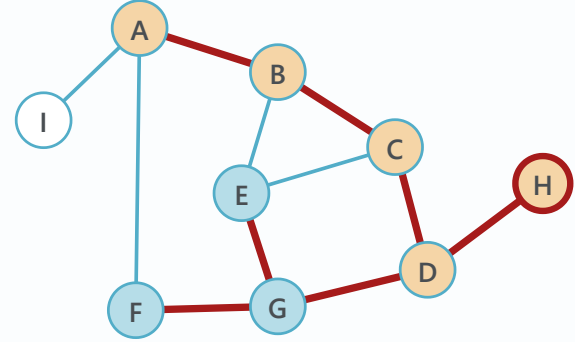
dfs(B)

dfs(A)

main()

ÇIKTI

A B C D G E F H



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	F

○ görülmedi ● yığında ● üstte ● çıktı

H'nin gidilecek komşusu kalmadı → **dfs(H)** biter, çerçeve yığından çıkar (geri dönüş).

Adım 4 / 14

DEĞİŞKENLER

v = H

Kod İzleme: dfs(A) (özyinelemeli) (devam)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

ÇAĞRI YIĞINI

dfs(D)

dfs(C)

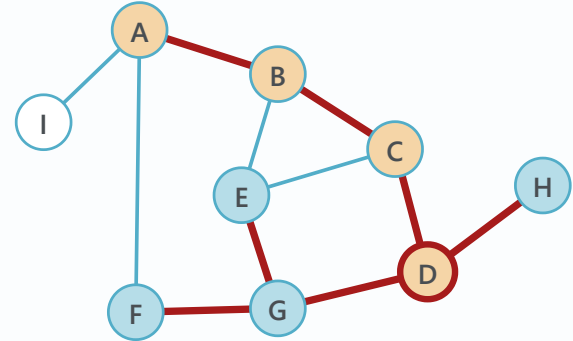
dfs(B)

dfs(A)

main()

ÇIKTI

A B C D G E F H



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	F

○ görülmedi ● yığında ● üstte ● çıktı

D'nin gidilecek komşusu kalmadı → **dfs(D)** biter, çerçeve yığından çıkar (geri dönüş).

Adım 5 / 14

DEĞİŞKENLER

v = D

Kod İzleme: dfs(A) (özyinelemeli) (devam)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

ÇAĞRI YIĞINI

dfs(C)

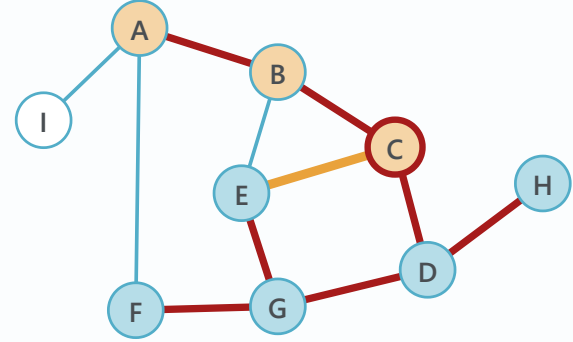
dfs(B)

dfs(A)

main()

ÇIKTI

A B C D G E F H



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	F

○ görülmedi ● yığında ● üstte ● çıktı

C'nin komşusu E zaten ziyaret edilmiş → atla.

Adım 6 / 14

DEĞİŞKENLER

v = C

u = E

Kod İzleme: dfs(A) (özyinelemeli) (devam)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

ÇAĞRI YIĞINI

dfs(C)

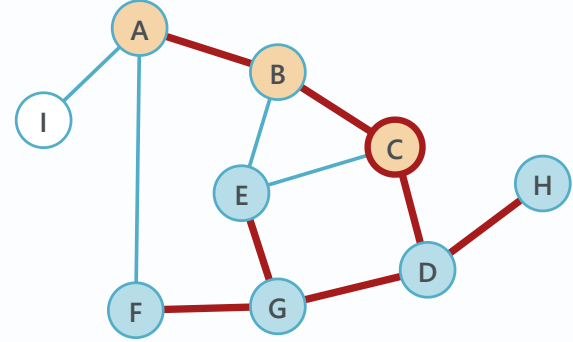
dfs(B)

dfs(A)

main()

ÇIKTI

A B C D G E F H



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	F

○ görülmedi ● yığında ● üstte ● çıktı

C'nin gidilecek komşusu kalmadı → **dfs(C)** biter, çerçeve yığından çıkar (geri dönüş).

Adım 7 / 14

DEĞİŞKENLER

v = C

Kod İzleme: dfs(A) (özyinelemeli) (devam)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

ÇAĞRI YIĞINI

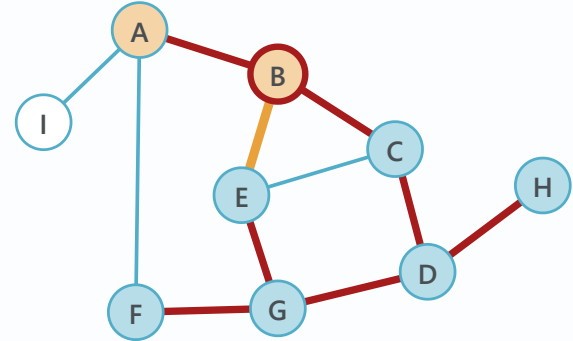
dfs(B)

dfs(A)

main()

ÇIKTI

A B C D G E F H



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	F

○ görülmedi ● yığında ● üstte ● çıktı

B'nin komşusu E zaten ziyaret edilmiş → atla.

Adım 8 / 14

DEĞİŞKENLER

v = B

u = E

Kod İzleme: dfs(A) (özyinelemeli) (devam)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

ÇAĞRI YIĞINI

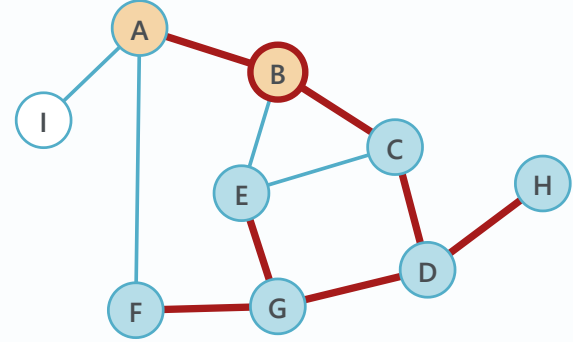
dfs(B)

dfs(A)

main()

ÇIKTI

A B C D G E F H



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	F

○ görülmedi ● yığında ● üstte ● çıktı

B'nin gidilecek komşusu kalmadı → **dfs(B)** biter, çerçeve yığından çıkar (geri dönüş).

Adım 9 / 14

DEĞİŞKENLER

v = B

Kod İzleme: dfs(A) (özyinelemeli) (devam)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

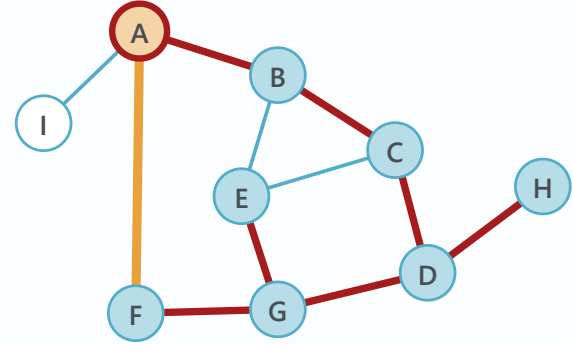
ÇAĞRI YIĞINI

dfs(A)

main()

ÇIKTI

A B C D G E F H



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	F

○ görülmedi ● yığında ● üstte ● çıktı

A'nın komşusu F zaten ziyaret edilmiş → atla.

Adım 10 / 14

DEĞİŞKENLER

v = A

u = F

Kod İzleme: dfs(A) (özyinelemeli) (devam)

```

1 void dfs(int v, boolean[] ziyaret) {
  ► ziyaret[v] = true;
  ► System.out.print(v + " ");
  4 for (int u : komsu.get(v))
  5   if (!ziyaret[u])
  6     dfs(u, ziyaret);
  7 }

```

ÇAĞRI YIĞINI

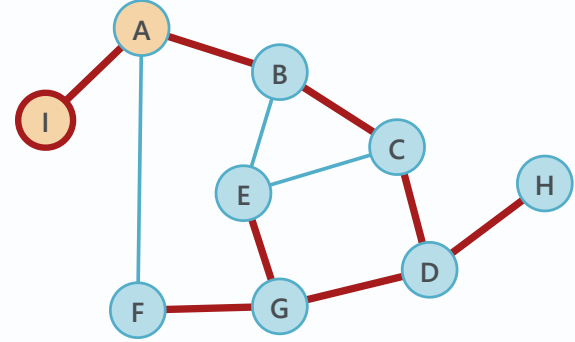
dfs(I)

dfs(A)

main()

ÇIKTI

A B C D G E F H I



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	T

○ görülmedi ● yığında ● üstte ● çıktı

A'nın komşusu I ziyaret edilmemiş → **dfs(I)**
çağrılır: yeni çerçeve açılır, I işaretlenip yazdırılır.

Adım 11 / 14

DEĞİŞKENLER

v = I

Kod İzleme: dfs(A) (özyinelemeli) (devam)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

ÇAĞRI YIĞINI

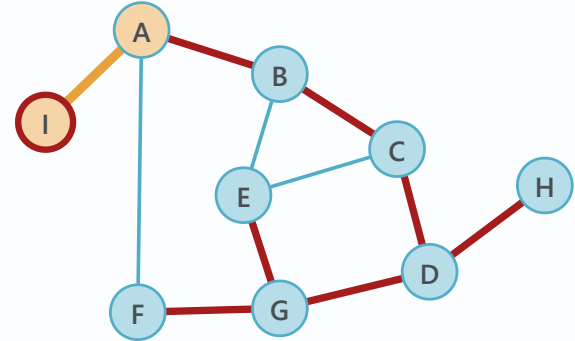
dfs(I)

dfs(A)

main()

ÇIKTI

A B C D G E F H I



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	T

○ görülmedi ● yığında ● üstte ● çıktı

I'nın komşusu A zaten ziyaret edilmiş → atla.

Adım 12 / 14

DEĞİŞKENLER

v = I

u = A

Kod İzleme: dfs(A) (özyinelemeli) (devam)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

ÇAĞRI YIĞINI

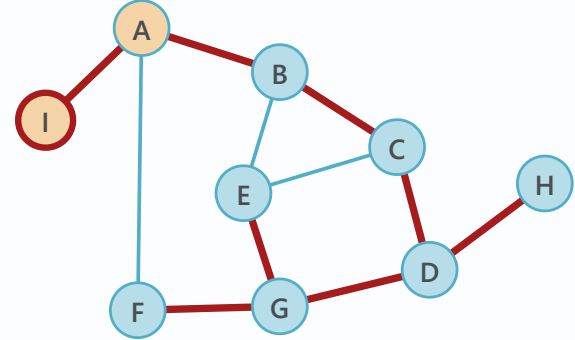
dfs(I)

dfs(A)

main()

ÇIKTI

A B C D G E F H I



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	T

○ görülmedi ● yığında ● üstte ● çıktı

I'nın gidilecek komşusu kalmadı → **dfs(I)** biter, çerçeve yığından çıkar (geri dönüş).

Adım 13 / 14

DEĞİŞKENLER

v = I

Kod İzleme: dfs(A) (özyinelemeli) (devam)

```

1 void dfs(int v, boolean[] ziyaret) {
2   ziyaret[v] = true;
3   System.out.print(v + " ");
4   for (int u : komsu.get(v))
5     if (!ziyaret[u])
6       dfs(u, ziyaret);
7 }

```

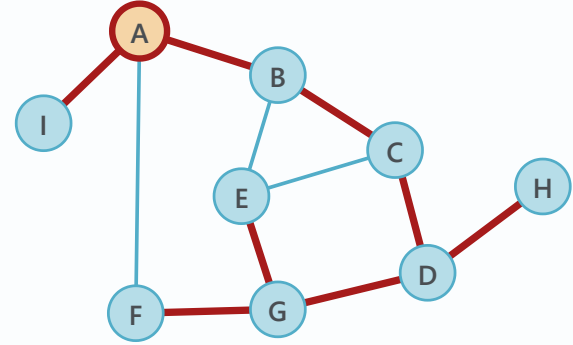
ÇAĞRI YIĞINI

dfs(A)

main()

ÇIKTI

A B C D G E F H I



ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	T

○ görülmedi ● yığında ● üstte ● çıktı

dfs(A) bitir, yığın boşalır. Sıra: **A B C D G E F H**

I.

Adım 14 / 14

DEĞİŞKENLER

v = A

Kod İzleme: dfsYigin(A)

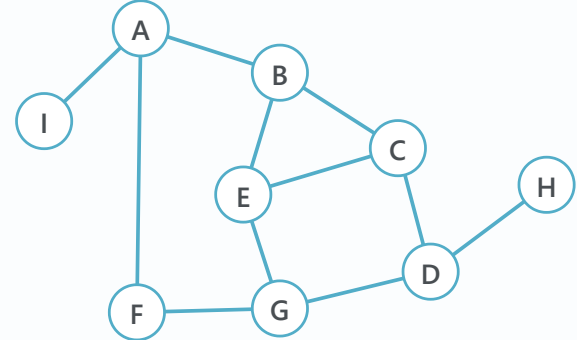
```

1 void dfsYigin(int s) {
2     ▶ boolean[] ziyaret = new boolean[n];
3     ▶ Deque<Integer> yigin = new ArrayDeque<>();
4     yigin.push(s); ziyaret[s] = true;
5     while (!yigin.isEmpty()) {
6         int v = yigin.peek();
7         Integer yeni = null;
8         for (int u : komsu.get(v))
9             if (!ziyaret[u]) { yeni = u; break; }
10        if (yeni == null) yigin.pop(); // geri
11        else { ziyaret[yeni] = true;
12                yigin.push(yeni); }
13    }
14 }

```

DEĞİŞKENLER

s = A



yigin (alt → üst)

(boş)								
-------	--	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
F	F	F	F	F	F	F	F	F

○ görülmedi ● yığında ● üstte ● çıktı

Başlangıç s = A. **ziyaret** tümü false, yığın boş.

Kod İzleme: dfsYigin(A)

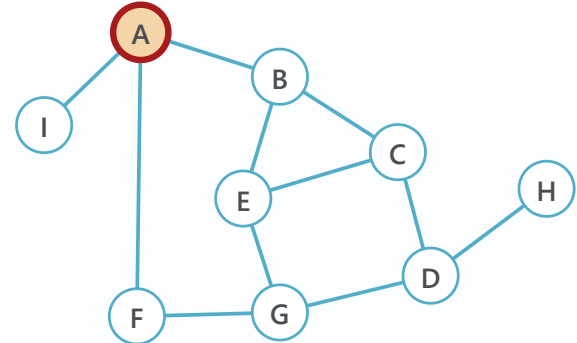
```

1 void dfsYigin(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Deque<Integer> yigin = new ArrayDeque<>();
4   ► yigin.push(s); ziyaret[s] = true;
5   while (!yigin.isEmpty()) {
6     int v = yigin.peek();
7     Integer yeni = null;
8     for (int u : komsu.get(v))
9       if (!ziyaret[u]) { yeni = u; break; }
10    if (yeni == null) yigin.pop(); // geri
11    else { ziyaret[yeni] = true;
12          yigin.push(yeni); }
13  }
14 }

```

DEĞİŞKENLER

s = A



yigin (alt → üst)

A								
---	--	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	F	F	F	F	F	F	F	F

○ görülmedi ● yığında ● üstte ● çıktı

A yığına itilir ve işaretlenir. Yığının üstü = şu an bulunduğumuz düğüm.

Kod İzleme: dfsYigin(A)

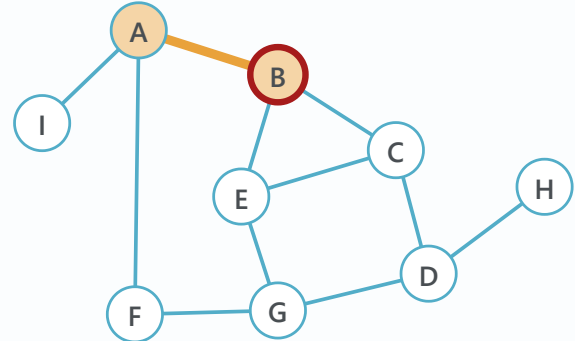
```

1 void dfsYigin(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Deque<Integer> yigin = new ArrayDeque<>();
4   yigin.push(s); ziyaret[s] = true;
5   while (!yigin.isEmpty()) {
6     ▶ int v = yigin.peek();
7     Integer yeni = null;
8     for (int u : komsu.get(v))
9       ▶ if (!ziyaret[u]) { yeni = u; break; }
10    if (yeni == null) yigin.pop(); // geri
11    ▶ else { ziyaret[yeni] = true;
12          ▶ yigin.push(yeni); }
13  }
14 }

```

DEĞİŞKENLER

v = A yeni = B



yigin (alt → üst)

A	B							
---	---	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	F	F	F	F	F	F	F

○ görülmedi ● yığında ● üstte ● çıktı

peek = **A**; ilk ziyaret edilmemiş komşusu **B** (break)
→ işaretlenir ve yığına itilir.

Kod İzleme: dfsYigin(A)

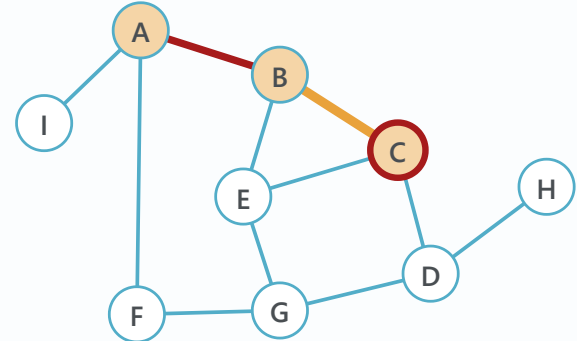
```

1 void dfsYigin(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Deque<Integer> yigin = new ArrayDeque<>();
4   yigin.push(s); ziyaret[s] = true;
5   while (!yigin.isEmpty()) {
6     ▶ int v = yigin.peek();
7     Integer yeni = null;
8     for (int u : komsu.get(v))
9       ▶ if (!ziyaret[u]) { yeni = u; break; }
10    if (yeni == null) yigin.pop(); // geri
11    ▶ else { ziyaret[yeni] = true;
12          ▶ yigin.push(yeni); }
13  }
14 }

```

DEĞİŞKENLER

v = B yeni = C



yigin (alt → üst)

A	B	C						
---	---	---	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	F	F	F	F	F	F

○ görülmedi ● yığında ● üstte ● çıktı

peek = B; ilk ziyaret edilmemiş komşusu C (break)
→ işaretlenir ve yığına itilir.

Kod İzleme: dfsYigin(A)

```

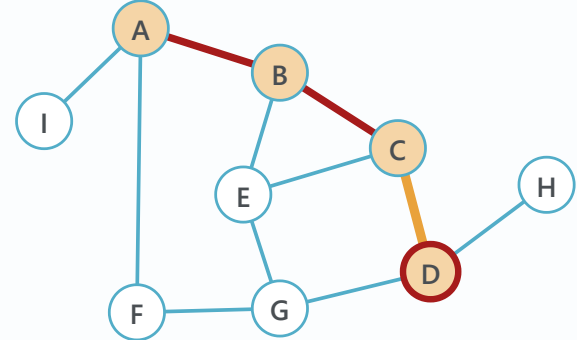
1 void dfsYigin(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Deque<Integer> yigin = new ArrayDeque<>();
4   yigin.push(s); ziyaret[s] = true;
5   while (!yigin.isEmpty()) {
6     ▶ int v = yigin.peek();
7     Integer yeni = null;
8     for (int u : komsu.get(v))
9       ▶ if (!ziyaret[u]) { yeni = u; break; }
10    if (yeni == null) yigin.pop(); // geri
11    ▶ else { ziyaret[yeni] = true;
12          ▶ yigin.push(yeni); }
13  }
14 }

```

DEĞİŞKENLER

v = C

yeni = D



yigin (alt → üst)

A	B	C	D					
---	---	---	---	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	F	F	F	F	F

○ görülmedi ● yığında ● üstte ● çıktı

peek = C; ilk ziyaret edilmemiş komşusu D (break)
→ işaretlenir ve yığına itilir.

Kod İzleme: dfsYigin(A)

```

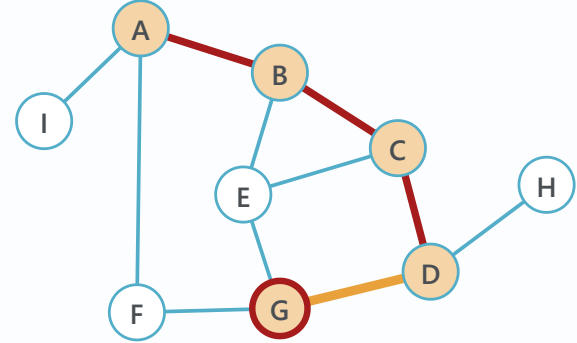
1 void dfsYigin(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Deque<Integer> yigin = new ArrayDeque<>();
4   yigin.push(s); ziyaret[s] = true;
5   while (!yigin.isEmpty()) {
6     ► int v = yigin.peek();
7     Integer yeni = null;
8     for (int u : komsu.get(v))
9       ► if (!ziyaret[u]) { yeni = u; break; }
10    if (yeni == null) yigin.pop(); // geri
11    ► else { ziyaret[yeni] = true;
12          ► yigin.push(yeni); }
13  }
14 }

```

DEĞİŞKENLER

v = D

yeni = G



yigin (alt → üst)

A	B	C	D	G				
---	---	---	---	---	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	F	F	T	F	F

○ görülmedi ● yığında ● üstte ● çıktı

peek = D; ilk ziyaret edilmemiş komşusu G (break)
→ işaretlenir ve yığına itilir.

Kod İzleme: dfsYigin(A)

```

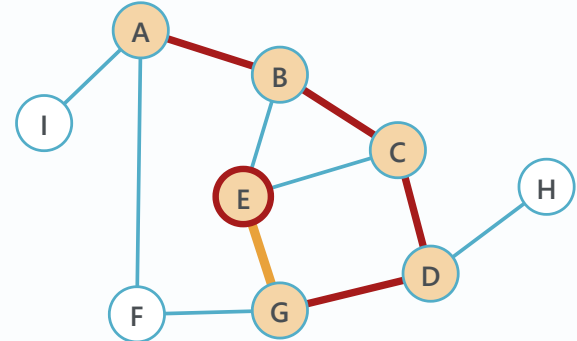
1 void dfsYigin(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Deque<Integer> yigin = new ArrayDeque<>();
4   yigin.push(s); ziyaret[s] = true;
5   while (!yigin.isEmpty()) {
6     ▶ int v = yigin.peek();
7     Integer yeni = null;
8     for (int u : komsu.get(v))
9       ▶ if (!ziyaret[u]) { yeni = u; break; }
10    if (yeni == null) yigin.pop(); // geri
11    ▶ else { ziyaret[yeni] = true;
12          ▶ yigin.push(yeni); }
13  }
14 }

```

DEĞİŞKENLER

v = G

yeni = E



yigin (alt → üst)

A	B	C	D	G	E		
---	---	---	---	---	---	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	F	T	F	F

○ görülmedi ● yığında ● üstte ● çıktı

peek = G; ilk ziyaret edilmemiş komşusu E (break)
→ işaretlenir ve yığına itilir.

Kod İzleme: dfsYigin(A)

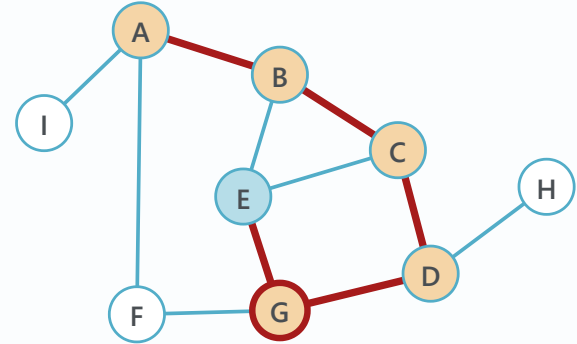
```

1 void dfsYigin(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Deque<Integer> yigin = new ArrayDeque<>();
4   yigin.push(s); ziyaret[s] = true;
5   while (!yigin.isEmpty()) {
6     ► int v = yigin.peek();
7     Integer yeni = null;
8     for (int u : komsu.get(v))
9       if (!ziyaret[u]) { yeni = u; break; }
10    ► if (yeni == null) yigin.pop(); // geri
11    else { ziyaret[yeni] = true;
12          yigin.push(yeni); }
13  }
14 }

```

DEĞİŞKENLER

v = E yeni = null



yigin (alt → üst)

A	B	C	D	G				
---	---	---	---	---	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	F	T	F	F

○ görülmedi ● yığında ● üstte ● çıktı

peek = E; ziyaret edilmemiş komşusu yok (yeni = null) → **pop**, geri dönülür.

Kod İzleme: dfsYigin(A)

```

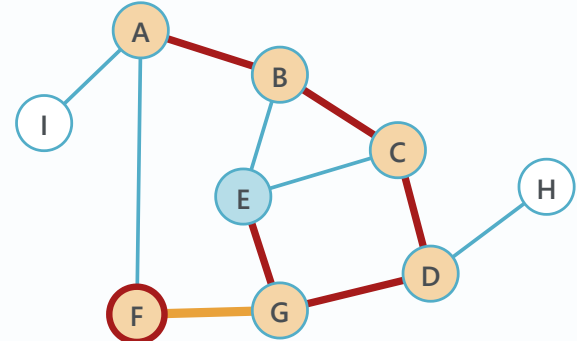
1 void dfsYigin(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Deque<Integer> yigin = new ArrayDeque<>();
4   yigin.push(s); ziyaret[s] = true;
5   while (!yigin.isEmpty()) {
6     ▶ int v = yigin.peek();
7     Integer yeni = null;
8     for (int u : komsu.get(v))
9       ▶ if (!ziyaret[u]) { yeni = u; break; }
10    if (yeni == null) yigin.pop(); // geri
11    ▶ else { ziyaret[yeni] = true;
12          ▶ yigin.push(yeni); }
13  }
14 }

```

DEĞİŞKENLER

v = G

yeni = F



yigin (alt → üst)

A	B	C	D	G	F		
---	---	---	---	---	---	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	F	F

○ görülmedi ● yığında ● üstte ● çıktı

peek = **G**; ilk ziyaret edilmemiş komşusu **F** (break)
→ işaretlenir ve yığına itilir.

Kod İzleme: dfsYigin(A)

```

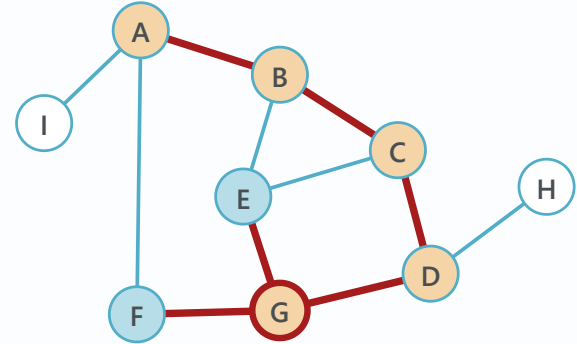
1 void dfsYigin(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Deque<Integer> yigin = new ArrayDeque<>();
4   yigin.push(s); ziyaret[s] = true;
5   while (!yigin.isEmpty()) {
6     ► int v = yigin.peek();
7     Integer yeni = null;
8     for (int u : komsu.get(v))
9       if (!ziyaret[u]) { yeni = u; break; }
10    ► if (yeni == null) yigin.pop(); // geri
11    else { ziyaret[yeni] = true;
12          yigin.push(yeni); }
13  }
14 }

```

DEĞİŞKENLER

v = F

yeni = null



yigin (alt → üst)

A	B	C	D	G				
---	---	---	---	---	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	F	F

○ görülmedi ● yığında ● üstte ● çıktı

peek = F; ziyaret edilmemiş komşusu yok (yeni = null) → **pop**, geri dönülür.

Kod İzleme: dfsYigin(A)

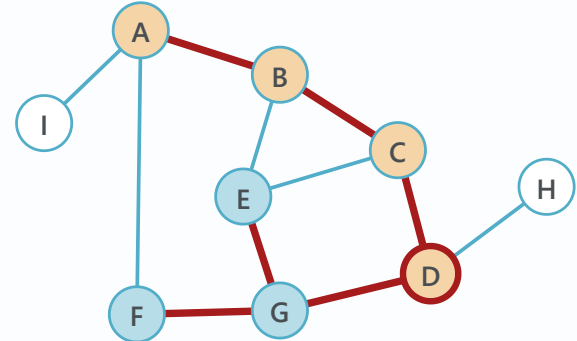
```

1 void dfsYigin(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Deque<Integer> yigin = new ArrayDeque<>();
4   yigin.push(s); ziyaret[s] = true;
5   while (!yigin.isEmpty()) {
6     ► int v = yigin.peek();
7     Integer yeni = null;
8     for (int u : komsu.get(v))
9       if (!ziyaret[u]) { yeni = u; break; }
10    ► if (yeni == null) yigin.pop(); // geri
11    else { ziyaret[yeni] = true;
12          yigin.push(yeni); }
13  }
14 }

```

DEĞİŞKENLER

v = G yeni = null



yigin (alt → üst)

A	B	C	D					
---	---	---	---	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	F	F

○ görülmedi ● yığında ● üstte ● çıktı

peek = **G**; ziyaret edilmemiş komşusu yok (yeni = null) → **pop**, geri dönülür.

Kod İzleme: dfsYigin(A)

```

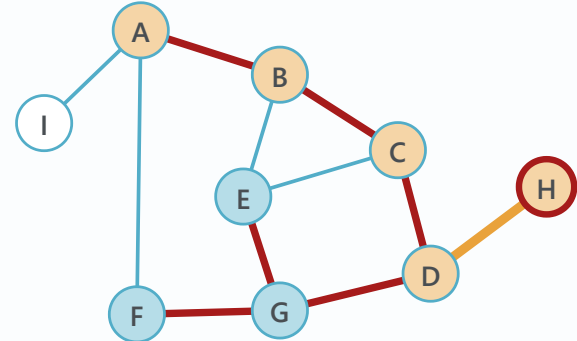
1 void dfsYigin(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Deque<Integer> yigin = new ArrayDeque<>();
4   yigin.push(s); ziyaret[s] = true;
5   while (!yigin.isEmpty()) {
6     ▶ int v = yigin.peek();
7     Integer yeni = null;
8     for (int u : komsu.get(v))
9       ▶ if (!ziyaret[u]) { yeni = u; break; }
10    if (yeni == null) yigin.pop(); // geri
11    ▶ else { ziyaret[yeni] = true;
12          ▶ yigin.push(yeni); }
13  }
14 }

```

DEĞİŞKENLER

v = D

yeni = H



yigin (alt → üst)

A	B	C	D	H				
---	---	---	---	---	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	F

○ görülmedi ● yığında ● üstte ● çıktı

peek = D; ilk ziyaret edilmemiş komşusu H (break)
→ işaretlenir ve yığına itilir.

Kod İzleme: dfsYigin(A)

```

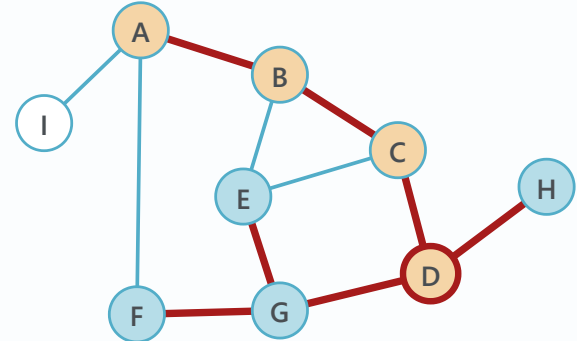
1 void dfsYigin(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Deque<Integer> yigin = new ArrayDeque<>();
4   yigin.push(s); ziyaret[s] = true;
5   while (!yigin.isEmpty()) {
6     ► int v = yigin.peek();
7     Integer yeni = null;
8     for (int u : komsu.get(v))
9       if (!ziyaret[u]) { yeni = u; break; }
10    ► if (yeni == null) yigin.pop(); // geri
11    else { ziyaret[yeni] = true;
12          yigin.push(yeni); }
13  }
14 }

```

DEĞİŞKENLER

v = H

yeni = null



yigin (alt → üst)

A	B	C	D					
---	---	---	---	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	F

○ görülmedi ● yığında ● üstte ● çıktı

peek = H; ziyaret edilmemiş komşusu yok (yeni = null) → **pop**, geri dönülür.

Kod İzleme: dfsYigin(A)

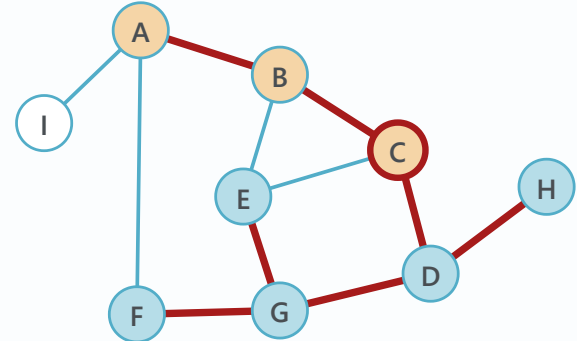
```

1 void dfsYigin(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Deque<Integer> yigin = new ArrayDeque<>();
4   yigin.push(s); ziyaret[s] = true;
5   while (!yigin.isEmpty()) {
6     ► int v = yigin.peek();
7     Integer yeni = null;
8     for (int u : komsu.get(v))
9       if (!ziyaret[u]) { yeni = u; break; }
10    ► if (yeni == null) yigin.pop(); // geri
11    else { ziyaret[yeni] = true;
12          yigin.push(yeni); }
13  }
14 }

```

DEĞİŞKENLER

v = D yeni = null



yigin (alt → üst)

A	B	C						
---	---	---	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	F

○ görülmedi ● yığında ● üstte ● çıktı

peek = D; ziyaret edilmemiş komşusu yok (yeni = null) → **pop**, geri dönülür.

Kod İzleme: dfsYigin(A)

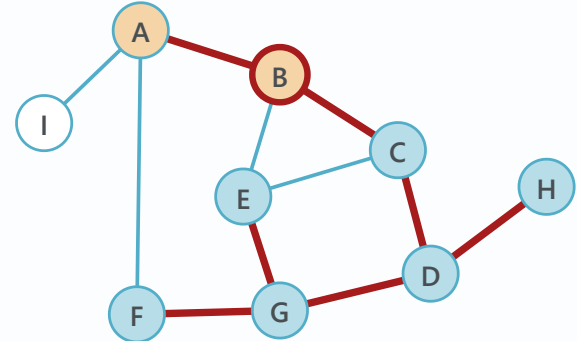
```

1 void dfsYigin(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Deque<Integer> yigin = new ArrayDeque<>();
4   yigin.push(s); ziyaret[s] = true;
5   while (!yigin.isEmpty()) {
6     ► int v = yigin.peek();
7     Integer yeni = null;
8     for (int u : komsu.get(v))
9       if (!ziyaret[u]) { yeni = u; break; }
10    ► if (yeni == null) yigin.pop(); // geri
11    else { ziyaret[yeni] = true;
12          yigin.push(yeni); }
13  }
14 }

```

DEĞİŞKENLER

v = C yeni = null



yigin (alt → üst)

A	B							
---	---	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	F

○ görülmedi ● yığında ● üstte ● çıktı

peek = C; ziyaret edilmemiş komşusu yok (yeni = null) → **pop**, geri dönülür.

Kod İzleme: dfsYigin(A)

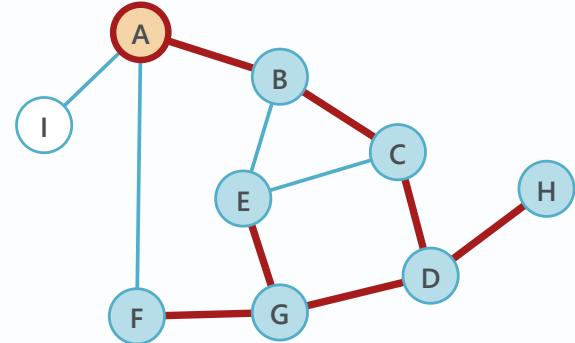
```

1 void dfsYigin(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Deque<Integer> yigin = new ArrayDeque<>();
4   yigin.push(s); ziyaret[s] = true;
5   while (!yigin.isEmpty()) {
6     ► int v = yigin.peek();
7     Integer yeni = null;
8     for (int u : komsu.get(v))
9       if (!ziyaret[u]) { yeni = u; break; }
10    ► if (yeni == null) yigin.pop(); // geri
11    else { ziyaret[yeni] = true;
12          yigin.push(yeni); }
13  }
14 }

```

DEĞİŞKENLER

v = B yeni = null



yigin (alt → üst)

A								
---	--	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	F

○ görülmedi ● yığında ● üstte ● çıktı

peek = B; ziyaret edilmemiş komşusu yok (yeni = null) → **pop**, geri dönülür.

Kod İzleme: dfsYigin(A)

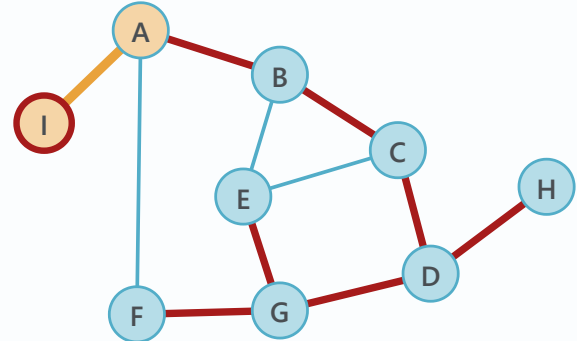
```

1 void dfsYigin(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Deque<Integer> yigin = new ArrayDeque<>();
4   yigin.push(s); ziyaret[s] = true;
5   while (!yigin.isEmpty()) {
6     ► int v = yigin.peek();
7     Integer yeni = null;
8     for (int u : komsu.get(v))
9       ► if (!ziyaret[u]) { yeni = u; break; }
10    if (yeni == null) yigin.pop(); // geri
11    ► else { ziyaret[yeni] = true;
12          ► yigin.push(yeni); }
13  }
14 }

```

DEĞİŞKENLER

v = A yeni = I



yigin (alt → üst)

A	I							
---	---	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	T

○ görülmedi ● yığında ● üstte ● çıktı

peek = A; ilk ziyaret edilmemiş komşusu I (break)
→ işaretlenir ve yığına itilir.

Kod İzleme: dfsYigin(A)

```

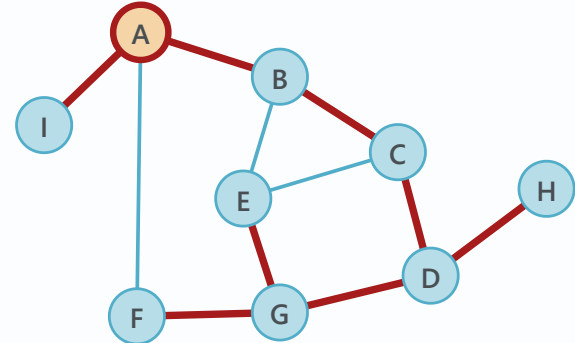
1 void dfsYigin(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Deque<Integer> yigin = new ArrayDeque<>();
4   yigin.push(s); ziyaret[s] = true;
5   while (!yigin.isEmpty()) {
6     ► int v = yigin.peek();
7     Integer yeni = null;
8     for (int u : komsu.get(v))
9       if (!ziyaret[u]) { yeni = u; break; }
10    ► if (yeni == null) yigin.pop(); // geri
11    else { ziyaret[yeni] = true;
12          yigin.push(yeni); }
13  }
14 }

```

DEĞİŞKENLER

v = I

yeni = null



yigin (alt → üst)

A								
---	--	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	T

○ görülmedi ● yığında ● üstte ● çıktı

peek = I; ziyaret edilmemiş komşusu yok (yeni = null) → **pop**, geri dönülür.

Kod İzleme: dfsYigin(A)

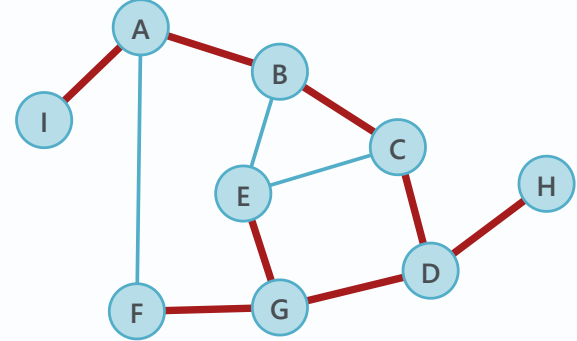
```

1 void dfsYigin(int s) {
2   boolean[] ziyaret = new boolean[n];
3   Deque<Integer> yigin = new ArrayDeque<>();
4   yigin.push(s); ziyaret[s] = true;
5   while (!yigin.isEmpty()) {
6     ► int v = yigin.peek();
7     Integer yeni = null;
8     for (int u : komsu.get(v))
9       if (!ziyaret[u]) { yeni = u; break; }
10    ► if (yeni == null) yigin.pop(); // geri
11    else { ziyaret[yeni] = true;
12          yigin.push(yeni); }
13  }
14 }

```

DEĞİŞKENLER

v = A yeni = null



yigin (alt → üst)

(boş)								
-------	--	--	--	--	--	--	--	--

ziyaret[]

A	B	C	D	E	F	G	H	I
T	T	T	T	T	T	T	T	T

○ görülmedi ● yığında ● üstte ● çıktı

peek = **A**; ziyaret edilmemiş komşusu yok (yeni = null) → **pop**, geri dönüş. Yığın boş: bitti. Ziyaret sırası özyinelemeli sürümle aynı.

BFS ve DFS Karşılaştırması

	BFS	DFS
Yardımcı yapı	Kuyruk (FIFO)	Yığın (LIFO) / özyineleme
Zaman (komşuluk listesi)	$O(n + m)$	$O(n + m)$
Ek bellek	$O(n)$: en geniş seviye kuyrukta	$O(n)$: en uzun yol yığında
En kısa yol?	Evet (ağırlıksız çizgede)	Hayır
Tipik kullanım	en az adım, seviye bulma	döngü tespiti, bileşenler, labirent

Neden $O(n + m)$? Her düğüm bir kez işlenir ve tüm komşuluk kayıtları taranır. Yönsüz çizgede her kenar iki listede görünür; $2m$ yine $O(m)$ 'dir.

Sıralar, komşuların alfabetik sırayla gezildiği varsayımıyla elde edildi; komşu sırası değişirse ziyaret sırası da değişir.

Kod İzleme: DFS ile bağlantılı bileşenleri numaralama



Tüm düğümler **-1** (henüz hiçbir bileşene atanmadı).

Adım 1 / 18

```
1 int bileşenler(int[] bil) {  
2     Arrays.fill(bil, -1);  
3     int say = 0;  
4     for (int s = 0; s < n; s++)  
5         if (bil[s] == -1) dfs(s, say++, bil);  
6     return say;  
7 }  
8 void dfs(int v, int c, int[] bil) {  
9     bil[v] = c;  
10    for (int u : komşu.get(v))  
11        if (bil[u] == -1) dfs(u, c, bil);  
12 }
```

DEĞİŞKENLER

bil = [-1, -1, -1, -1, -1, -1]

ÇAĞRI YIĞINI

bileşenler(bil)

main()

Kod İzleme: DFS ile bağlantılı bileşenleri numaralama



Bileşen sayacı **say = 0**.

Adım 2 / 18

```

1 int bileşenler(int[] bil) {
2   Arrays.fill(bil, -1);
3   int say = 0;
4   for (int s = 0; s < n; s++)
5     if (bil[s] == -1) dfs(s, say++, bil);
6   return say;
7 }
8 void dfs(int v, int c, int[] bil) {
9   bil[v] = c;
10  for (int u : komşu.get(v))
11    if (bil[u] == -1) dfs(u, c, bil);
12 }

```

DEĞİŞKENLER

say = 0

bil = [-1, -1, -1, -1, -1, -1]

ÇAĞRI YIĞINI

bileşenler(bil)

main()

Kod İzleme: DFS ile bağlantılı bileşenleri numaralamama



bil[0] = -1: **yeni bileşen**. say++ eski değeri (0) verir, say 1 olur; dfs(0, 0) çağrılır.

Adım 3 / 18

```

1 int bileşenler(int[] bil) {
2   Arrays.fill(bil, -1);
3   int say = 0;
4   for (int s = 0; s < n; s++)
5     if (bil[s] == -1) dfs(s, say++, bil);
6   return say;
7 }
8 void dfs(int v, int c, int[] bil) {
9   bil[v] = c;
10  for (int u : komşu.get(v))
11    if (bil[u] == -1) dfs(u, c, bil);
12 }

```

DEĞİŞKENLER

s = 0

say = 1

bil = [-1, -1, -1, -1, -1, -1]

ÇAĞRI YIĞINI

bileşenler(bil)

main()

Kod İzleme: DFS ile bağlantılı bileşenleri numaralama



bil[0] = 0: 0, 0 numaralı bileşene boyandı.

Adım 4 / 18

```

1 int bileşenler(int[] bil) {
2   Arrays.fill(bil, -1);
3   int say = 0;
4   for (int s = 0; s < n; s++)
5     if (bil[s] == -1) dfs(s, say++, bil);
6   return say;
7 }
8 void dfs(int v, int c, int[] bil) {
9   ► bil[v] = c;
10  for (int u : komşu.get(v))
11    if (bil[u] == -1) dfs(u, c, bil);
12 }

```

DEĞİŞKENLER

s = 0

say = 1

v = 0

c = 0

bil = [0, -1, -1, -1, -1, -1]

ÇAĞRI YIĞINI

dfs(0, 0)

bileşenler(bil)

main()

Kod İzleme: DFS ile bağlantılı bileşenleri numaralama



bil[1] = 0: 1, 0 numaralı bileşene boyandı.

Adım 5 / 18

```

1 int bileşenler(int[] bil) {
2     Arrays.fill(bil, -1);
3     int say = 0;
4     for (int s = 0; s < n; s++)
5         if (bil[s] == -1) dfs(s, say++, bil);
6     return say;
7 }
8 void dfs(int v, int c, int[] bil) {
9     ► bil[v] = c;
10    for (int u : komşu.get(v))
11        if (bil[u] == -1) dfs(u, c, bil);
12 }

```

DEĞİŞKENLER

s = 0

say = 1

v = 1

c = 0

bil = [0, 0, -1, -1, -1, -1]

ÇAĞRI YIĞINI

dfs(1, 0)

dfs(0, 0)

bileşenler(bil)

main()

Kod İzleme: DFS ile bağlantılı bileşenleri numaralama



Komşu 0 zaten boyalı (bil = 0) → atla. Her kenara iki kez bakılır: toplam $O(n + m)$.

Adım 6 / 18

```

1 int bileşenler(int[] bil) {
2   Arrays.fill(bil, -1);
3   int say = 0;
4   for (int s = 0; s < n; s++)
5     if (bil[s] == -1) dfs(s, say++, bil);
6   return say;
7 }
8 void dfs(int v, int c, int[] bil) {
9   bil[v] = c;
10  for (int u : komşu.get(v))
11    if (bil[u] == -1) dfs(u, c, bil);
12 }

```

DEĞİŞKENLER

s = 0

say = 1

v = 1

c = 0

u = 0

bil = [0, 0, -1, -1, -1, -1]

ÇAĞRI YIĞINI

dfs(1, 0)

dfs(0, 0)

bileşenler(bil)

main()

Kod İzleme: DFS ile bağlantılı bileşenleri numaralama



bil[2] = 0: 2, 0 numaralı bileşene boyandı.

Adım 7 / 18

```

1 int bileşenler(int[] bil) {
2   Arrays.fill(bil, -1);
3   int say = 0;
4   for (int s = 0; s < n; s++)
5     if (bil[s] == -1) dfs(s, say++, bil);
6   return say;
7 }
8 void dfs(int v, int c, int[] bil) {
9   ► bil[v] = c;
10  for (int u : komşu.get(v))
11    if (bil[u] == -1) dfs(u, c, bil);
12 }

```

DEĞİŞKENLER

s = 0

say = 1

v = 2

c = 0

bil = [0, 0, 0, -1, -1, -1]

ÇAĞRI YIĞINI

dfs(2, 0)

dfs(0, 0)

bileşenler(bil)

main()

Kod İzleme: DFS ile bağlantılı bileşenleri numaralama



Komşu 0 zaten boyalı (bil = 0) → atla. Her kenara iki kez bakılır: toplam $O(n + m)$.

Adım 8 / 18

```

1 int bileşenler(int[] bil) {
2   Arrays.fill(bil, -1);
3   int say = 0;
4   for (int s = 0; s < n; s++)
5     if (bil[s] == -1) dfs(s, say++, bil);
6   return say;
7 }
8 void dfs(int v, int c, int[] bil) {
9   bil[v] = c;
10  for (int u : komşu.get(v))
11    if (bil[u] == -1) dfs(u, c, bil);
12 }

```

DEĞİŞKENLER

s = 0

say = 1

v = 2

c = 0

u = 0

bil = [0, 0, 0, -1, -1, -1]

ÇAĞRI YIĞINI

dfs(2, 0)

dfs(0, 0)

bileşenler(bil)

main()

Kod İzleme: DFS ile bağlantılı bileşenleri numaralamama



bil[1] = 0: 1 önceki bir DFS'te zaten boyandı → atla.

Adım 9 / 18

```

1 int bileşenler(int[] bil) {
2   Arrays.fill(bil, -1);
3   int say = 0;
4   for (int s = 0; s < n; s++)
5     if (bil[s] == -1) dfs(s, say++, bil);
6   return say;
7 }
8 void dfs(int v, int c, int[] bil) {
9   bil[v] = c;
10  for (int u : komşu.get(v))
11    if (bil[u] == -1) dfs(u, c, bil);
12 }

```

DEĞİŞKENLER

s = 1 say = 1

bil = [0, 0, 0, -1, -1, -1]

ÇAĞRI YIĞINI

bileşenler(bil)

main()

Kod İzleme: DFS ile bağlantılı bileşenleri numaralama



bil[2] = 0: 2 önceki bir DFS'te zaten boyandı → atla.

Adım 10 / 18

```

1 int bileşenler(int[] bil) {
2   Arrays.fill(bil, -1);
3   int say = 0;
4   for (int s = 0; s < n; s++)
5     if (bil[s] == -1) dfs(s, say++, bil);
6   return say;
7 }
8 void dfs(int v, int c, int[] bil) {
9   bil[v] = c;
10  for (int u : komşu.get(v))
11    if (bil[u] == -1) dfs(u, c, bil);
12 }

```

DEĞİŞKENLER

s = 2

say = 1

bil = [0, 0, 0, -1, -1, -1]

ÇAĞRI YIĞINI

bileşenler(bil)

main()

Kod İzleme: DFS ile bağlantılı bileşenleri numaralamama



bil[3] = -1: **yeni bileşen.** say++ eski değeri (1) verir, say 2 olur; dfs(3, 1) çağrılır.

Adım 11 / 18

```

1 int bileşenler(int[] bil) {
2   Arrays.fill(bil, -1);
3   int say = 0;
4   for (int s = 0; s < n; s++)
5     if (bil[s] == -1) dfs(s, say++, bil);
6   return say;
7 }
8 void dfs(int v, int c, int[] bil) {
9   bil[v] = c;
10  for (int u : komşu.get(v))
11    if (bil[u] == -1) dfs(u, c, bil);
12 }

```

DEĞİŞKENLER

s = 3

say = 2

bil = [0, 0, 0, -1, -1, -1]

ÇAĞRI YIĞINI

bileşenler(bil)

main()

Kod İzleme: DFS ile bağlantılı bileşenleri numaralama



bil[3] = 1: 3, 1 numaralı bileşene boyandı.

Adım 12 / 18

```

1 int bileşenler(int[] bil) {
2     Arrays.fill(bil, -1);
3     int say = 0;
4     for (int s = 0; s < n; s++)
5         if (bil[s] == -1) dfs(s, say++, bil);
6     return say;
7 }
8 void dfs(int v, int c, int[] bil) {
9     ► bil[v] = c;
10    for (int u : komşu.get(v))
11        if (bil[u] == -1) dfs(u, c, bil);
12 }

```

DEĞİŞKENLER

s = 3

say = 2

v = 3

c = 1

bil = [0, 0, 0, 1, -1, -1]

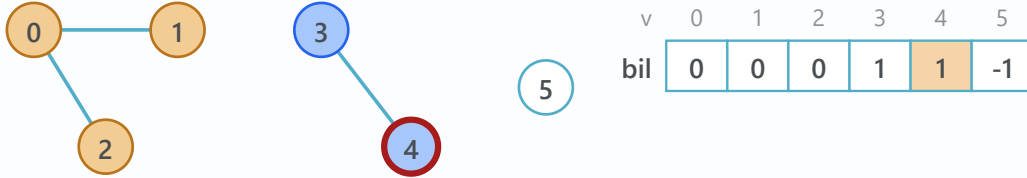
ÇAĞRI YIĞINI

dfs(3, 1)

bileşenler(bil)

main()

Kod İzleme: DFS ile bağlantılı bileşenleri numaralama



bil[4] = 1: 4, 1 numaralı bileşene boyandı.

Adım 13 / 18

```

1 int bileşenler(int[] bil) {
2     Arrays.fill(bil, -1);
3     int say = 0;
4     for (int s = 0; s < n; s++)
5         if (bil[s] == -1) dfs(s, say++, bil);
6     return say;
7 }
8 void dfs(int v, int c, int[] bil) {
9     ► bil[v] = c;
10    for (int u : komşu.get(v))
11        if (bil[u] == -1) dfs(u, c, bil);
12 }

```

DEĞİŞKENLER

s = 3

say = 2

v = 4

c = 1

bil = [0, 0, 0, 1, 1, -1]

ÇAĞRI YIĞINI

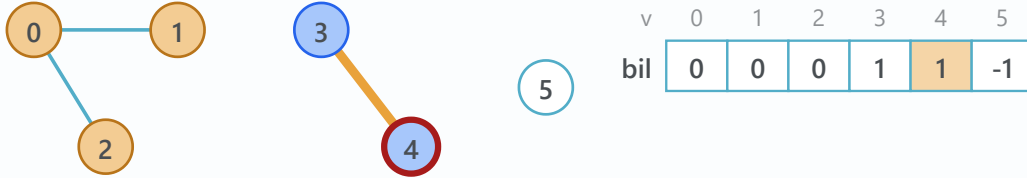
dfs(4, 1)

dfs(3, 1)

bileşenler(bil)

main()

Kod İzleme: DFS ile bağlantılı bileşenleri numaralama



Komşu 3 zaten boyalı (bil = 1) → atla. Her kenara iki kez bakılır: toplam $O(n + m)$.

Adım 14 / 18

```

1 int bileşenler(int[] bil) {
2     Arrays.fill(bil, -1);
3     int say = 0;
4     for (int s = 0; s < n; s++)
5         if (bil[s] == -1) dfs(s, say++, bil);
6     return say;
7 }
8 void dfs(int v, int c, int[] bil) {
9     bil[v] = c;
10    for (int u : komşu.get(v))
11        if (bil[u] == -1) dfs(u, c, bil);
12 }

```

DEĞİŞKENLER

s = 3

say = 2

v = 4

c = 1

u = 3

bil = [0, 0, 0, 1, 1, -1]

ÇAĞRI YIĞINI

dfs(4, 1)

dfs(3, 1)

bileşenler(bil)

main()

Kod İzleme: DFS ile bağlantılı bileşenleri numaralama



bil[4] = 1: 4 önceki bir DFS'te zaten boyandı → atla.

Adım 15 / 18

```

1 int bilesenler(int[] bil) {
2   Arrays.fill(bil, -1);
3   int say = 0;
4   for (int s = 0; s < n; s++)
5     if (bil[s] == -1) dfs(s, say++, bil);
6   return say;
7 }
8 void dfs(int v, int c, int[] bil) {
9   bil[v] = c;
10  for (int u : komsu.get(v))
11    if (bil[u] == -1) dfs(u, c, bil);
12 }

```

DEĞİŞKENLER

s = 4 say = 2

bil = [0, 0, 0, 1, 1, -1]

ÇAĞRI YIĞINI

bilesenler(bil)

main()

Kod İzleme: DFS ile bağlantılı bileşenleri numaralama



bil[5] = -1: **yeni bileşen.** say++ eski değeri (2) verir, say 3 olur; dfs(5, 2) çağrılır.

Adım 16 / 18

```

1 int bileşenler(int[] bil) {
2   Arrays.fill(bil, -1);
3   int say = 0;
4   ▶ for (int s = 0; s < n; s++)
5     ▶ if (bil[s] == -1) dfs(s, say++, bil);
6   return say;
7 }
8 void dfs(int v, int c, int[] bil) {
9   bil[v] = c;
10  for (int u : komşu.get(v))
11    if (bil[u] == -1) dfs(u, c, bil);
12 }

```

DEĞİŞKENLER

s = 5

say = 3

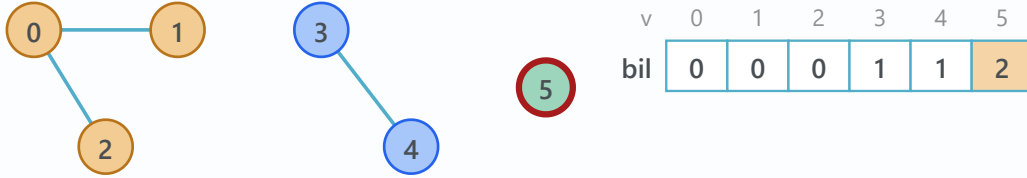
bil = [0, 0, 0, 1, 1, -1]

ÇAĞRI YIĞINI

bileşenler(bil)

main()

Kod İzleme: DFS ile bağlantılı bileşenleri numaralama



bil[5] = 2: 5, 2 numaralı bileşene boyandı.

Adım 17 / 18

```

1 int bileşenler(int[] bil) {
2     Arrays.fill(bil, -1);
3     int say = 0;
4     for (int s = 0; s < n; s++)
5         if (bil[s] == -1) dfs(s, say++, bil);
6     return say;
7 }
8 void dfs(int v, int c, int[] bil) {
9     ► bil[v] = c;
10    for (int u : komşu.get(v))
11        if (bil[u] == -1) dfs(u, c, bil);
12 }

```

DEĞİŞKENLER

s = 5

say = 3

v = 5

c = 2

bil = [0, 0, 0, 1, 1, 2]

ÇAĞRI YIĞINI

dfs(5, 2)

bileşenler(bil)

main()

Kod İzleme: DFS ile bağlantılı bileşenleri numaralamama



3 bileşen: {0, 1, 2}, {3, 4}, {5}. Aynı bil değeri = aynı bileşen; `bil[u] == bil[v]` sorgusu artık O(1).

Adım 18 / 18

```

1 int bileşenler(int[] bil) {
2   Arrays.fill(bil, -1);
3   int say = 0;
4   for (int s = 0; s < n; s++)
5     if (bil[s] == -1) dfs(s, say++, bil);
6   return say;
7 }
8 void dfs(int v, int c, int[] bil) {
9   bil[v] = c;
10  for (int u : komşu.get(v))
11    if (bil[u] == -1) dfs(u, c, bil);
12 }
```

DEĞİŞKENLER

say = 3

dönüş = 3

bil = [0, 0, 0, 1, 1, 2]

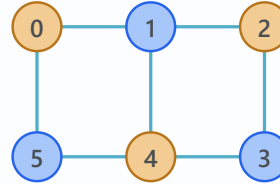
ÇAĞRI YIĞINI

bileşenler(bil)

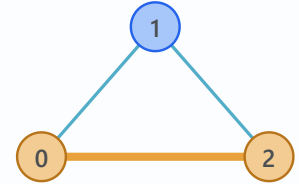
main()

İki Parçalı (Bipartite) Çizge Testi

- **İki parçalı** (bipartite): düğümler iki gruba ayrılır, her kenar gruplar *arasında*.
- Test = **BFS ile 2-renkleme**: komşuya zıt renk ver; aynı renkli iki komşu → değil.
- Eşdeğer: **tek uzunlukta döngü yok**.



iki parçalı ✓ (çift döngüler)



üçgen X: 0 ve 2 aynı renk

```
// renk[] = -1 ile başlar; s'den BFS
renk[s] = 0;  q.add(s);
while (!q.isEmpty()) {
    int w = q.poll();
    for (int u : komsu.get(w))
        if (renk[u] == -1) {
            renk[u] = 1 - renk[w];  q.add(u);
        } else if (renk[u] == renk[w]) return false;
}
return true;
```

BFS ile aynı maliyet: $O(n + m)$. Kullanım: öğrenci-proje eşleştirme, iki vardiyalı çizelgeleme, çakışmasız gruplama.

Topolojik Sıralama: Yalnızca DAG

- Bir yönlü çizgenin düğümlerini, her $u \rightarrow v$ kenarında u , v 'den önce gelecek biçimde sıralar.
- Yalnızca **yönlü döngüsüz çizgelerde** (DAG) mümkündür.
- Sonuç tek olmak zorunda değildir; aynı anda seçilebilen birden çok giriş derecesi 0 düğüm olabilir.
- Kullanım: ders ön koşulları, görev bağımlılıkları, derleme ve iş akışı planlama.

Çizgede döngü varsa hiçbir topolojik sıra yoktur: döngüdeki her görev başka bir görevin önce yapılmasını bekler.

Zaman

$O(n + m)$

Kahn(G):

```
q = girişDerecesi 0 olan düğümler
sonuc = []
while q boş değil:
    u = q'dan çıkar
    sonuc'a u ekle
    for u'dan çıkan her (u, v):
        girişDerecesi[v]--
        if girişDerecesi[v] == 0:
            q'ya v ekle
if sonuc.size < n:
    döngü var
return sonuc
```

Alternatif

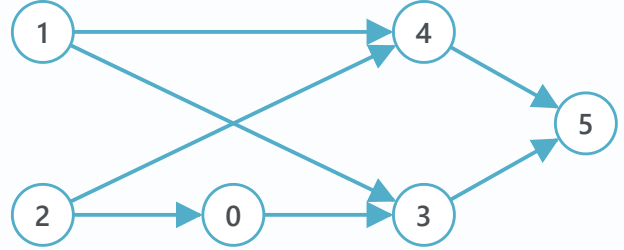
DFS bitiş zamanlarını ters çevirmek; geri kenar bulunursa döngü vardır.

Kod İzleme: Topolojik Sıralama (Kahn)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2     int n = adj.size();
3     int[] giris = new int[n];
4     for (List<Integer> l : adj)
5         for (int v : l) giris[v]++;
6     Queue<Integer> q = new ArrayDeque<>();
7     for (int v = 0; v < n; v++)
8         if (giris[v] == 0) q.add(v);
9     List<Integer> sonuc = new ArrayList<>();
10    while (!q.isEmpty()) {
11        int u = q.poll();
12        sonuc.add(u);
13        for (int v : adj.get(u))
14            if (--giris[v] == 0) q.add(v);
15    }
16    return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	0	0	0	0	0	0

q

(boş)					
-------	--	--	--	--	--

sonuc

[]					
----	--	--	--	--	--

Ders ön koşulu örneği: 6 düğüm, 7 yönlü kenar (u → v: u, v'den önce). **giris[]** sıfırlarla başlar.

Adım 1 / 11

DEĞİŞKENLER

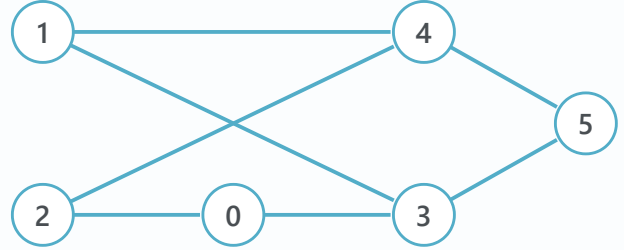
n = 6

Kod İzleme: Topolojik Sıralama (Kahn)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   ▶ for (List<Integer> l : adj)
5   ▶   for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    int u = q.poll();
12    sonuc.add(u);
13    for (int v : adj.get(u))
14      if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	0	0	0	1	0	0

q

(boş)					
-------	--	--	--	--	--

sonuc

[]					
----	--	--	--	--	--

0'dan çıkan kenarlar 0→3: hedeflerin giriş derecesi birer artar.

Adım 2 / 11

DEĞİŞKENLER

n = 6

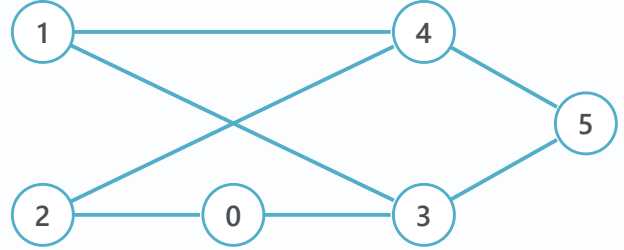
l = adj[0]

Kod İzleme: Topolojik Sıralama (Kahn)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   ▶ for (List<Integer> l : adj)
5   ▶   for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    int u = q.poll();
12    sonuc.add(u);
13    for (int v : adj.get(u))
14      if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	0	0	0	2	1	0

q

(boş)					
-------	--	--	--	--	--

sonuc

[]					
----	--	--	--	--	--

1'den çıkan kenarlar 1→3, 1→4: hedeflerin giriş derecesi birer artar.

Adım 3 / 11

DEĞİŞKENLER

n = 6

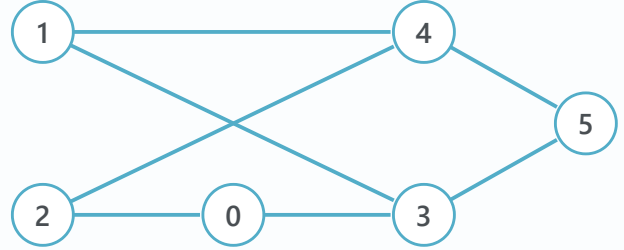
l = adj[1]

Kod İzleme: Topolojik Sıralama (Kahn)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   ▶ for (List<Integer> l : adj)
5   ▶   for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    int u = q.poll();
12    sonuc.add(u);
13    for (int v : adj.get(u))
14      if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	1	0	0	2	2	0

q

(boş)					
-------	--	--	--	--	--

sonuc

[]					
----	--	--	--	--	--

2'den çıkan kenarlar 2→0, 2→4: hedeflerin giriş derecesi birer artar.

Adım 4 / 11

DEĞİŞKENLER

n = 6

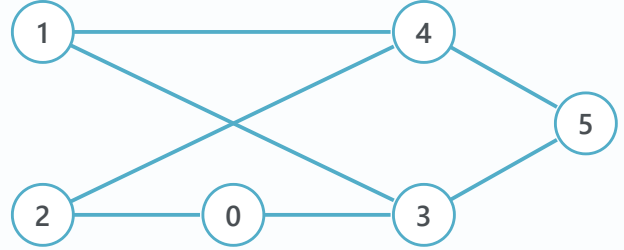
l = adj[2]

Kod İzleme: Topolojik Sıralama (Kahn)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   ▶ for (List<Integer> l : adj)
5   ▶   for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    int u = q.poll();
12    sonuc.add(u);
13    for (int v : adj.get(u))
14      if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	1	0	0	2	2	1

q

(boş)					
-------	--	--	--	--	--

sonuc

[]					
----	--	--	--	--	--

3'ten çıkan kenarlar 3→5: hedeflerin giriş derecesi birer artar.

Adım 5 / 11

DEĞİŞKENLER

n = 6

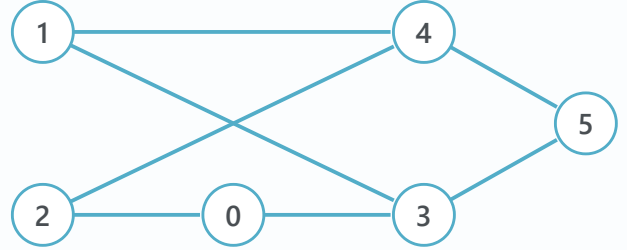
l = adj[3]

Kod İzleme: Topolojik Sıralama (Kahn)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   ▶ for (List<Integer> l : adj)
5   ▶   for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    int u = q.poll();
12    sonuc.add(u);
13    for (int v : adj.get(u))
14      if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	1	0	0	2	2	2

q

(boş)					
-------	--	--	--	--	--

sonuc

[]					
----	--	--	--	--	--

4'ten çıkan kenarlar 4→5: hedeflerin giriş derecesi birer artar.

Adım 6 / 11

DEĞİŞKENLER

n = 6

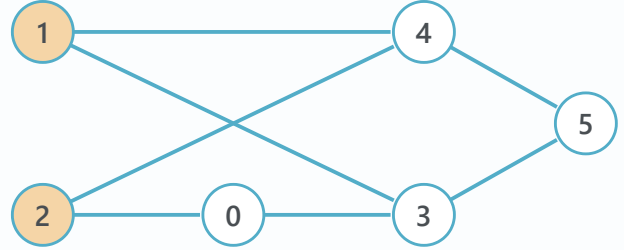
l = adj[4]

Kod İzleme: Topolojik Sıralama (Kahn)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   for (List<Integer> l : adj)
5     for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    int u = q.poll();
12    sonuc.add(u);
13    for (int v : adj.get(u))
14      if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	1	0	0	2	2	2

q

1	2				
---	---	--	--	--	--

sonuc

[]					
----	--	--	--	--	--

Giriş derecesi 0 olanlar (1, 2) hiçbir şeye bağımlı değil → kuyruğa. (5'in listesi boş olduğundan sayma turu bir şey değiştirmede.)

Adım 7 / 11

DEĞİŞKENLER

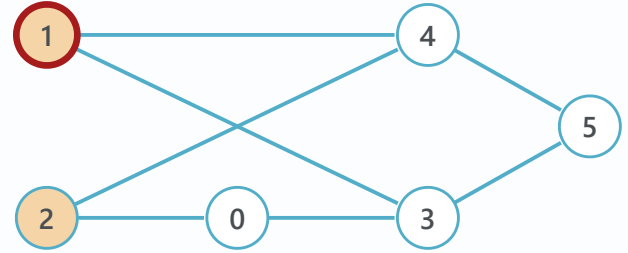
n = 6

Kod İzleme: Topolojik Sıralama (Kahn)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   for (List<Integer> l : adj)
5     for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    ▶ int u = q.poll();
12    ▶ sonuc.add(u);
13    for (int v : adj.get(u))
14      if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	1	0	0	2	2	2

q

2					
---	--	--	--	--	--

sonuc

1					
---	--	--	--	--	--

1 kuyruktan çıkar ve sonuca eklenir: ön koşullarının hepsi zaten sonuçta.

Adım 8 / 11

DEĞİŞKENLER

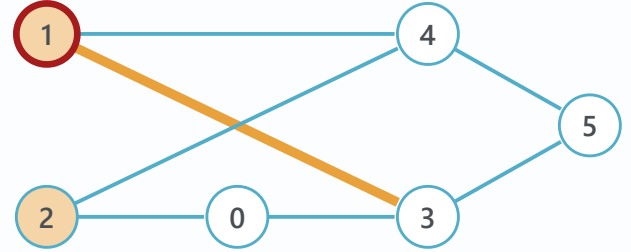
u = 1

Kod İzleme: Topolojik Sıralama (Kahn)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   for (List<Integer> l : adj)
5     for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    int u = q.poll();
12    sonuc.add(u);
13    for (int v : adj.get(u))
14      if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	1	0	0	1	2	2

q

2					
---	--	--	--	--	--

sonuc

1					
---	--	--	--	--	--

1→3 silinmiş sayılır: giris[3] = 1; 3 hâlâ başka bir ön koşul bekliyor.

Adım 9 / 11

DEĞİŞKENLER

u = 1

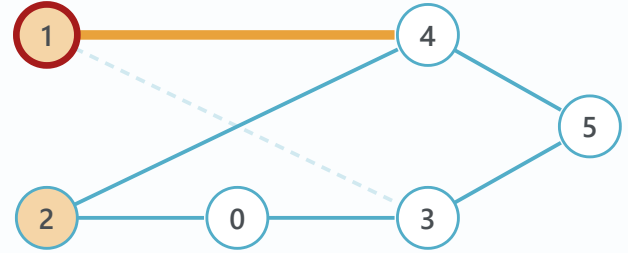
v = 3

Kod İzleme: Topolojik Sıralama (Kahn)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   for (List<Integer> l : adj)
5     for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    int u = q.poll();
12    sonuc.add(u);
13    for (int v : adj.get(u))
14      if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	1	0	0	1	1	2

q

2					
---	--	--	--	--	--

sonuc

1					
---	--	--	--	--	--

1→4 silinmiş sayılır: giris[4] = 1; 4 hâlâ başka bir ön koşul bekliyor.

Adım 10 / 11

DEĞİŞKENLER

u = 1

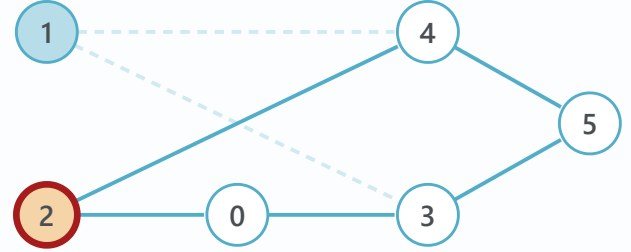
v = 4

Kod İzleme: Topolojik Sıralama (Kahn)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   for (List<Integer> l : adj)
5     for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    ▶ int u = q.poll();
12    ▶ sonuc.add(u);
13    for (int v : adj.get(u))
14      if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	1	0	0	1	1	2

q

(boş)					
-------	--	--	--	--	--

sonuc

1	2				
---	---	--	--	--	--

2 kuyruktan çıkar ve sonuca eklenir: ön koşullarının hepsi zaten sonuçta.

Adım 11 / 11

DEĞİŞKENLER

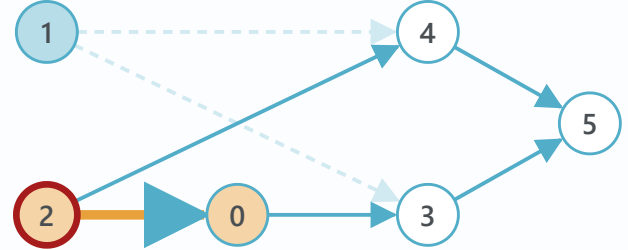
u = 2

Kod İzleme: Topolojik Sıralama (Kahn) (devam)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   for (List<Integer> l : adj)
5     for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    int u = q.poll();
12    sonuc.add(u);
13    for (int v : adj.get(u))
14      if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	0	0	0	1	1	2

q

0					
---	--	--	--	--	--

sonuc

1	2				
---	---	--	--	--	--

2→0 silinmiş sayılır: $\text{giris}[0] = 0 \rightarrow 0$ artık hazır, kuyruğa eklenir.

Adım 1 / 10

DEĞİŞKENLER

u = 2

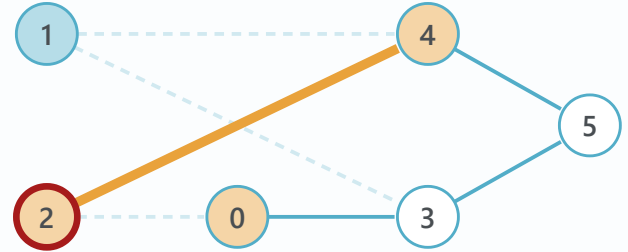
v = 0

Kod İzleme: Topolojik Sıralama (Kahn) (devam)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   for (List<Integer> l : adj)
5     for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    int u = q.poll();
12    sonuc.add(u);
13    for (int v : adj.get(u))
14      if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	0	0	0	1	0	2

q

0	4				
---	---	--	--	--	--

sonuc

1	2				
---	---	--	--	--	--

2→4 silinmiş sayılır: $giris[4] = 0 \rightarrow 4$ artık hazır, kuyruğa eklenir.

Adım 2 / 10

DEĞİŞKENLER

u = 2

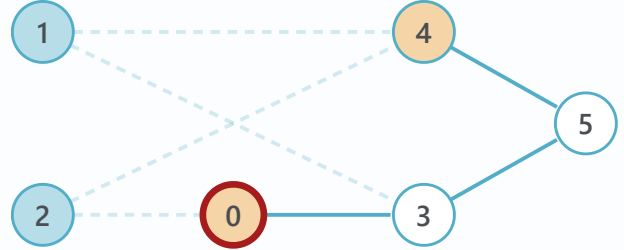
v = 4

Kod İzleme: Topolojik Sıralama (Kahn) (devam)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   for (List<Integer> l : adj)
5     for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    ▶ int u = q.poll();
12    ▶ sonuc.add(u);
13    for (int v : adj.get(u))
14      if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	0	0	0	1	0	2

q

4					
---	--	--	--	--	--

sonuc

1	2	0			
---	---	---	--	--	--

0 kuyruktan çıkar ve sonuca eklenir: ön koşullarının hepsi zaten sonuçta.

Adım 3 / 10

DEĞİŞKENLER

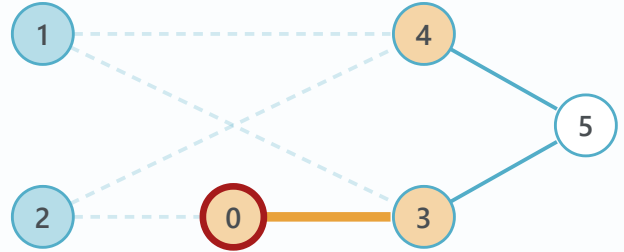
u = 0

Kod İzleme: Topolojik Sıralama (Kahn) (devam)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   for (List<Integer> l : adj)
5     for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    int u = q.poll();
12    sonuc.add(u);
13    for (int v : adj.get(u))
14      ▶ if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	0	0	0	0	0	2

q

4	3				
---	---	--	--	--	--

sonuc

1	2	0			
---	---	---	--	--	--

0→3 silinmiş sayılır: $\text{giris}[3] = 0 \rightarrow 3$ artık hazır, kuyruğa eklenir.

Adım 4 / 10

DEĞİŞKENLER

u = 0

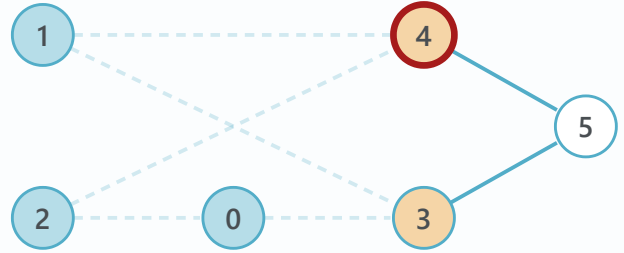
v = 3

Kod İzleme: Topolojik Sıralama (Kahn) (devam)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   for (List<Integer> l : adj)
5     for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    ▶ int u = q.poll();
12    ▶ sonuc.add(u);
13    for (int v : adj.get(u))
14      if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	0	0	0	0	0	2

q

3					
---	--	--	--	--	--

sonuc

1	2	0	4		
---	---	---	---	--	--

4 kuyruktan çıkar ve sonuca eklenir: ön koşullarının hepsi zaten sonuçta.

Adım 5 / 10

DEĞİŞKENLER

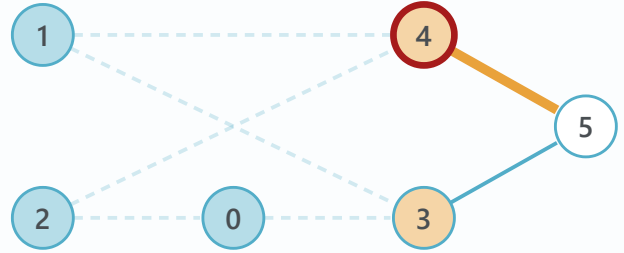
u = 4

Kod İzleme: Topolojik Sıralama (Kahn) (devam)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   for (List<Integer> l : adj)
5     for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    int u = q.poll();
12    sonuc.add(u);
13    for (int v : adj.get(u))
14      ▶ if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	0	0	0	0	0	1

q

3					
---	--	--	--	--	--

sonuc

1	2	0	4		
---	---	---	---	--	--

4→5 silinmiş sayılır: giris[5] = 1; 5 hâlâ başka bir ön koşul bekliyor.

Adım 6 / 10

DEĞİŞKENLER

u = 4

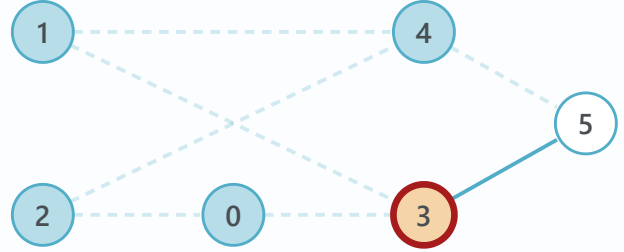
v = 5

Kod İzleme: Topolojik Sıralama (Kahn) (devam)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   for (List<Integer> l : adj)
5     for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    ▶ int u = q.poll();
12    ▶ sonuc.add(u);
13    for (int v : adj.get(u))
14      if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	0	0	0	0	0	1

q

(boş)					
-------	--	--	--	--	--

sonuc

1	2	0	4	3	
---	---	---	---	---	--

3 kuyruktan çıkar ve sonuca eklenir: ön koşullarının hepsi zaten sonuçta.

Adım 7 / 10

DEĞİŞKENLER

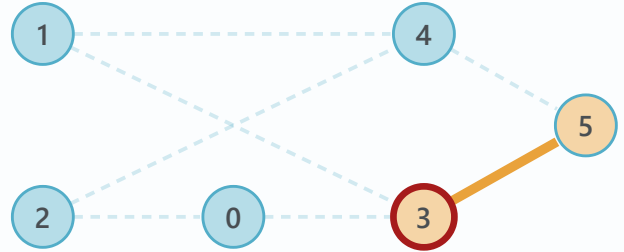
u = 3

Kod İzleme: Topolojik Sıralama (Kahn) (devam)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   for (List<Integer> l : adj)
5     for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    int u = q.poll();
12    sonuc.add(u);
13    for (int v : adj.get(u))
14      ▶ if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	0	0	0	0	0	0

q

5					
---	--	--	--	--	--

sonuc

1	2	0	4	3	
---	---	---	---	---	--

3→5 silinmiş sayılır: $giris[5] = 0 \rightarrow 5$ artık hazır, kuyruğa eklenir.

Adım 8 / 10

DEĞİŞKENLER

u = 3

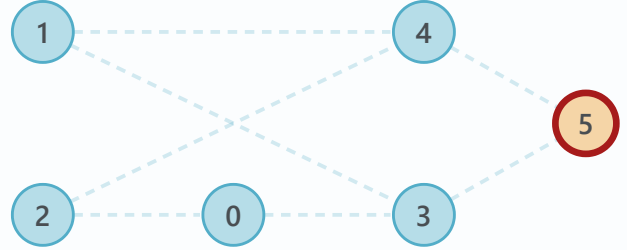
v = 5

Kod İzleme: Topolojik Sıralama (Kahn) (devam)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   for (List<Integer> l : adj)
5     for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    ▶ int u = q.poll();
12    ▶ sonuc.add(u);
13    for (int v : adj.get(u))
14      if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	0	0	0	0	0	0

q

(boş)					
-------	--	--	--	--	--

sonuc

1	2	0	4	3	5
---	---	---	---	---	---

5 kuyruktan çıkar ve sonuca eklenir: ön koşullarının hepsi zaten sonuçta.

Adım 9 / 10

DEĞİŞKENLER

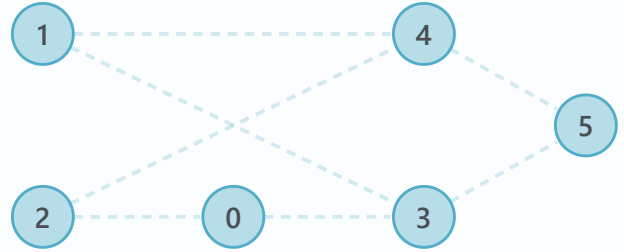
u = 5

Kod İzleme: Topolojik Sıralama (Kahn) (devam)

```

1 List<Integer> kahn(List<List<Integer>> adj) {
2   int n = adj.size();
3   int[] giris = new int[n];
4   for (List<Integer> l : adj)
5     for (int v : l) giris[v]++;
6   Queue<Integer> q = new ArrayDeque<>();
7   for (int v = 0; v < n; v++)
8     if (giris[v] == 0) q.add(v);
9   List<Integer> sonuc = new ArrayList<>();
10  while (!q.isEmpty()) {
11    int u = q.poll();
12    sonuc.add(u);
13    for (int v : adj.get(u))
14      if (--giris[v] == 0) q.add(v);
15  }
16  return sonuc; // boyut < n → döngü
17 }

```



giris[] (giriş derecesi)

	0	1	2	3	4	5
giris	0	0	0	0	0	0

q

(boş)					
-------	--	--	--	--	--

sonuc

1	2	0	4	3	5
---	---	---	---	---	---

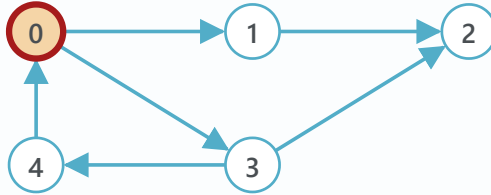
Kuyruk boş; sonuç **1 2 0 4 3 5** ve boyutu $6 = n \rightarrow$ döngü yok. Her kenar bir kez azaltıldı: **$O(n + m)$** .

Adım 10 / 10

DEĞİŞKENLER

—

Kod İzleme: yönlü çizgede döngü (3 renkli DFS)



v	0	1	2	3	4
renk	1	0	0	0	0

0 beyaz · 1 gri · 2 siyah

renk[0] = 1 (gri): 0 şu anki DFS yolunda.

Adım 1 / 15

```

1 // renk: 0 beyaz, 1 gri (yolda), 2 siyah
2 boolean donguVar(int v, int[] renk) {
3     ▶ renk[v] = 1;
4     for (int u : komsu.get(v)) {
5         if (renk[u] == 1) return true; // geri kenar
6         if (renk[u] == 0 && donguVar(u, renk))
7             return true;
8     }
9     renk[v] = 2;
10    return false;
11 }
  
```

DEĞİŞKENLER

v = 0

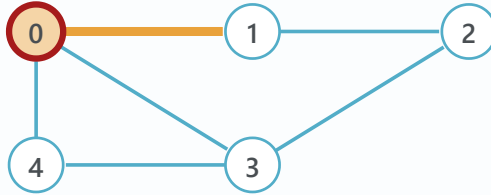
renk = [1, 0, 0, 0, 0]

ÇAĞRI YIĞINI

donguVar(0)

main()

Kod İzleme: yönlü çizgede döngü (3 renkli DFS)



v	0	1	2	3	4
renk	1	0	0	0	0

0 beyaz · 1 gri · 2 siyah

Kenar 0 → 1: 1 beyaz → özyinelemeli çağrı.

Adım 2 / 15

```

1 // renk: 0 beyaz, 1 gri (yolda), 2 siyah
2 boolean donguVar(int v, int[] renk) {
3     renk[v] = 1;
4     for (int u : komsu.get(v)) {
5         if (renk[u] == 1) return true; // geri kenar
6         if (renk[u] == 0 && donguVar(u, renk))
7             return true;
8     }
9     renk[v] = 2;
10    return false;
11 }
  
```

DEĞİŞKENLER

v = 0

u = 1

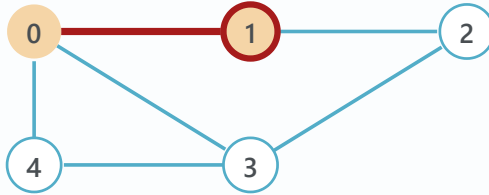
renk = [1, 0, 0, 0, 0]

ÇAĞRI YIĞINI

donguVar(0)

main()

Kod İzleme: yönlü çizgede döngü (3 renkli DFS)



v	0	1	2	3	4
renk	1	1	0	0	0

0 beyaz · 1 gri · 2 siyah

renk[1] = 1 (gri): 1 şu anki DFS yolunda.

Adım 3 / 15

```

1 // renk: 0 beyaz, 1 gri (yolda), 2 siyah
2 boolean donguVar(int v, int[] renk) {
3     ▶ renk[v] = 1;
4     for (int u : komsu.get(v)) {
5         if (renk[u] == 1) return true; // geri kenar
6         if (renk[u] == 0 && donguVar(u, renk))
7             return true;
8     }
9     renk[v] = 2;
10    return false;
11 }

```

DEĞİŞKENLER

v = 1

renk = [1, 1, 0, 0, 0]

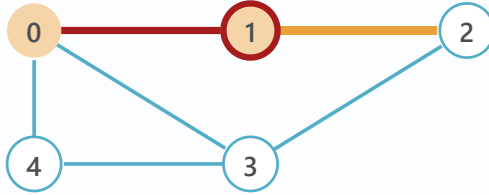
ÇAĞRI YIĞINI

donguVar(1)

donguVar(0)

main()

Kod İzleme: yönlü çizgede döngü (3 renkli DFS)



v	0	1	2	3	4
renk	1	1	0	0	0

0 beyaz · 1 gri · 2 siyah

Kenar 1 → 2: 2 beyaz → özyinelemeli çağrı.

Adım 4 / 15

```

1 // renk: 0 beyaz, 1 gri (yolda), 2 siyah
2 boolean donguVar(int v, int[] renk) {
3     renk[v] = 1;
4     for (int u : komsu.get(v)) {
5         if (renk[u] == 1) return true; // geri kenar
6         if (renk[u] == 0 && donguVar(u, renk))
7             return true;
8     }
9     renk[v] = 2;
10    return false;
11 }

```

DEĞİŞKENLER

v = 1

u = 2

renk = [1, 1, 0, 0, 0]

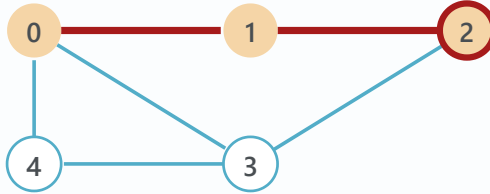
ÇAĞRI YIĞINI

donguVar(1)

donguVar(0)

main()

Kod İzleme: yönlü çizgede döngü (3 renkli DFS)



v	0	1	2	3	4
renk	1	1	1	0	0

0 beyaz · 1 gri · 2 siyah

renk[2] = 1 (gri): 2 şu anki DFS yolunda.

Adım 5 / 15

```

1 // renk: 0 beyaz, 1 gri (yolda), 2 siyah
2 boolean donguVar(int v, int[] renk) {
3     ▶ renk[v] = 1;
4     for (int u : komsu.get(v)) {
5         if (renk[u] == 1) return true; // geri kenar
6         if (renk[u] == 0 && donguVar(u, renk))
7             return true;
8     }
9     renk[v] = 2;
10    return false;
11 }

```

DEĞİŞKENLER

v = 2

renk = [1, 1, 1, 0, 0]

ÇAĞRI YIĞINI

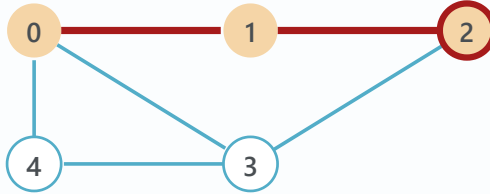
donguVar(2)

donguVar(1)

donguVar(0)

main()

Kod İzleme: yönlü çizgede döngü (3 renkli DFS)



v	0	1	2	3	4
renk	1	1	2	0	0

0 beyaz · 1 gri · 2 siyah

2'nin çıkan kenarı yok: **renk[2] = 2 (siyah)**, false döner; yığından çıkar.

Adım 6 / 15

```

1 // renk: 0 beyaz, 1 gri (yolda), 2 siyah
2 boolean donguVar(int v, int[] renk) {
3     renk[v] = 1;
4     for (int u : komsu.get(v)) {
5         if (renk[u] == 1) return true; // geri kenar
6         if (renk[u] == 0 && donguVar(u, renk))
7             return true;
8     }
9     renk[v] = 2;
10    return false;
11 }

```

DEĞİŞKENLER

v = 2

dönüş = false

renk = [1, 1, 2, 0, 0]

ÇAĞRI YIĞINI

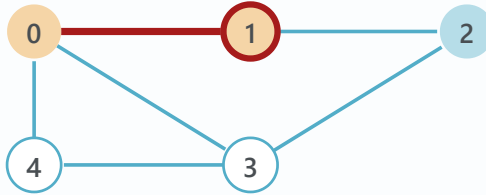
donguVar(2)

donguVar(1)

donguVar(0)

main()

Kod İzleme: yönlü çizgede döngü (3 renkli DFS)



v	0	1	2	3	4
renk	1	2	2	0	0

0 beyaz · 1 gri · 2 siyah

1'in tüm komşuları bitti: **renk[1] = 2 (siyah)**, false döner; yığından çıkar.

Adım 7 / 15

```

1 // renk: 0 beyaz, 1 gri (yolda), 2 siyah
2 boolean donguVar(int v, int[] renk) {
3     renk[v] = 1;
4     for (int u : komsu.get(v)) {
5         if (renk[u] == 1) return true; // geri kenar
6         if (renk[u] == 0 && donguVar(u, renk))
7             return true;
8     }
9     renk[v] = 2;
10    return false;
11 }
```

DEĞİŞKENLER

v = 1 dönüş = false

renk = [1, 2, 2, 0, 0]

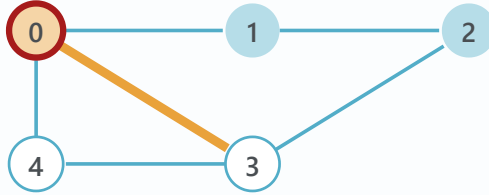
ÇAĞRI YIĞINI

donguVar(1)

donguVar(0)

main()

Kod İzleme: yönlü çizgede döngü (3 renkli DFS)



v	0	1	2	3	4
renk	1	2	2	0	0

0 beyaz · 1 gri · 2 siyah

Kenar 0 → 3: 3 beyaz → özyinelemeli çağrı.

Adım 8 / 15

```

1 // renk: 0 beyaz, 1 gri (yolda), 2 siyah
2 boolean donguVar(int v, int[] renk) {
3     renk[v] = 1;
4     for (int u : komsu.get(v)) {
5         if (renk[u] == 1) return true; // geri kenar
6         if (renk[u] == 0 && donguVar(u, renk))
7             return true;
8     }
9     renk[v] = 2;
10    return false;
11 }

```

DEĞİŞKENLER

v = 0 u = 3

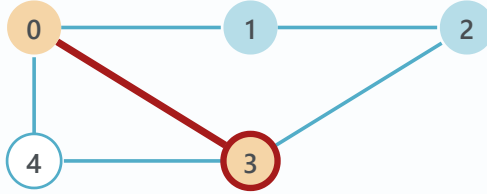
renk = [1, 2, 2, 0, 0]

ÇAĞRI YIĞINI

donguVar(0)

main()

Kod İzleme: yönlü çizgede döngü (3 renkli DFS)



v	0	1	2	3	4
renk	1	2	2	1	0

0 beyaz · 1 gri · 2 siyah

renk[3] = 1 (gri): 3 şu anki DFS yolunda.

Adım 9 / 15

```

1 // renk: 0 beyaz, 1 gri (yolda), 2 siyah
2 boolean donguVar(int v, int[] renk) {
3     renk[v] = 1;
4     for (int u : komsu.get(v)) {
5         if (renk[u] == 1) return true; // geri kenar
6         if (renk[u] == 0 && donguVar(u, renk))
7             return true;
8     }
9     renk[v] = 2;
10    return false;
11 }

```

DEĞİŞKENLER

v = 3

renk = [1, 2, 2, 1, 0]

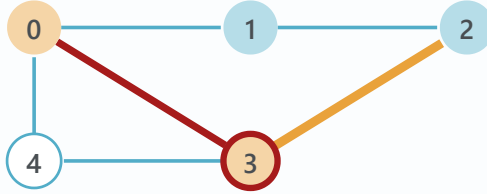
ÇAĞRI YIĞINI

donguVar(3)

donguVar(0)

main()

Kod İzleme: yönlü çizgede döngü (3 renkli DFS)



v	0	1	2	3	4
renk	1	2	2	1	0

0 beyaz · 1 gri · 2 siyah

Kenar 3 → 2: 2 **siyah** (bitmiş). Döngü değil: 2'den geri 3'e yol yok. Sadece gri düğüm döngü demektir.

Adım 10 / 15

```

1 // renk: 0 beyaz, 1 gri (yolda), 2 siyah
2 boolean donguVar(int v, int[] renk) {
3     renk[v] = 1;
4     for (int u : komsu.get(v)) {
5         if (renk[u] == 1) return true; // geri kenar
6         if (renk[u] == 0 && donguVar(u, renk))
7             return true;
8     }
9     renk[v] = 2;
10    return false;
11 }

```

DEĞİŞKENLER

v = 3

u = 2

renk = [1, 2, 2, 1, 0]

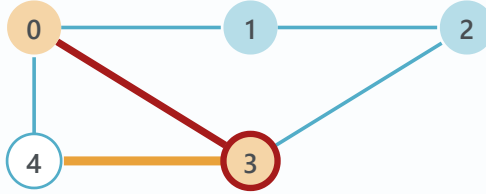
ÇAĞRI YIĞINI

donguVar(3)

donguVar(0)

main()

Kod İzleme: yönlü çizgede döngü (3 renkli DFS)



v	0	1	2	3	4
renk	1	2	2	1	0

0 beyaz · 1 gri · 2 siyah

Kenar 3 → 4: 4 beyaz → özyinelemeli çağrı.

Adım 11 / 15

```

1 // renk: 0 beyaz, 1 gri (yolda), 2 siyah
2 boolean donguVar(int v, int[] renk) {
3     renk[v] = 1;
4     for (int u : komsu.get(v)) {
5         if (renk[u] == 1) return true; // geri kenar
6         if (renk[u] == 0 && donguVar(u, renk))
7             return true;
8     }
9     renk[v] = 2;
10    return false;
11 }

```

DEĞİŞKENLER

v = 3

u = 4

renk = [1, 2, 2, 1, 0]

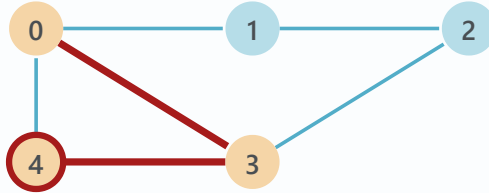
ÇAĞRI YIĞINI

donguVar(3)

donguVar(0)

main()

Kod İzleme: yönlü çizgede döngü (3 renkli DFS)



v	0	1	2	3	4
renk	1	2	2	1	1

0 beyaz · 1 gri · 2 siyah

renk[4] = 1 (gri): 4 şu anki DFS yolunda.

Adım 12 / 15

```

1 // renk: 0 beyaz, 1 gri (yolda), 2 siyah
2 boolean donguVar(int v, int[] renk) {
3     ▶ renk[v] = 1;
4     for (int u : komsu.get(v)) {
5         if (renk[u] == 1) return true; // geri kenar
6         if (renk[u] == 0 && donguVar(u, renk))
7             return true;
8     }
9     renk[v] = 2;
10    return false;
11 }

```

DEĞİŞKENLER

v = 4

renk = [1, 2, 2, 1, 1]

ÇAĞRI YIĞINI

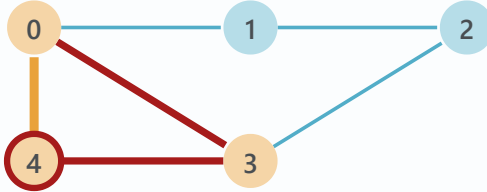
donguVar(4)

donguVar(3)

donguVar(0)

main()

Kod İzleme: yönlü çizgede döngü (3 renkli DFS)



v	0	1	2	3	4
renk	1	2	2	1	1

0 beyaz · 1 gri · 2 siyah

geri kenar → DÖNGÜ

Kenar 4 → 0: 0 **gri**, yani yolda! Bu bir **geri kenar** → döngü 0 → 3 → 4 → 0 bulundu.

Adım 13 / 15

```

1 // renk: 0 beyaz, 1 gri (yolda), 2 siyah
2 boolean donguVar(int v, int[] renk) {
3     renk[v] = 1;
4     for (int u : komsu.get(v)) {
5         ► if (renk[u] == 1) return true; // geri kenar
6         if (renk[u] == 0 && donguVar(u, renk))
7             return true;
8     }
9     renk[v] = 2;
10    return false;
11 }

```

DEĞİŞKENLER

v = 4

u = 0

renk = [1, 2, 2, 1, 1]

ÇAĞRI YIĞINI

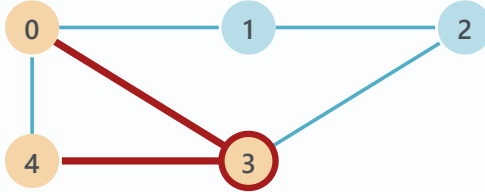
donguVar(4)

donguVar(3)

donguVar(0)

main()

Kod İzleme: yönlü çizgede döngü (3 renkli DFS)



v	0	1	2	3	4
renk	1	2	2	1	1

0 beyaz · 1 gri · 2 siyah

geri kenar → DÖNGÜ

Altaki çağrı **true** döndü → 3 de true döndürür.

Adım 14 / 15

```

1 // renk: 0 beyaz, 1 gri (yolda), 2 siyah
2 boolean donguVar(int v, int[] renk) {
3     renk[v] = 1;
4     for (int u : komsu.get(v)) {
5         if (renk[u] == 1) return true; // geri kenar
6         if (renk[u] == 0 && donguVar(u, renk))
7             return true;
8     }
9     renk[v] = 2;
10    return false;
11 }

```

DEĞİŞKENLER

v = 3 u = 4 dönüş = true

renk = [1, 2, 2, 1, 1]

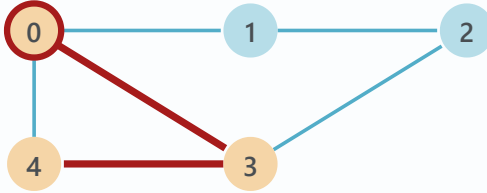
ÇAĞRI YIĞINI

donguVar(3)

donguVar(0)

main()

Kod İzleme: yönlü çizgede döngü (3 renkli DFS)



v	0	1	2	3	4
renk	1	2	2	1	1

0 beyaz · 1 gri · 2 siyah

geri kenar → DÖNGÜ

Altaki çağrı **true** döndü → 0 da true döndürür.

Adım 15 / 15

```

1 // renk: 0 beyaz, 1 gri (yolda), 2 siyah
2 boolean donguVar(int v, int[] renk) {
3     renk[v] = 1;
4     for (int u : komsu.get(v)) {
5         if (renk[u] == 1) return true; // geri kenar
6         if (renk[u] == 0 && donguVar(u, renk))
7             return true;
8     }
9     renk[v] = 2;
10    return false;
11 }

```

DEĞİŞKENLER

v = 0

u = 3

dönüş = true

renk = [1, 2, 2, 1, 1]

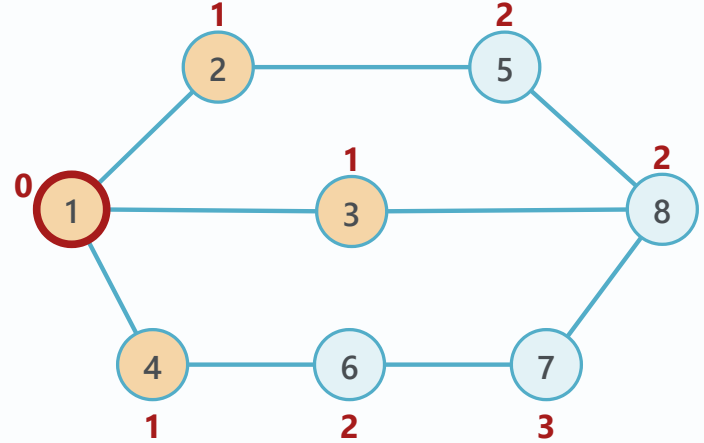
ÇAĞRI YIĞINI

donguVar(0)

main()

Ağırlıksız En Kısa Yol Problemi

- Kaynak düğümden diğer **tüm düğümlere** olan en kısa yolları bulur (tek kaynaklı en kısa yol).
- Her kenarın aynı ağırlığa (1) sahip olduğu varsayılır: yol uzunluğu = kenar sayısı.
- Çözüm: **BFS**. Seviye k'daki düğümün uzaklığı tam olarak k'dır.



Kaynak 1; düğümlerin yanındaki sayı 1'e uzaklık

7'ye iki en kısa yol var: $1 \rightarrow 4 \rightarrow 6 \rightarrow 7$ ve $1 \rightarrow 3 \rightarrow 8 \rightarrow 7$ (ikisi de 3 kenar). BFS bunlardan birini bulur; uzunluk her durumda 3'tür.

Ağırlıksız En Kısa Yol Algoritması

Sözde kod

```

mesafe[tüm v] = ∞
mesafe[s] = 0
kuyruk.ekle(s)
while kuyruk boş değil:
    w = kuyruk.çıkar()
    for w'nin her komşusu u:
        if mesafe[u] == ∞:
            mesafe[u] = mesafe[w] + 1
            kuyruk.ekle(u)
return mesafe

```

"mesafe = ∞" aynı zamanda "ziyaret edilmedi" demektir; ayrı bir dizi gerekmez. Java'da `cizge` : düğüm → komşu listesi (`Map<Integer, List<Integer>>`), çünkü düğümler 1'den başlayarak numaralanmış.

Java · O(n + m)

```

Map<Integer, Integer> enKisaYol(int s) {
    final int INF = Integer.MAX_VALUE; // ∞
    Map<Integer, Integer> mesafe =
        new HashMap<>();
    for (int v : çizge.keySet())
        mesafe.put(v, INF);
    mesafe.put(s, 0);
    Queue<Integer> q = new ArrayDeque<>();
    q.offer(s);
    while (!q.isEmpty()) {
        int w = q.poll();
        for (int u : çizge.get(w))
            if (mesafe.get(u) == INF) {
                mesafe.put(u, mesafe.get(w) + 1);
                q.offer(u);
            }
    }
    return mesafe;
}

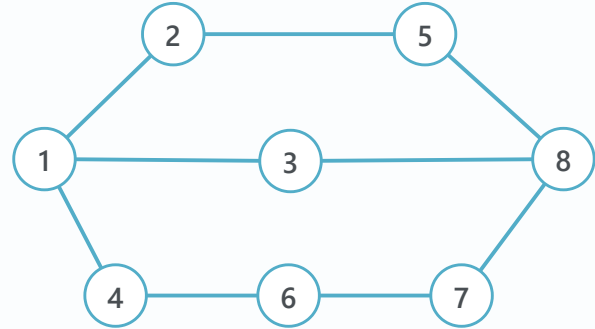
```

Kod İzleme: enKisaYol(1)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2     final int INF = Integer.MAX_VALUE; // ∞
3     Map<Integer, Integer> mesafe =
4         new HashMap<>();
5     for (int v : cizge.keySet())
6         mesafe.put(v, INF);
7     mesafe.put(s, 0);
8     Queue<Integer> q = new ArrayDeque<>();
9     q.offer(s);
10    while (!q.isEmpty()) {
11        int w = q.poll();
12        for (int u : cizge.get(w))
13            if (mesafe.get(u) == INF) {
14                mesafe.put(u, mesafe.get(w) + 1);
15                q.offer(u);
16            }
17    }
18    return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	∞	∞	∞	∞	∞	∞	∞	∞
q	(boş)							

Kaynak $s = 1$ (giriş slaytındaki çizge). Tüm mesafeler **INF** (∞): "henüz ulaşılmadı" = "ziyaret edilmedi".

Adım 1 / 13

DEĞİŞKENLER

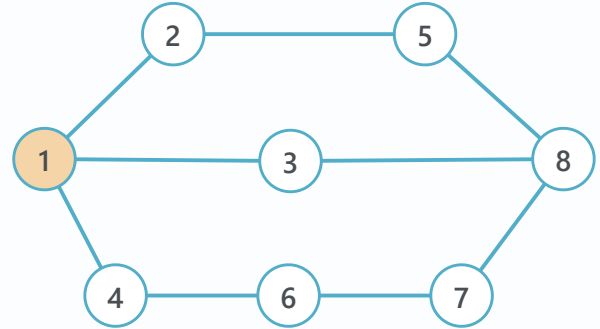
$s = 1$

Kod İzleme: enKisaYol(1)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2     final int INF = Integer.MAX_VALUE; // ∞
3     Map<Integer, Integer> mesafe =
4         new HashMap<>();
5     for (int v : cizge.keySet())
6         mesafe.put(v, INF);
7     ▶ mesafe.put(s, 0);
8     ▶ Queue<Integer> q = new ArrayDeque<>();
9     ▶ q.offer(s);
10    while (!q.isEmpty()) {
11        int w = q.poll();
12        for (int u : cizge.get(w))
13            if (mesafe.get(u) == INF) {
14                mesafe.put(u, mesafe.get(w) + 1);
15                q.offer(u);
16            }
17    }
18    return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	∞	∞	∞	∞	∞	∞	∞
q	1							

mesafe[1] = 0 ve 1 kuyruğa girer.

Adım 2 / 13

DEĞİŞKENLER

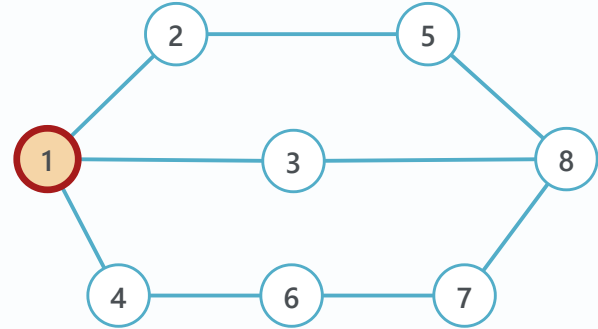
s = 1

Kod İzleme: enKisaYol(1)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2     final int INF = Integer.MAX_VALUE; // ∞
3     Map<Integer, Integer> mesafe =
4         new HashMap<>();
5     for (int v : cizge.keySet())
6         mesafe.put(v, INF);
7     mesafe.put(s, 0);
8     Queue<Integer> q = new ArrayDeque<>();
9     q.offer(s);
10    while (!q.isEmpty()) {
11        ▶ int w = q.poll();
12        for (int u : cizge.get(w))
13            if (mesafe.get(u) == INF) {
14                mesafe.put(u, mesafe.get(w) + 1);
15                q.offer(u);
16            }
17    }
18    return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	∞	∞	∞	∞	∞	∞	∞
q	(boş)							

1 çıkarılır; mesafe[1] = 0. Komşuları 2, 3, 4 incelenir.

Adım 3 / 13

DEĞİŞKENLER

s = 1

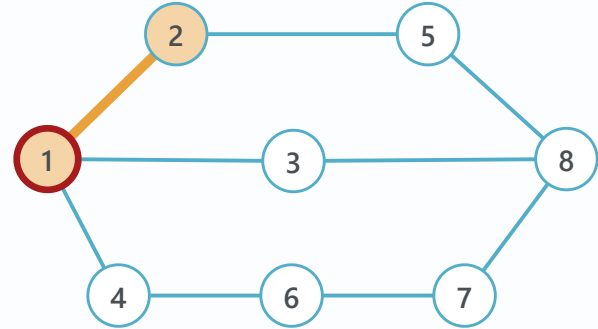
w = 1

Kod İzleme: enKisaYol(1)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2     final int INF = Integer.MAX_VALUE; // ∞
3     Map<Integer, Integer> mesafe =
4         new HashMap<>();
5     for (int v : cizge.keySet())
6         mesafe.put(v, INF);
7     mesafe.put(s, 0);
8     Queue<Integer> q = new ArrayDeque<>();
9     q.offer(s);
10    while (!q.isEmpty()) {
11        int w = q.poll();
12        for (int u : cizge.get(w))
13            ▶ if (mesafe.get(u) == INF) {
14                ▶     mesafe.put(u, mesafe.get(w) + 1);
15                ▶     q.offer(u);
16            }
17    }
18    return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	∞	∞	∞	∞	∞	∞
q		2						

mesafe[2] = INF → **mesafe[2] = 0 + 1 = 1**, 2
kuyruğa eklenir.

Adım 4 / 13

DEĞİŞKENLER

s = 1

w = 1

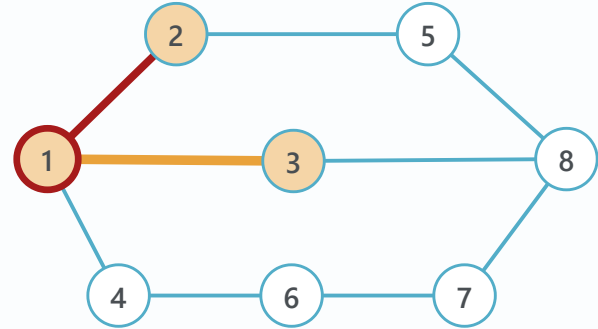
u = 2

Kod İzleme: enKisaYol(1)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2   final int INF = Integer.MAX_VALUE; // ∞
3   Map<Integer, Integer> mesafe =
4     new HashMap<>();
5   for (int v : cizge.keySet())
6     mesafe.put(v, INF);
7   mesafe.put(s, 0);
8   Queue<Integer> q = new ArrayDeque<>();
9   q.offer(s);
10  while (!q.isEmpty()) {
11    int w = q.poll();
12    for (int u : cizge.get(w))
13      ▶ if (mesafe.get(u) == INF) {
14        ▶   mesafe.put(u, mesafe.get(w) + 1);
15        ▶   q.offer(u);
16      }
17  }
18  return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	∞	∞	∞	∞	∞
q		2	3					

mesafe[3] = INF → **mesafe[3] = 0 + 1 = 1**, 3
kuyruğa eklenir.

Adım 5 / 13

DEĞİŞKENLER

s = 1

w = 1

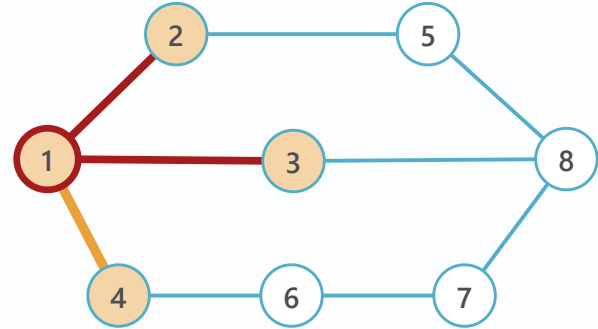
u = 3

Kod İzleme: enKisaYol(1)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2     final int INF = Integer.MAX_VALUE; // ∞
3     Map<Integer, Integer> mesafe =
4         new HashMap<>();
5     for (int v : cizge.keySet())
6         mesafe.put(v, INF);
7     mesafe.put(s, 0);
8     Queue<Integer> q = new ArrayDeque<>();
9     q.offer(s);
10    while (!q.isEmpty()) {
11        int w = q.poll();
12        for (int u : cizge.get(w))
13            ▶ if (mesafe.get(u) == INF) {
14                ▶ mesafe.put(u, mesafe.get(w) + 1);
15                ▶ q.offer(u);
16            }
17    }
18    return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	∞	∞	∞	∞
q		2	3	4				

mesafe[4] = INF → **mesafe[4] = 0 + 1 = 1**, 4 kuyruğa eklenir.

Adım 6 / 13

DEĞİŞKENLER

s = 1

w = 1

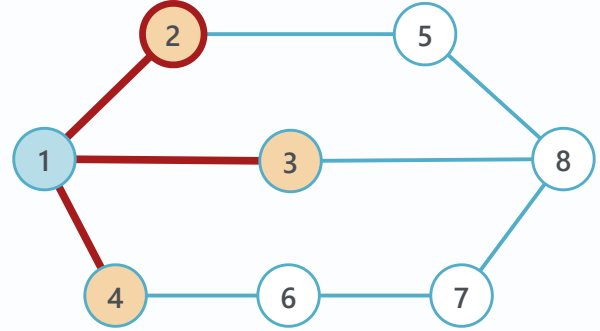
u = 4

Kod İzleme: enKisaYol(1)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2     final int INF = Integer.MAX_VALUE; // ∞
3     Map<Integer, Integer> mesafe =
4         new HashMap<>();
5     for (int v : cizge.keySet())
6         mesafe.put(v, INF);
7     mesafe.put(s, 0);
8     Queue<Integer> q = new ArrayDeque<>();
9     q.offer(s);
10    while (!q.isEmpty()) {
11        ▶ int w = q.poll();
12        for (int u : cizge.get(w))
13            if (mesafe.get(u) == INF) {
14                mesafe.put(u, mesafe.get(w) + 1);
15                q.offer(u);
16            }
17    }
18    return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	∞	∞	∞	∞
q		3	4					

2 çıkarılır; mesafe[2] = 1. Komşuları 1, 5 incelenir.

Adım 7 / 13

DEĞİŞKENLER

s = 1

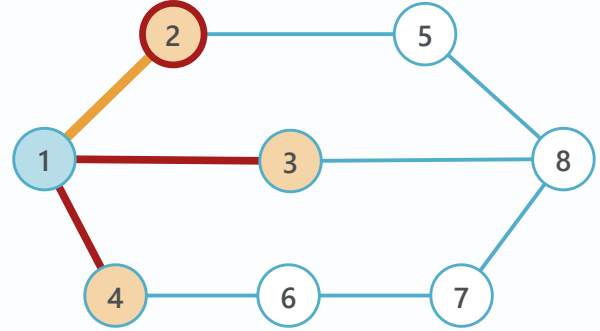
w = 2

Kod İzleme: enKisaYol(1)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2     final int INF = Integer.MAX_VALUE; // ∞
3     Map<Integer, Integer> mesafe =
4         new HashMap<>();
5     for (int v : cizge.keySet())
6         mesafe.put(v, INF);
7     mesafe.put(s, 0);
8     Queue<Integer> q = new ArrayDeque<>();
9     q.offer(s);
10    while (!q.isEmpty()) {
11        int w = q.poll();
12        for (int u : cizge.get(w))
13            ► if (mesafe.get(u) == INF) {
14                mesafe.put(u, mesafe.get(w) + 1);
15                q.offer(u);
16            }
17    }
18    return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	∞	∞	∞	∞
q		3	4					

1: mesafesi zaten belli (0) → değişmez. BFS'de ilk bulunan mesafe en kısıdır.

Adım 8 / 13

DEĞİŞKENLER

s = 1

w = 2

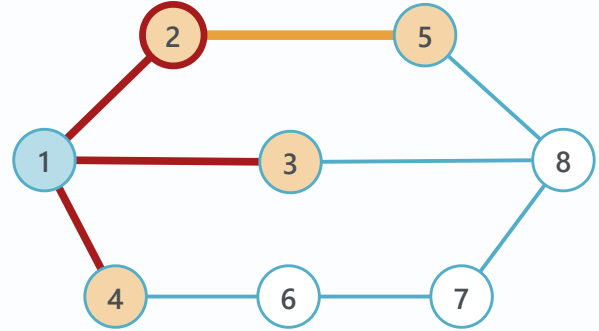
u = 1

Kod İzleme: enKisaYol(1)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2   final int INF = Integer.MAX_VALUE; // ∞
3   Map<Integer, Integer> mesafe =
4     new HashMap<>();
5   for (int v : cizge.keySet())
6     mesafe.put(v, INF);
7   mesafe.put(s, 0);
8   Queue<Integer> q = new ArrayDeque<>();
9   q.offer(s);
10  while (!q.isEmpty()) {
11    int w = q.poll();
12    for (int u : cizge.get(w))
13      ▶ if (mesafe.get(u) == INF) {
14        ▶   mesafe.put(u, mesafe.get(w) + 1);
15        ▶   q.offer(u);
16      }
17  }
18  return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	2	∞	∞	∞
q		3	4	5				

mesafe[5] = INF → **mesafe[5] = 1 + 1 = 2**, 5
kuyruğa eklenir.

Adım 9 / 13

DEĞİŞKENLER

s = 1

w = 2

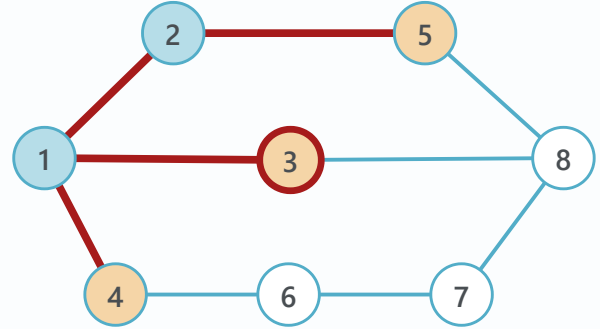
u = 5

Kod İzleme: enKisaYol(1)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2     final int INF = Integer.MAX_VALUE; // ∞
3     Map<Integer, Integer> mesafe =
4         new HashMap<>();
5     for (int v : cizge.keySet())
6         mesafe.put(v, INF);
7     mesafe.put(s, 0);
8     Queue<Integer> q = new ArrayDeque<>();
9     q.offer(s);
10    while (!q.isEmpty()) {
11        ▶ int w = q.poll();
12        for (int u : cizge.get(w))
13            if (mesafe.get(u) == INF) {
14                mesafe.put(u, mesafe.get(w) + 1);
15                q.offer(u);
16            }
17    }
18    return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	2	∞	∞	∞
q		4	5					

3 çıkarılır; mesafe[3] = 1. Komşuları 1, 8 incelenir.

Adım 10 / 13

DEĞİŞKENLER

s = 1

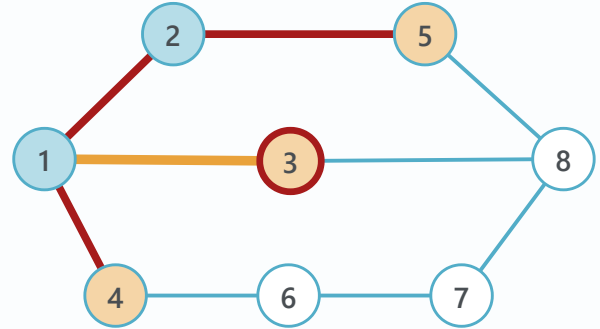
w = 3

Kod İzleme: enKisaYol(1)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2   final int INF = Integer.MAX_VALUE; // ∞
3   Map<Integer, Integer> mesafe =
4     new HashMap<>();
5   for (int v : cizge.keySet())
6     mesafe.put(v, INF);
7   mesafe.put(s, 0);
8   Queue<Integer> q = new ArrayDeque<>();
9   q.offer(s);
10  while (!q.isEmpty()) {
11    int w = q.poll();
12    for (int u : cizge.get(w))
13      ► if (mesafe.get(u) == INF) {
14        mesafe.put(u, mesafe.get(w) + 1);
15        q.offer(u);
16      }
17  }
18  return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	2	∞	∞	∞
q	4	5						

1: mesafesi zaten belli (0) → değişmez. BFS'de ilk bulunan mesafe en kısıdır.

Adım 11 / 13

DEĞİŞKENLER

s = 1

w = 3

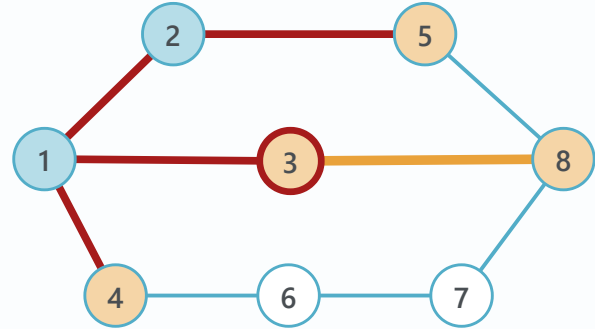
u = 1

Kod İzleme: enKisaYol(1)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2     final int INF = Integer.MAX_VALUE; // ∞
3     Map<Integer, Integer> mesafe =
4         new HashMap<>();
5     for (int v : cizge.keySet())
6         mesafe.put(v, INF);
7     mesafe.put(s, 0);
8     Queue<Integer> q = new ArrayDeque<>();
9     q.offer(s);
10    while (!q.isEmpty()) {
11        int w = q.poll();
12        for (int u : cizge.get(w))
13            ▶ if (mesafe.get(u) == INF) {
14                ▶     mesafe.put(u, mesafe.get(w) + 1);
15                ▶     q.offer(u);
16            }
17    }
18    return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	2	∞	∞	2
q		4	5	8				

mesafe[8] = INF → **mesafe[8] = 1 + 1 = 2**, 8
kuyruğa eklenir.

Adım 12 / 13

DEĞİŞKENLER

s = 1

w = 3

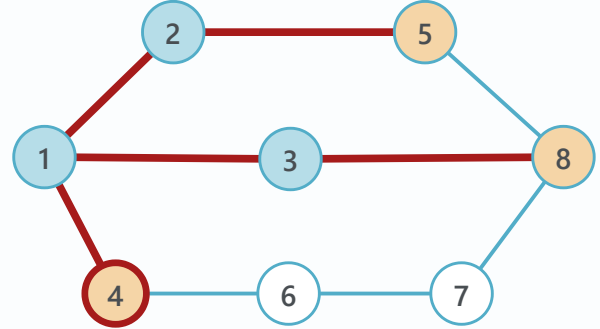
u = 8

Kod İzleme: enKisaYol(1)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2   final int INF = Integer.MAX_VALUE; // ∞
3   Map<Integer, Integer> mesafe =
4     new HashMap<>();
5   for (int v : cizge.keySet())
6     mesafe.put(v, INF);
7   mesafe.put(s, 0);
8   Queue<Integer> q = new ArrayDeque<>();
9   q.offer(s);
10  while (!q.isEmpty()) {
11    ▶ int w = q.poll();
12    for (int u : cizge.get(w))
13      if (mesafe.get(u) == INF) {
14        mesafe.put(u, mesafe.get(w) + 1);
15        q.offer(u);
16      }
17  }
18  return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	2	∞	∞	2
q		5	8					

4 çıkarılır; mesafe[4] = 1. Komşuları 1, 6 incelenir.

Adım 13 / 13

DEĞİŞKENLER

s = 1

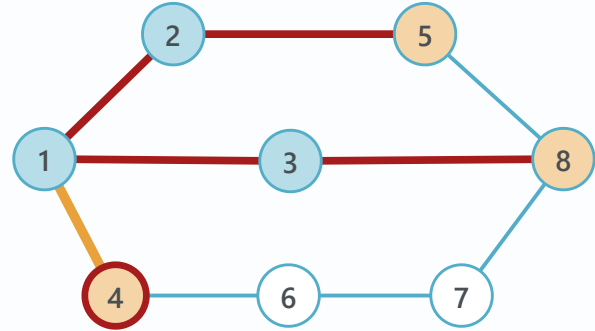
w = 4

Kod İzleme: enKisaYol(1) (devam)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2     final int INF = Integer.MAX_VALUE; // ∞
3     Map<Integer, Integer> mesafe =
4         new HashMap<>();
5     for (int v : cizge.keySet())
6         mesafe.put(v, INF);
7     mesafe.put(s, 0);
8     Queue<Integer> q = new ArrayDeque<>();
9     q.offer(s);
10    while (!q.isEmpty()) {
11        int w = q.poll();
12        for (int u : cizge.get(w))
13            ▶ if (mesafe.get(u) == INF) {
14                mesafe.put(u, mesafe.get(w) + 1);
15                q.offer(u);
16            }
17    }
18    return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	2	∞	∞	2
q		5	8					

1: mesafesi zaten belli (0) → değişmez. BFS'de ilk bulunan mesafe en kısıdır.

Adım 1 / 12

DEĞİŞKENLER

s = 1

w = 4

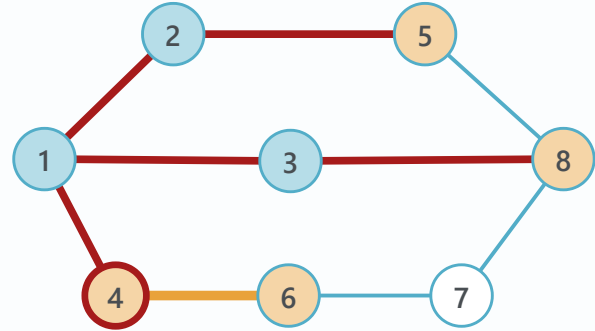
u = 1

Kod İzleme: enKisaYol(1) (devam)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2     final int INF = Integer.MAX_VALUE; // ∞
3     Map<Integer, Integer> mesafe =
4         new HashMap<>();
5     for (int v : cizge.keySet())
6         mesafe.put(v, INF);
7     mesafe.put(s, 0);
8     Queue<Integer> q = new ArrayDeque<>();
9     q.offer(s);
10    while (!q.isEmpty()) {
11        int w = q.poll();
12        for (int u : cizge.get(w))
13            ▶ if (mesafe.get(u) == INF) {
14                ▶ mesafe.put(u, mesafe.get(w) + 1);
15                ▶ q.offer(u);
16            }
17    }
18    return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	2	2	∞	2
q		5	8	6				

mesafe[6] = INF → **mesafe[6] = 1 + 1 = 2**, 6
kuyruğa eklenir.

Adım 2 / 12

DEĞİŞKENLER

s = 1

w = 4

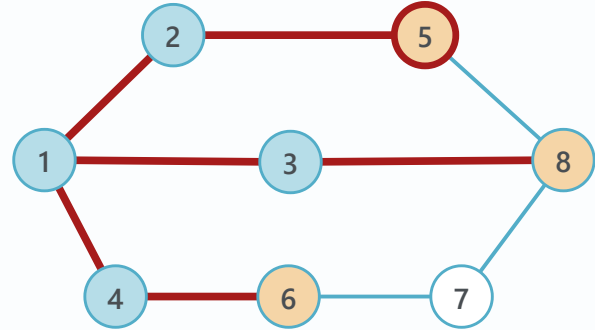
u = 6

Kod İzleme: enKisaYol(1) (devam)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2     final int INF = Integer.MAX_VALUE; // ∞
3     Map<Integer, Integer> mesafe =
4         new HashMap<>();
5     for (int v : cizge.keySet())
6         mesafe.put(v, INF);
7     mesafe.put(s, 0);
8     Queue<Integer> q = new ArrayDeque<>();
9     q.offer(s);
10    while (!q.isEmpty()) {
11        ▶ int w = q.poll();
12        for (int u : cizge.get(w))
13            if (mesafe.get(u) == INF) {
14                mesafe.put(u, mesafe.get(w) + 1);
15                q.offer(u);
16            }
17    }
18    return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	2	2	∞	2
q	8	6						

5 çıkarılır; mesafe[5] = 2. Komşuları 2, 8 incelenir.

Adım 3 / 12

DEĞİŞKENLER

s = 1

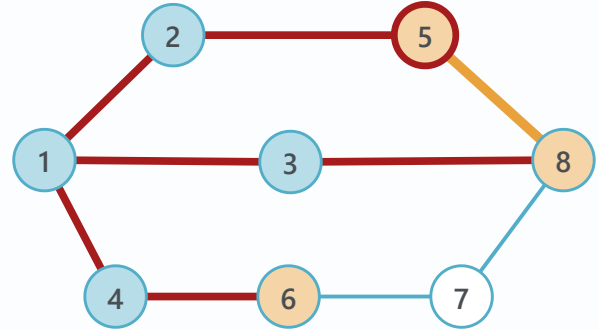
w = 5

Kod İzleme: enKisaYol(1) (devam)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2   final int INF = Integer.MAX_VALUE; // ∞
3   Map<Integer, Integer> mesafe =
4     new HashMap<>();
5   for (int v : cizge.keySet())
6     mesafe.put(v, INF);
7   mesafe.put(s, 0);
8   Queue<Integer> q = new ArrayDeque<>();
9   q.offer(s);
10  while (!q.isEmpty()) {
11    int w = q.poll();
12    for (int u : cizge.get(w))
13      ► if (mesafe.get(u) == INF) {
14        mesafe.put(u, mesafe.get(w) + 1);
15        q.offer(u);
16      }
17  }
18  return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	2	2	∞	2
q	8	6						

2, 8: mesafesi zaten belli (1, 2) → değişmez. BFS'de ilk bulunan mesafe en kısadır.

Adım 4 / 12

DEĞİŞKENLER

s = 1

w = 5

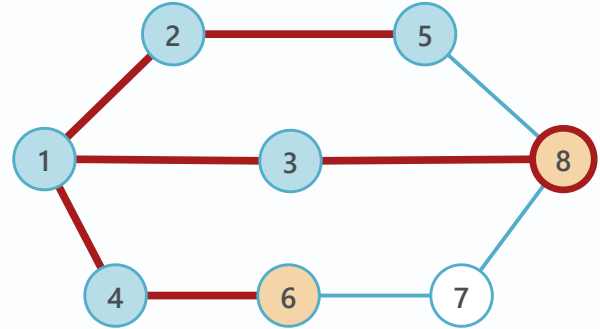
u = 8

Kod İzleme: enKisaYol(1) (devam)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2     final int INF = Integer.MAX_VALUE; // ∞
3     Map<Integer, Integer> mesafe =
4         new HashMap<>();
5     for (int v : cizge.keySet())
6         mesafe.put(v, INF);
7     mesafe.put(s, 0);
8     Queue<Integer> q = new ArrayDeque<>();
9     q.offer(s);
10    while (!q.isEmpty()) {
11        ▶ int w = q.poll();
12        for (int u : cizge.get(w))
13            if (mesafe.get(u) == INF) {
14                mesafe.put(u, mesafe.get(w) + 1);
15                q.offer(u);
16            }
17    }
18    return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	2	2	∞	2
q		6						

8 çıkarılır; mesafe[8] = 2. Komşuları 3, 5, 7 incelenir.

Adım 5 / 12

DEĞİŞKENLER

s = 1

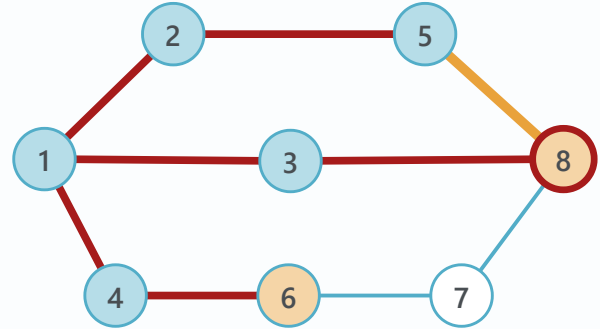
w = 8

Kod İzleme: enKisaYol(1) (devam)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2     final int INF = Integer.MAX_VALUE; // ∞
3     Map<Integer, Integer> mesafe =
4         new HashMap<>();
5     for (int v : cizge.keySet())
6         mesafe.put(v, INF);
7     mesafe.put(s, 0);
8     Queue<Integer> q = new ArrayDeque<>();
9     q.offer(s);
10    while (!q.isEmpty()) {
11        int w = q.poll();
12        for (int u : cizge.get(w))
13            ► if (mesafe.get(u) == INF) {
14                mesafe.put(u, mesafe.get(w) + 1);
15                q.offer(u);
16            }
17    }
18    return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	2	2	∞	2
q	6							

3, 5: mesafesi zaten belli (1, 2) → değişmez. BFS'de ilk bulunan mesafe en kısıdır.

Adım 6 / 12

DEĞİŞKENLER

s = 1

w = 8

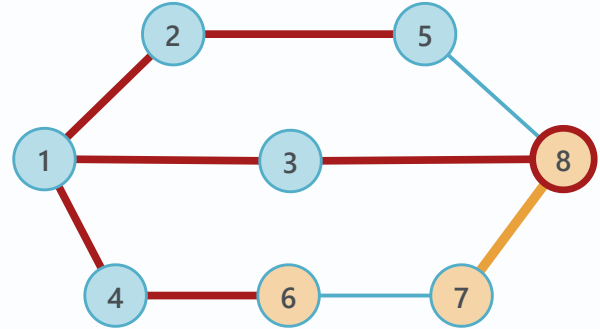
u = 5

Kod İzleme: enKisaYol(1) (devam)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2   final int INF = Integer.MAX_VALUE; // ∞
3   Map<Integer, Integer> mesafe =
4     new HashMap<>();
5   for (int v : cizge.keySet())
6     mesafe.put(v, INF);
7   mesafe.put(s, 0);
8   Queue<Integer> q = new ArrayDeque<>();
9   q.offer(s);
10  while (!q.isEmpty()) {
11    int w = q.poll();
12    for (int u : cizge.get(w))
13      ▶ if (mesafe.get(u) == INF) {
14        ▶   mesafe.put(u, mesafe.get(w) + 1);
15        ▶   q.offer(u);
16      }
17  }
18  return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	2	2	3	2
q		6	7					

mesafe[7] = INF → **mesafe[7] = 2 + 1 = 3**, 7
kuyruğa eklenir.

Adım 7 / 12

DEĞİŞKENLER

s = 1

w = 8

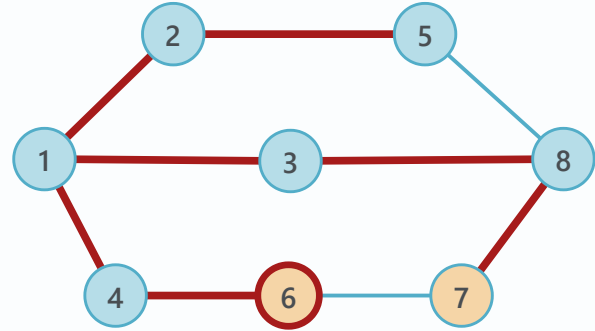
u = 7

Kod İzleme: enKisaYol(1) (devam)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2     final int INF = Integer.MAX_VALUE; // ∞
3     Map<Integer, Integer> mesafe =
4         new HashMap<>();
5     for (int v : cizge.keySet())
6         mesafe.put(v, INF);
7     mesafe.put(s, 0);
8     Queue<Integer> q = new ArrayDeque<>();
9     q.offer(s);
10    while (!q.isEmpty()) {
11        ▶ int w = q.poll();
12        for (int u : cizge.get(w))
13            if (mesafe.get(u) == INF) {
14                mesafe.put(u, mesafe.get(w) + 1);
15                q.offer(u);
16            }
17    }
18    return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	2	2	3	2
q	7							

6 çıkarılır; mesafe[6] = 2. Komşuları 4, 7 incelenir.

Adım 8 / 12

DEĞİŞKENLER

s = 1

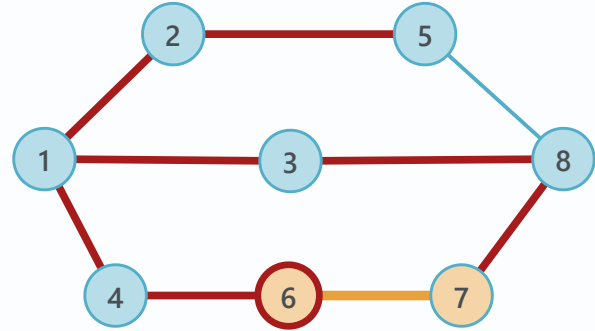
w = 6

Kod İzleme: enKisaYol(1) (devam)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2   final int INF = Integer.MAX_VALUE; // ∞
3   Map<Integer, Integer> mesafe =
4     new HashMap<>();
5   for (int v : cizge.keySet())
6     mesafe.put(v, INF);
7   mesafe.put(s, 0);
8   Queue<Integer> q = new ArrayDeque<>();
9   q.offer(s);
10  while (!q.isEmpty()) {
11    int w = q.poll();
12    for (int u : cizge.get(w))
13      ► if (mesafe.get(u) == INF) {
14        mesafe.put(u, mesafe.get(w) + 1);
15        q.offer(u);
16      }
17  }
18  return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	2	2	3	2
q	7							

4, 7: mesafesi zaten belli (1, 3) → değişmez. BFS'de ilk bulunan mesafe en kısadır.

Adım 9 / 12

DEĞİŞKENLER

s = 1

w = 6

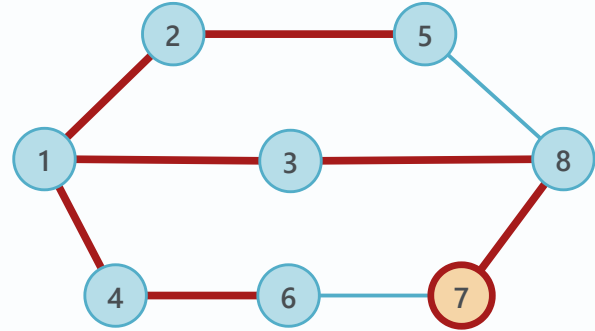
u = 7

Kod İzleme: enKisaYol(1) (devam)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2     final int INF = Integer.MAX_VALUE; // ∞
3     Map<Integer, Integer> mesafe =
4         new HashMap<>();
5     for (int v : cizge.keySet())
6         mesafe.put(v, INF);
7     mesafe.put(s, 0);
8     Queue<Integer> q = new ArrayDeque<>();
9     q.offer(s);
10    while (!q.isEmpty()) {
11        ▶ int w = q.poll();
12        for (int u : cizge.get(w))
13            if (mesafe.get(u) == INF) {
14                mesafe.put(u, mesafe.get(w) + 1);
15                q.offer(u);
16            }
17    }
18    return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	2	2	3	2

q	(boş)						
---	-------	--	--	--	--	--	--

7 çıkarılır; mesafe[7] = 3. Komşuları 6, 8 incelenir.

Adım 10 / 12

DEĞİŞKENLER

s = 1

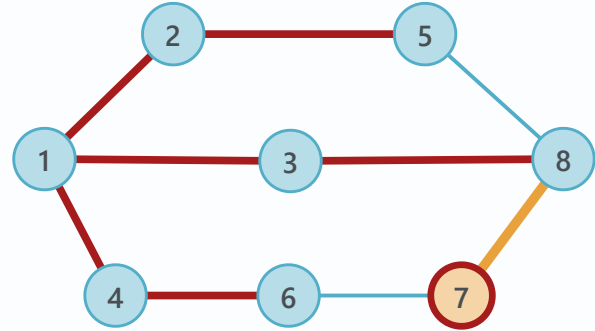
w = 7

Kod İzleme: enKisaYol(1) (devam)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2     final int INF = Integer.MAX_VALUE; // ∞
3     Map<Integer, Integer> mesafe =
4         new HashMap<>();
5     for (int v : cizge.keySet())
6         mesafe.put(v, INF);
7     mesafe.put(s, 0);
8     Queue<Integer> q = new ArrayDeque<>();
9     q.offer(s);
10    while (!q.isEmpty()) {
11        int w = q.poll();
12        for (int u : cizge.get(w))
13            ► if (mesafe.get(u) == INF) {
14                mesafe.put(u, mesafe.get(w) + 1);
15                q.offer(u);
16            }
17    }
18    return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	2	2	3	2
q	(boş)							

6, 8: mesafesi zaten belli (2, 2) → değişmez. BFS'de ilk bulunan mesafe en kısadır.

Adım 11 / 12

DEĞİŞKENLER

s = 1

w = 7

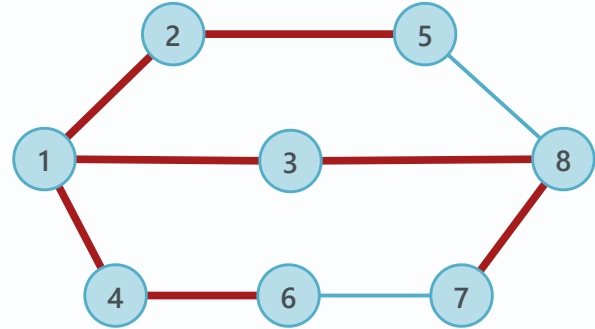
u = 8

Kod İzleme: enKisaYol(1) (devam)

```

1 Map<Integer, Integer> enKisaYol(int s) {
2     final int INF = Integer.MAX_VALUE; // ∞
3     Map<Integer, Integer> mesafe =
4         new HashMap<>();
5     for (int v : cizge.keySet())
6         mesafe.put(v, INF);
7     mesafe.put(s, 0);
8     Queue<Integer> q = new ArrayDeque<>();
9     q.offer(s);
10    while (!q.isEmpty()) {
11        int w = q.poll();
12        for (int u : cizge.get(w))
13            if (mesafe.get(u) == INF) {
14                mesafe.put(u, mesafe.get(w) + 1);
15                q.offer(u);
16            }
17    }
18    return mesafe;
19 }

```



	1	2	3	4	5	6	7	8
mesafe	0	1	1	1	2	2	3	2
q	(boş)							

Kuyruk boş → **return mesafe**. 7'ye uzaklık 3 (1 → 3 → 8 → 7 bulundu; 1 → 4 → 6 → 7 de 3 kenar). $O(n + m)$.

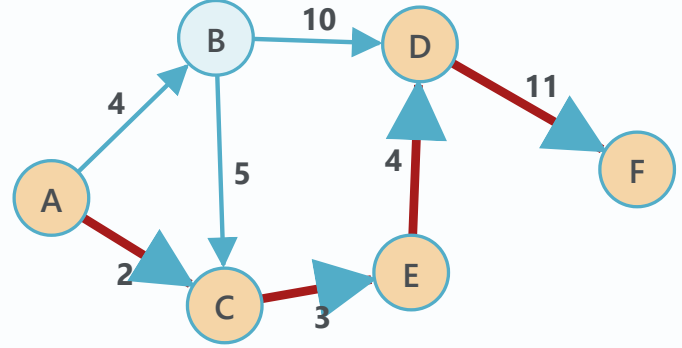
Adım 12 / 12

DEĞİŞKENLER

s = 1

Ağırlıklı En Kısa Yol Problemi

- Kaynak düğümden diğer tüm düğümlere olan en kısa yolları bulur.
- Her kenarın farklı bir ağırlığı olabilir: uzaklık, süre, maliyet...
- Yol uzunluğu = kenar ağırlıklarının toplamı.



A'dan F'ye en kısa yol: $A \rightarrow C \rightarrow E \rightarrow D \rightarrow F = 2 + 3 + 4 + 11 = 20$

BFS burada yetmez

En az kenarlı yol $A \rightarrow B \rightarrow D \rightarrow F$ (3 kenar) ama uzunluğu $4 + 10 + 11 =$
25

Çözüm: Dijkstra algoritması

Edsger W. Dijkstra, 1956. Ağırlıklar **negatif olmamalıdır**

Dijkstra Algoritması: Fikir ve Sözde Kod

- Kesinleşmemiş düğümlerden mesafesi en küçük olanı seç.
- Komşuların mesafesini **gevşetme** ile güncelle.
- Seçim için öncelikli kuyruk, yolu kurmak için `önceki[]` kullan.

Dijkstra(G, s):

for her düğüm v:

 mesafe[v] = ∞

 önceki[v] = tanımsız

mesafe[s] = 0

Q = tüm düğümler // öncelikli kuyruk

while Q boş değil:

 u = Q.çıkar() // en küçük mesafe[u]

for u'nun Q'daki her komşusu v:

 alt = mesafe[u] + ağırlık(u, v)

if alt < mesafe[v]:

 mesafe[v] = alt

 önceki[v] = u

 // Q'da v'nin önceliğini azalt

Dijkstra Algoritması: Java

```

long[] dijkstra(List<List<int[]>> adj, int s, int[] onceki) { // ağırlıklar ≥ 0
    long[] d = new long[adj.size()];
    Arrays.fill(d, Long.MAX_VALUE); d[s] = 0; Arrays.fill(onceki, -1);
    PriorityQueue<long[]> pq = // {düğüm, mesafe}
        new PriorityQueue<>((a, b) -> Long.compare(a[1], b[1]));
    pq.add(new long[]{s, 0});
    while (!pq.isEmpty()) {
        long[] e = pq.poll(); int u = (int) e[0];
        if (e[1] > d[u]) continue; // bayat kayıt: atla
        for (int[] k : adj.get(u)) {
            int v = k[0]; long alt = d[u] + k[1];
            if (alt < d[v]) { d[v] = alt; onceki[v] = u; pq.add(new long[]{v, alt}); }
        }
    }
    return d;
}

```

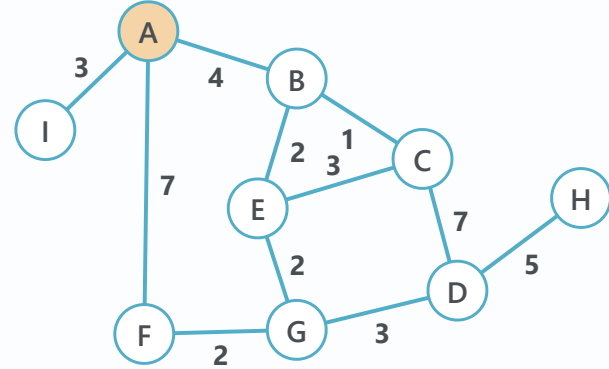
Öncelik = mesafe (düğüm numarası değil). Java'nın `PriorityQueue` 'sunda "önceliği azalt" yok: yeni kayıt eklenir, eskisi çıkınca *bayat* olarak atlanır. `onceki[v] = u` en kısa yol ağacını tutar; yol hedeften geriye izlenir.

Kod İzleme: dijkstra(adj, A)

```

▶ // d[] = ∞, d[s] = 0, onceki[] = -1
▶ pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4   long[] e = pq.poll();
5   int u = (int) e[0];
6   if (e[1] > d[u]) continue; // bayat
7   for (int[] k : adj.get(u)) {
8     int v = k[0];
9     long alt = d[u] + k[1];
10    if (alt < d[v]) {
11      d[v] = alt;
12      onceki[v] = u;
13      pq.add(new long[] {v, alt});
14    }
15  }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	∞	∞	∞	∞	∞	∞	∞	∞
onceki	-	-	-	-	-	-	-	-	-

pq (mesafeye göre sıralı gösterim; soluk = bayat)

A,0

Kaynak A: $d[A] = 0$, diğerleri ∞ . Kuyruğa {A, 0} eklenir.
Öncelik = **mesafe**.

DEĞİŞKENLER

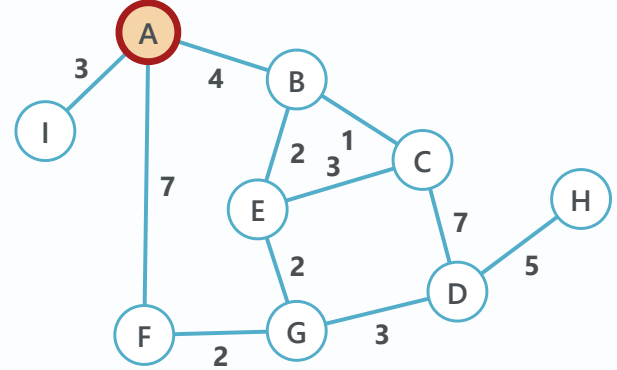
s = A

Kod İzleme: dijkstra(adj, A)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     ▶ long[] e = pq.poll();
5     ▶ int u = (int) e[0];
6     ▶ if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         int v = k[0];
9         long alt = d[u] + k[1];
10        if (alt < d[v]) {
11            d[v] = alt;
12            onceki[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	∞	∞	∞	∞	∞	∞	∞	∞
onceki	-	-	-	-	-	-	-	-	-

pq (mesafeye göre sıralı gösterim; soluk = bayat)
(boş)

En küçük kayıt {A, 0} çıkar; 0 = d[A] (güncel) → **A kesinleşir**. Komşuları B, F, I gevşetilir.

DEĞİŞKENLER

e = {A, 0}

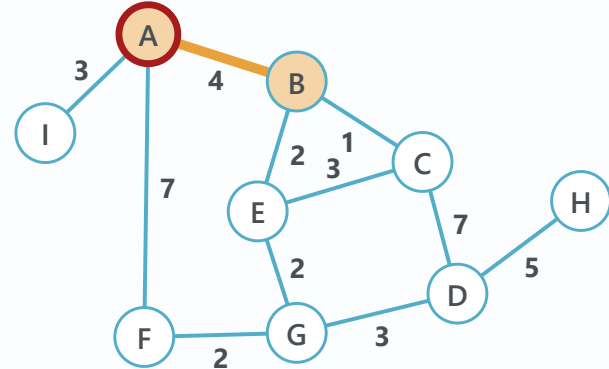
u = A

Kod İzleme: dijkstra(adj, A)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     long[] e = pq.poll();
5     int u = (int) e[0];
6     if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         ▶ int v = k[0];
9         ▶ long alt = d[u] + k[1];
10        ▶ if (alt < d[v]) {
11            ▶ d[v] = alt;
12            ▶ onceki[v] = u;
13            ▶ pq.add(new long[] {v, alt});
14        }
15    }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	4	∞	∞	∞	∞	∞	∞	∞
onceki	-	A	-	-	-	-	-	-	-

pq (mesafeye göre sıralı gösterim; soluk = bayat)

B,4

$v = B: alt = 0 + 4 = 4 < \infty \rightarrow d[B] = 4, onceki[B] = A, \{B, 4\}$ kuyruğa.

DEĞİŞKENLER

e = {A,0}

u = A

v = B

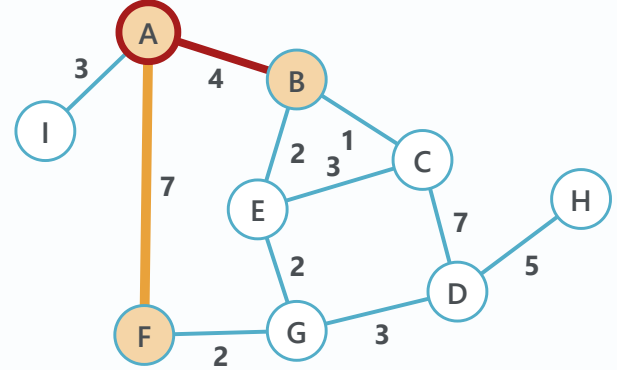
alt = 4

Kod İzleme: dijkstra(adj, A)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     long[] e = pq.poll();
5     int u = (int) e[0];
6     if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         ▶ int v = k[0];
9         ▶ long alt = d[u] + k[1];
10        ▶ if (alt < d[v]) {
11            ▶ d[v] = alt;
12            ▶ onceki[v] = u;
13            ▶ pq.add(new long[] {v, alt});
14        }
15    }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	4	∞	∞	∞	7	∞	∞	∞
onceki	-	A	-	-	-	A	-	-	-

pq (mesafeye göre sıralı gösterim; soluk = bayat)

B,4

F,7

$v = F: \text{alt} = 0 + 7 = 7 < \infty \rightarrow d[F] = 7, \text{onceki}[F] = A, \{F, 7\}$ kuyruğa.

DEĞİŞKENLER

e = {A,0}

u = A

v = F

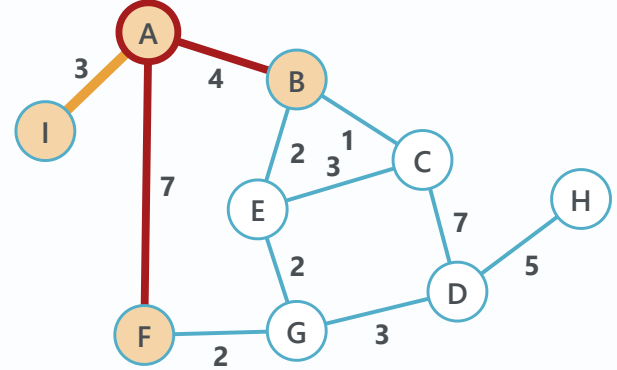
alt = 7

Kod İzleme: dijkstra(adj, A)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     long[] e = pq.poll();
5     int u = (int) e[0];
6     if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         ▶ int v = k[0];
9         ▶ long alt = d[u] + k[1];
10        ▶ if (alt < d[v]) {
11            ▶ d[v] = alt;
12            ▶ onceki[v] = u;
13            ▶ pq.add(new long[] {v, alt});
14        }
15    }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	4	∞	∞	∞	7	∞	∞	3
onceki	-	A	-	-	-	A	-	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

I,3 B,4 F,7

$v = I: \text{alt} = 0 + 3 = 3 < \infty \rightarrow d[I] = 3, \text{onceki}[I] = A, \{I, 3\}$ kuyruğa.

DEĞİŞKENLER

e = {A,0}

u = A

v = I

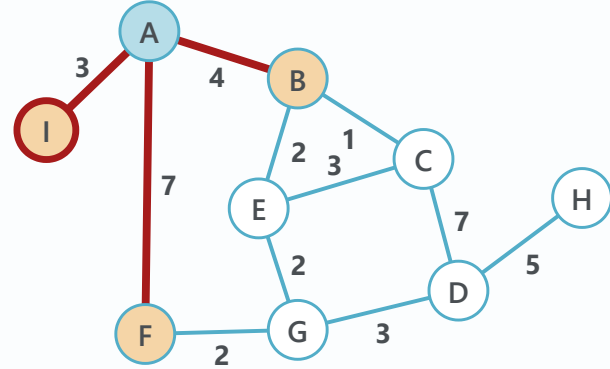
alt = 3

Kod İzleme: dijkstra(adj, A)

```

1 // d[] = ∞, d[s] = 0, oncesi[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     ▶ long[] e = pq.poll();
5     ▶ int u = (int) e[0];
6     ▶ if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         int v = k[0];
9         long alt = d[u] + k[1];
10        if (alt < d[v]) {
11            d[v] = alt;
12            oncesi[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	4	∞	∞	∞	7	∞	∞	3
oncesi	-	A	-	-	-	A	-	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

B,4

F,7

En küçük kayıt {I, 3} çıkar; 3 = d[I] (güncel) → **I kesinleşir**. Komşusu A gevşetilir.

DEĞİŞKENLER

e = {I, 3}

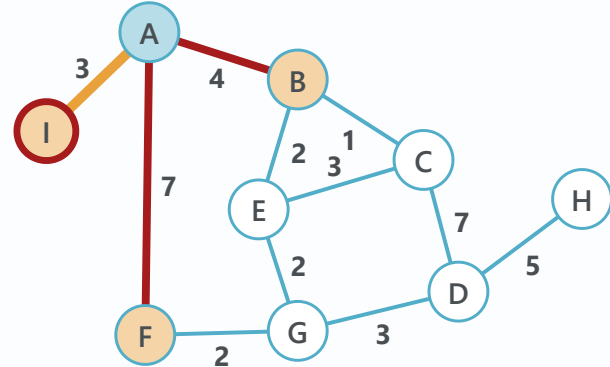
u = I

Kod İzleme: dijkstra(adj, A)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     long[] e = pq.poll();
5     int u = (int) e[0];
6     if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         ▶ int v = k[0];
9         ▶ long alt = d[u] + k[1];
10        ▶ if (alt < d[v]) {
11            d[v] = alt;
12            onceki[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	4	∞	∞	∞	7	∞	∞	3
onceki	-	A	-	-	-	A	-	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

B,4

F,7

$v = A: \text{alt} = 6 \geq d[A] = 0 \rightarrow$ güncelleme yok.

DEĞİŞKENLER

e = {I, 3}

u = I

v = A

alt = 6

Kod İzleme: dijkstra(adj, A)

```

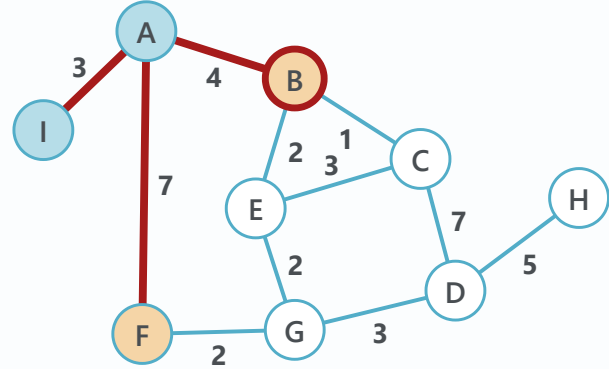
1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     ▶ long[] e = pq.poll();
5     ▶ int u = (int) e[0];
6     ▶ if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         int v = k[0];
9         long alt = d[u] + k[1];
10        if (alt < d[v]) {
11            d[v] = alt;
12            onceki[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```

DEĞİŞKENLER

e = {B, 4}

u = B



	A	B	C	D	E	F	G	H	I
d[]	0	4	∞	∞	∞	7	∞	∞	3
onceki	-	A	-	-	-	A	-	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

F, 7

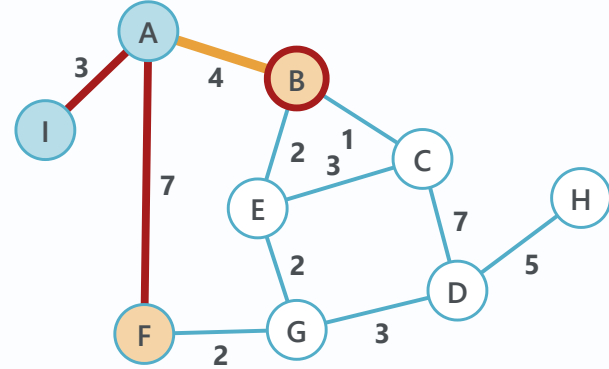
En küçük kayıt {B, 4} çıkar; 4 = d[B] (güncel) → **B kesinleşir**. Komşuları A, C, E gevşetilir.

Kod İzleme: dijkstra(adj, A)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     long[] e = pq.poll();
5     int u = (int) e[0];
6     if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         ▶ int v = k[0];
9         ▶ long alt = d[u] + k[1];
10        ▶ if (alt < d[v]) {
11            d[v] = alt;
12            onceki[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	4	∞	∞	∞	7	∞	∞	3
onceki	-	A	-	-	-	A	-	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

F,7

$v = A: alt = 8 \geq d[A] = 0 \rightarrow$ güncelleme yok.

DEĞİŞKENLER

e = {B,4}

u = B

v = A

alt = 8

Kod İzleme: dijkstra(adj, A)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     long[] e = pq.poll();
5     int u = (int) e[0];
6     if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         ▶ int v = k[0];
9         ▶ long alt = d[u] + k[1];
10        ▶ if (alt < d[v]) {
11            ▶ d[v] = alt;
12            ▶ onceki[v] = u;
13            ▶ pq.add(new long[] {v, alt});
14        }
15    }
16 }

```

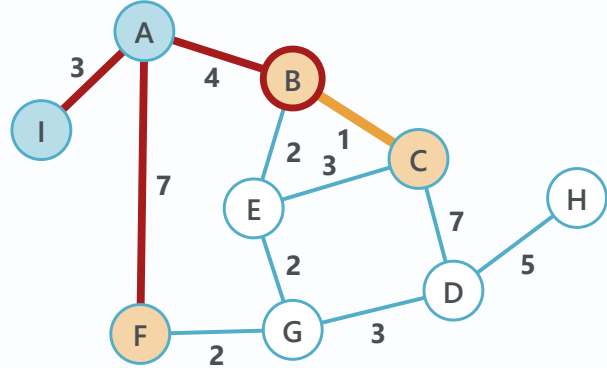
DEĞİŞKENLER

e = {B,4}

u = B

v = C

alt = 5



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	∞	∞	7	∞	∞	3
onceki	-	A	B	-	-	A	-	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

C,5

F,7

v = C: alt = 4 + 1 = 5 < ∞ → d[C] = 5, onceki[C] = B, {C, 5} kuyruğa.

Kod İzleme: dijkstra(adj, A)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     long[] e = pq.poll();
5     int u = (int) e[0];
6     if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         ▶ int v = k[0];
9         ▶ long alt = d[u] + k[1];
10        ▶ if (alt < d[v]) {
11            ▶ d[v] = alt;
12            ▶ onceki[v] = u;
13            ▶ pq.add(new long[] {v, alt});
14        }
15    }
16 }

```

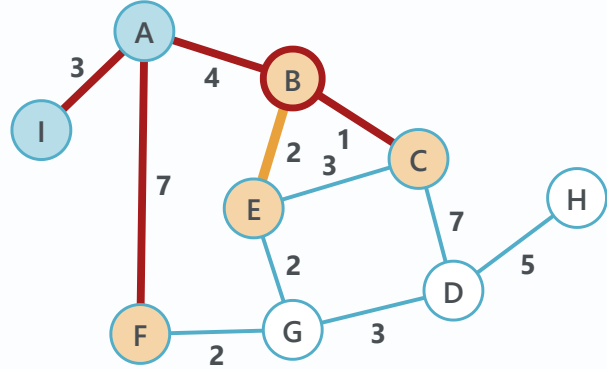
DEĞİŞKENLER

e = {B,4}

u = B

v = E

alt = 6



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	∞	6	7	∞	∞	3
onceki	-	A	B	-	B	A	-	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

C,5

E,6

F,7

v = E: alt = 4 + 2 = 6 < ∞ → d[E] = 6, onceki[E] = B, {E, 6} kuyruğa.

Kod İzleme: dijkstra(adj, A)

```

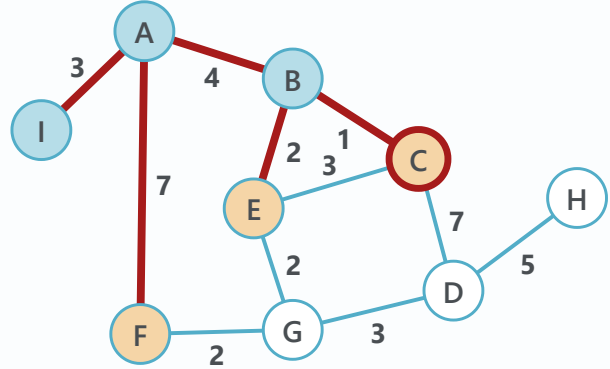
1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     ▶ long[] e = pq.poll();
5     ▶ int u = (int) e[0];
6     ▶ if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         int v = k[0];
9         long alt = d[u] + k[1];
10        if (alt < d[v]) {
11            d[v] = alt;
12            onceki[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```

DEĞİŞKENLER

e = {C, 5}

u = C



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	∞	6	7	∞	∞	3
onceki	-	A	B	-	B	A	-	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

E,6

F,7

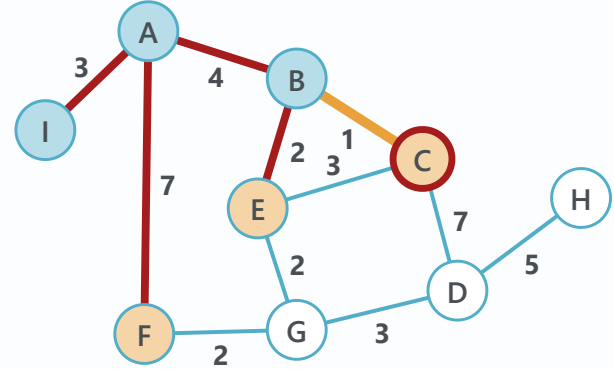
En küçük kayıt {C, 5} çıkar; 5 = d[C] (güncel) → **C kesinleşir**. Komşuları B, D, E gevşetilir.

Kod İzleme: dijkstra(adj, A)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     long[] e = pq.poll();
5     int u = (int) e[0];
6     if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         ▶ int v = k[0];
9         ▶ long alt = d[u] + k[1];
10        ▶ if (alt < d[v]) {
11            d[v] = alt;
12            onceki[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	∞	6	7	∞	∞	3
onceki	-	A	B	-	B	A	-	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

E,6

F,7

$v = B: \text{alt} = 6 \geq d[B] = 4 \rightarrow$ güncelleme yok.

DEĞİŞKENLER

e = {C,5}

u = C

v = B

alt = 6

Kod İzleme: dijkstra(adj, A)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     long[] e = pq.poll();
5     int u = (int) e[0];
6     if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         ▶ int v = k[0];
9         ▶ long alt = d[u] + k[1];
10        ▶ if (alt < d[v]) {
11            ▶ d[v] = alt;
12            ▶ onceki[v] = u;
13            ▶ pq.add(new long[] {v, alt});
14        }
15    }
16 }

```

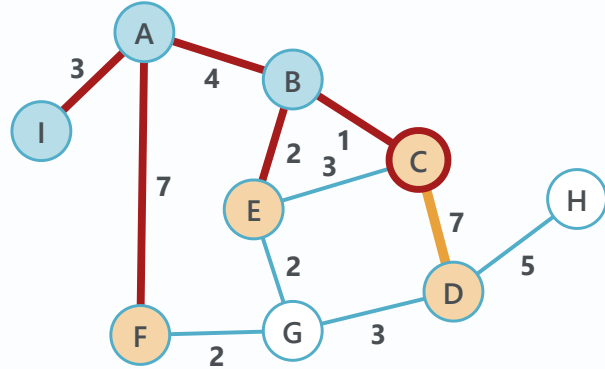
DEĞİŞKENLER

e = {C, 5}

u = C

v = D

alt = 12



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	12	6	7	∞	∞	3
onceki	-	A	B	C	B	A	-	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

E,6

F,7

D,12

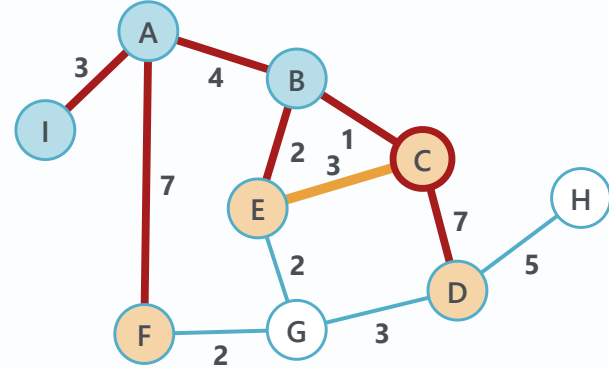
v = D: alt = 5 + 7 = **12** < ∞ → d[D] = 12, onceki[D] = C, {D, 12} kuyruğa.

Kod İzleme: dijkstra(adj, A)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     long[] e = pq.poll();
5     int u = (int) e[0];
6     if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         int v = k[0];
9         long alt = d[u] + k[1];
10        if (alt < d[v]) {
11            d[v] = alt;
12            onceki[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	12	6	7	∞	∞	3
onceki	-	A	B	C	B	A	-	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

E,6

F,7

D,12

$v = E: \text{alt} = 8 \geq d[E] = 6 \rightarrow$ güncelleme yok.

DEĞİŞKENLER

e = {C,5}

u = C

v = E

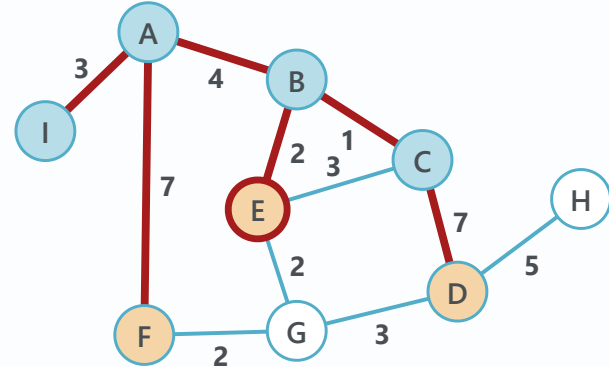
alt = 8

Kod İzleme: dijkstra(adj, A) (devam)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     ▶ long[] e = pq.poll();
5     ▶ int u = (int) e[0];
6     ▶ if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         int v = k[0];
9         long alt = d[u] + k[1];
10        if (alt < d[v]) {
11            d[v] = alt;
12            onceki[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	12	6	7	∞	∞	3
onceki	-	A	B	C	B	A	-	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

F,7

D,12

En küçük kayıt {E, 6} çıkar; 6 = d[E] (güncel) → **E kesinleşir**. Komşuları B, C, G gevşetilir.

DEĞİŞKENLER

e = {E, 6}

u = E

Kod İzleme: dijkstra(adj, A) (devam)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     long[] e = pq.poll();
5     int u = (int) e[0];
6     if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         ▶ int v = k[0];
9         ▶ long alt = d[u] + k[1];
10        ▶ if (alt < d[v]) {
11            d[v] = alt;
12            onceki[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```

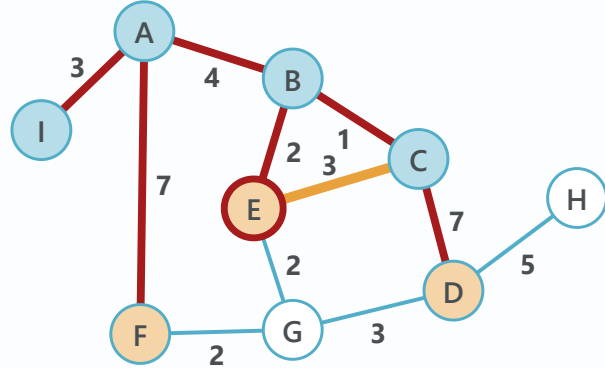
DEĞİŞKENLER

e = {E, 6}

u = E

v = C

alt = 9



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	12	6	7	∞	∞	3
onceki	-	A	B	C	B	A	-	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

F,7

D,12

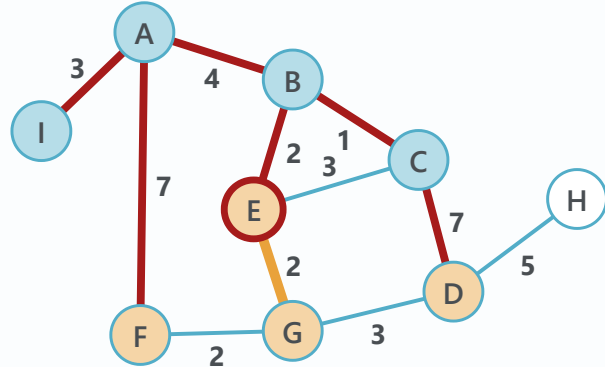
v = B: alt = 8 ≥ d[B] = 4; v = C: alt = 9 ≥ d[C] = 5 →
güncelleme yok.

Kod İzleme: dijkstra(adj, A) (devam)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     long[] e = pq.poll();
5     int u = (int) e[0];
6     if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         ▶ int v = k[0];
9         ▶ long alt = d[u] + k[1];
10        ▶ if (alt < d[v]) {
11            ▶ d[v] = alt;
12            ▶ onceki[v] = u;
13            ▶ pq.add(new long[] {v, alt});
14        }
15    }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	12	6	7	8	∞	3
onceki	-	A	B	C	B	A	E	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

F,7

G,8

D,12

$v = G$: $\text{alt} = 6 + 2 = 8 < \infty \rightarrow d[G] = 8, \text{onceki}[G] = E, \{G, 8\}$ kuyruğa.

DEĞİŞKENLER

e = {E,6}

u = E

v = G

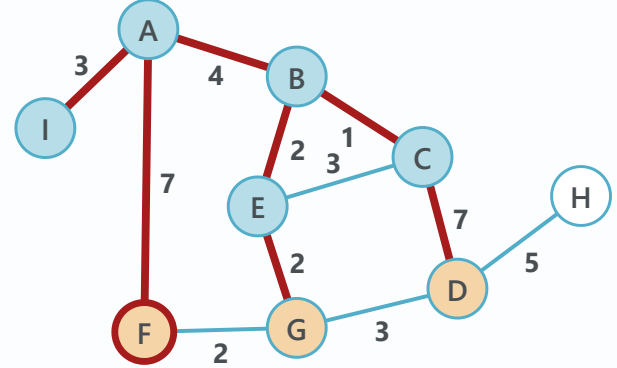
alt = 8

Kod İzleme: dijkstra(adj, A) (devam)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     ▶ long[] e = pq.poll();
5     ▶ int u = (int) e[0];
6     ▶ if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         int v = k[0];
9         long alt = d[u] + k[1];
10        if (alt < d[v]) {
11            d[v] = alt;
12            onceki[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	12	6	7	8	∞	3
onceki	-	A	B	C	B	A	E	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

G,8

D,12

En küçük kayıt {F, 7} çıkar; 7 = d[F] (güncel) → **F kesinleşir**. Komşuları A, G gevşetilir.

DEĞİŞKENLER

e = {F, 7}

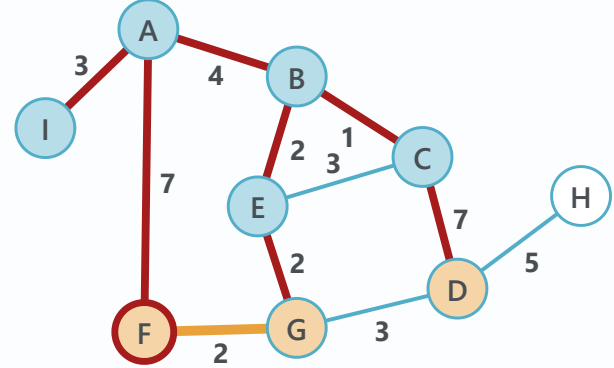
u = F

Kod İzleme: dijkstra(adj, A) (devam)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     long[] e = pq.poll();
5     int u = (int) e[0];
6     if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         ▶ int v = k[0];
9         ▶ long alt = d[u] + k[1];
10        ▶ if (alt < d[v]) {
11            d[v] = alt;
12            onceki[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	12	6	7	8	∞	3
onceki	-	A	B	C	B	A	E	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

G,8

D,12

v = A: alt = 14 ≥ d[A] = 0; v = G: alt = 9 ≥ d[G] = 8 →
güncelleme yok.

DEĞİŞKENLER

e = {F,7}

u = F

v = G

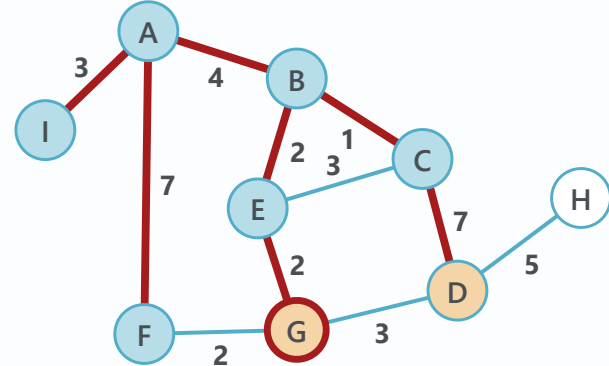
alt = 9

Kod İzleme: dijkstra(adj, A) (devam)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     ▶ long[] e = pq.poll();
5     ▶ int u = (int) e[0];
6     ▶ if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         int v = k[0];
9         long alt = d[u] + k[1];
10        if (alt < d[v]) {
11            d[v] = alt;
12            onceki[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	12	6	7	8	∞	3
onceki	-	A	B	C	B	A	E	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

D,12

En küçük kayıt {G, 8} çıkar; 8 = d[G] (güncel) → **G kesinleşir**. Komşuları D, E, F gevşetilir.

DEĞİŞKENLER

e = {G, 8}

u = G

Kod İzleme: dijkstra(adj, A) (devam)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     long[] e = pq.poll();
5     int u = (int) e[0];
6     if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         ▶ int v = k[0];
9         ▶ long alt = d[u] + k[1];
10        ▶ if (alt < d[v]) {
11            ▶ d[v] = alt;
12            ▶ onceki[v] = u;
13            ▶ pq.add(new long[] {v, alt});
14        }
15    }
16 }

```

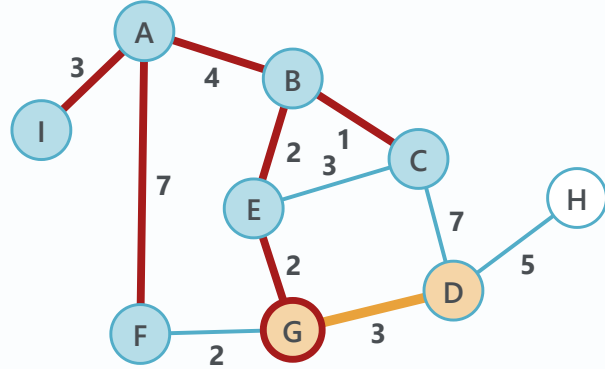
DEĞİŞKENLER

e = {G, 8}

u = G

v = D

alt = 11



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	11	6	7	8	∞	3
onceki	-	A	B	G	B	A	E	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

D, 11

~~D, 12~~

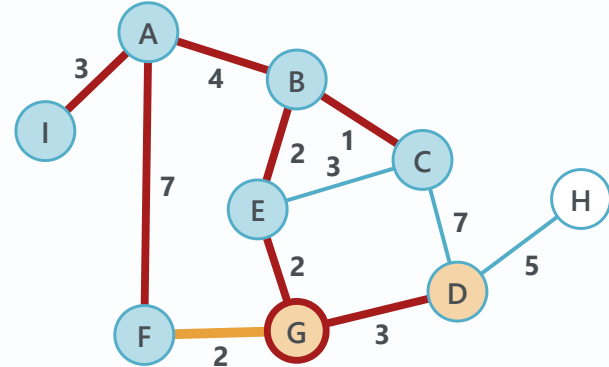
v = D: alt = 8 + 3 = **11** < 12 → d[D] = 11, onceki[D] = G, {D, 11} kuyruğa. Eski {D, 12} kaydı kuyruқта **bayat** kalır.

Kod İzleme: dijkstra(adj, A) (devam)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     long[] e = pq.poll();
5     int u = (int) e[0];
6     if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         ▶ int v = k[0];
9         ▶ long alt = d[u] + k[1];
10        ▶ if (alt < d[v]) {
11            d[v] = alt;
12            onceki[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	11	6	7	8	∞	3
onceki	-	A	B	G	B	A	E	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

D,11

~~D,12~~

v = E: alt = 10 ≥ d[E] = 6; v = F: alt = 10 ≥ d[F] = 7 →
güncelleme yok.

DEĞİŞKENLER

e = {G,8}

u = G

v = F

alt = 10

Kod İzleme: dijkstra(adj, A) (devam)

```

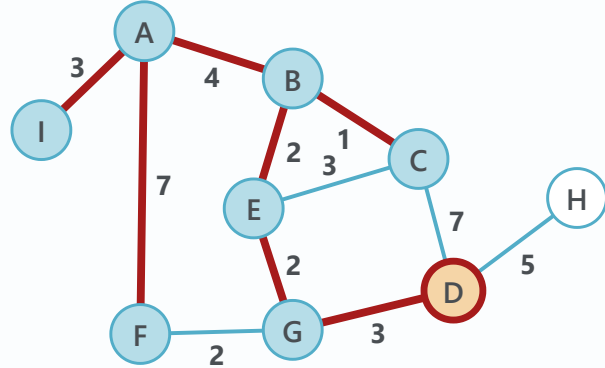
1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     ▶ long[] e = pq.poll();
5     ▶ int u = (int) e[0];
6     ▶ if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         int v = k[0];
9         long alt = d[u] + k[1];
10        if (alt < d[v]) {
11            d[v] = alt;
12            onceki[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```

DEĞİŞKENLER

e = {D, 11}

u = D



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	11	6	7	8	∞	3
onceki	-	A	B	G	B	A	E	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

~~D, 12~~

En küçük kayıt {D, 11} çıkar; 11 = d[D] (güncel) → **D kesinleşir**. Komşuları C, G, H gevrşetilir.

Kod İzleme: dijkstra(adj, A) (devam)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     long[] e = pq.poll();
5     int u = (int) e[0];
6     if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         ▶ int v = k[0];
9         ▶ long alt = d[u] + k[1];
10        ▶ if (alt < d[v]) {
11            d[v] = alt;
12            onceki[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```

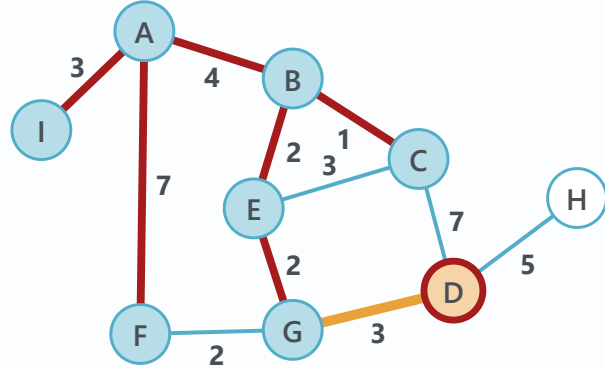
DEĞİŞKENLER

e = {D, 11}

u = D

v = G

alt = 14



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	11	6	7	8	∞	3
onceki	-	A	B	G	B	A	E	-	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

~~D, 12~~

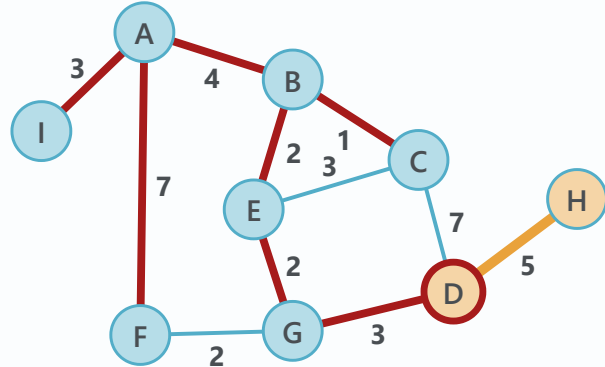
v = C: alt = 18 ≥ d[C] = 5; v = G: alt = 14 ≥ d[G] = 8
 → güncelleme yok.

Kod İzleme: dijkstra(adj, A) (devam)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     long[] e = pq.poll();
5     int u = (int) e[0];
6     if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         ▶ int v = k[0];
9         ▶ long alt = d[u] + k[1];
10        ▶ if (alt < d[v]) {
11            ▶ d[v] = alt;
12            ▶ onceki[v] = u;
13            ▶ pq.add(new long[] {v, alt});
14        }
15    }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	11	6	7	8	16	3
onceki	-	A	B	G	B	A	E	D	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

~~D,12~~ H,16

$v = H: \text{alt} = 11 + 5 = 16 < \infty \rightarrow d[H] = 16, \text{onceki}[H] = D, \{H, 16\}$ kuyruğa.

DEĞİŞKENLER

e = {D, 11}

u = D

v = H

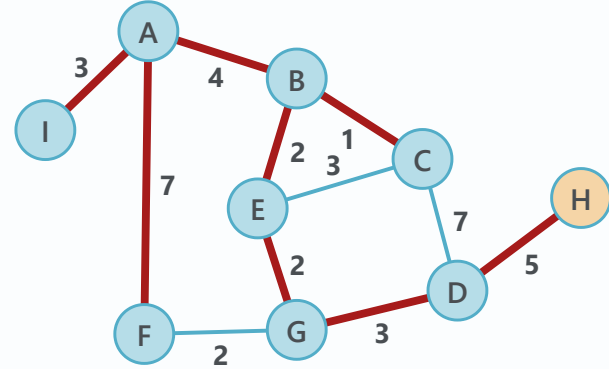
alt = 16

Kod İzleme: dijkstra(adj, A) (devam)

```

1 // d[] = ∞, d[s] = 0, oncesi[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     ▶ long[] e = pq.poll();
5     ▶ int u = (int) e[0];
6     ▶ if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         int v = k[0];
9         long alt = d[u] + k[1];
10        if (alt < d[v]) {
11            d[v] = alt;
12            oncesi[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	11	6	7	8	16	3
onceki	-	A	B	G	B	A	E	D	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)

H,16

{D, 12} çıktı ama d[D] = 11 < 12: **bayat kayı**t (daha iyi yol sonradan bulundu) → **continue**, atlanır.

DEĞİŞKENLER

e = {D,12}

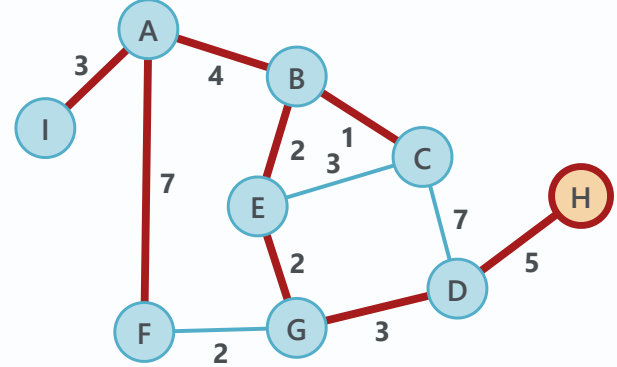
u = D

Kod İzleme: dijkstra(adj, A) (devam)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     ▶ long[] e = pq.poll();
5     ▶ int u = (int) e[0];
6     ▶ if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         int v = k[0];
9         long alt = d[u] + k[1];
10        if (alt < d[v]) {
11            d[v] = alt;
12            onceki[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	11	6	7	8	16	3
onceki	-	A	B	G	B	A	E	D	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)
(boş)

En küçük kayıt {H, 16} çıkar; 16 = d[H] (güncel) → **H kesinleşir**. Komşusu D gevşetilir.

DEĞİŞKENLER

e = {H, 16}

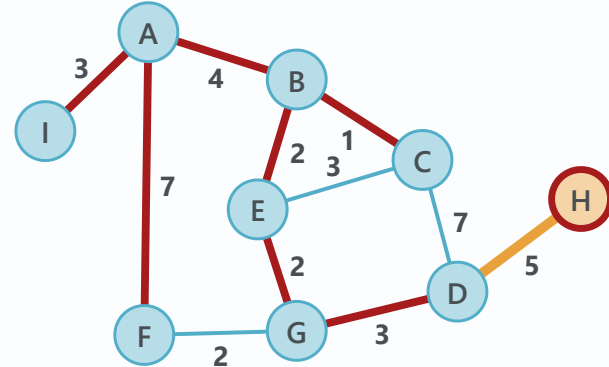
u = H

Kod İzleme: dijkstra(adj, A) (devam)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     long[] e = pq.poll();
5     int u = (int) e[0];
6     if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         ▶ int v = k[0];
9         ▶ long alt = d[u] + k[1];
10        ▶ if (alt < d[v]) {
11            d[v] = alt;
12            onceki[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```



	A	B	C	D	E	F	G	H	I
d[]	0	4	5	11	6	7	8	16	3
onceki	-	A	B	G	B	A	E	D	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)
(boş)

v = D: alt = 21 ≥ d[D] = 11 → güncelleme yok.

DEĞİŞKENLER

e = {H, 16}

u = H

v = D

alt = 21

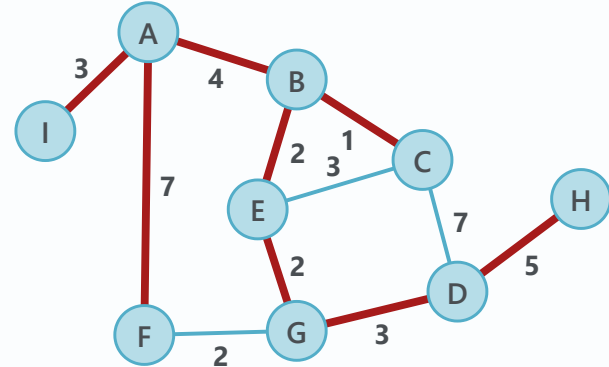
Kod İzleme: dijkstra(adj, A) (devam)

```

1 // d[] = ∞, d[s] = 0, onceki[] = -1
2 pq.add(new long[] {s, 0});
3 while (!pq.isEmpty()) {
4     long[] e = pq.poll();
5     int u = (int) e[0];
6     if (e[1] > d[u]) continue; // bayat
7     for (int[] k : adj.get(u)) {
8         int v = k[0];
9         long alt = d[u] + k[1];
10        if (alt < d[v]) {
11            d[v] = alt;
12            onceki[v] = u;
13            pq.add(new long[] {v, alt});
14        }
15    }
16 }

```

DEĞİŞKENLER

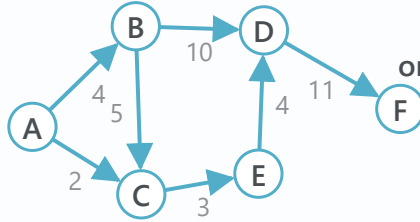


	A	B	C	D	E	F	G	H	I
d[]	0	4	5	11	6	7	8	16	3
onceki	-	A	B	G	B	A	E	D	A

pq (mesafeye göre sıralı gösterim; soluk = bayat)
(boş)

Kuyruk boş → bitti. onceki[] geri izlenir: $H \leftarrow D \leftarrow G \leftarrow E \leftarrow B \leftarrow A$. En kısa yol **A → B → E → G → D → H = 16**.

Kod İzleme: `onceki[]` ile en kısa yolu yazdırma



v	A	B	C	D	E	F
onceki	-	A	A	E	C	D
y						

addFirst → hedeften kaynağa doğru, başa ekle

Dijkstra'nın bıraktığı `onceki[]` : her düğüm için en kısa yolda *kendinden önceki* düğüm. Boş liste y.

Adım 1 / 14

```

1 List<Integer> yol(int[] onceki, int s, int t) {
2   ► LinkedList<Integer> y = new LinkedList<>();
3   for (int v = t; v != -1; v = onceki[v])
4     y.addFirst(v);
5   if (y.getFirst() != s) return List.of();
6   return y;           // s ... t
7 }

```

DEĞİŞKENLER

onceki = [-, A, A, E, C, D]

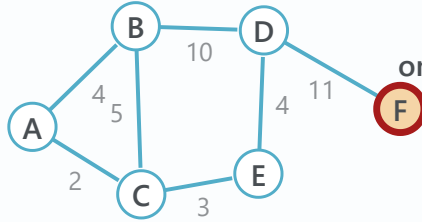
y = []

ÇAĞRI YIĞINI

yol(onceki, A, F)

main()

Kod İzleme: `onceki[]` ile en kısa yolu yazdırma



v	A	B	C	D	E	F
onceki	-	A	A	E	C	D
y						

addFirst → hedeften kaynağa doğru, başa ekle

Hedeften başla: **v = F**.

Adım 2 / 14

```

1 List<Integer> yol(int[] onceki, int s, int t) {
2   LinkedList<Integer> y = new LinkedList<>();
3   for (int v = t; v != -1; v = onceki[v])
4     y.addFirst(v);
5   if (y.getFirst() != s) return List.of();
6   return y;                // s ... t
7 }

```

DEĞİŞKENLER

onceki = [-, A, A, E, C, D]

v = F

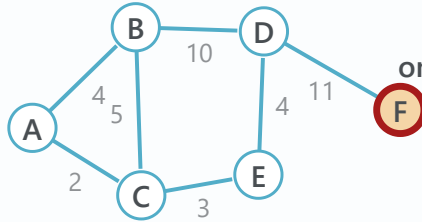
y = []

ÇAĞRI YIĞINI

yol(onceki, A, F)

main()

Kod İzleme: `onceki[]` ile en kısa yolu yazdırma



v	A	B	C	D	E	F
onceki	-	A	A	E	C	D
y	F					

addFirst → hedeften kaynağa doğru, başa ekle

F listenin **başına** eklenir (sonra gelenler daha önceki düğümler).

Adım 3 / 14

```

1 List<Integer> yol(int[] onceki, int s, int t) {
2   LinkedList<Integer> y = new LinkedList<>();
3   for (int v = t; v != -1; v = onceki[v])
4     y.addFirst(v);
5   if (y.getFirst() != s) return List.of();
6   return y;                // s ... t
7 }

```

DEĞİŞKENLER

onceki = [-, A, A, E, C, D]

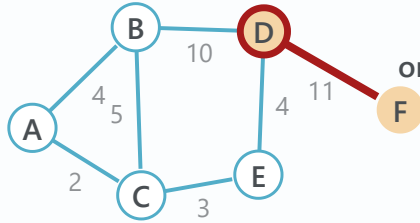
v = F y = [F]

ÇAĞRI YIĞINI

yol(onceki, A, F)

main()

Kod İzleme: `onceki[]` ile en kısa yolu yazdırma



v	A	B	C	D	E	F
onceki	-	A	A	E	C	D

y	F					
---	---	--	--	--	--	--

addFirst → hedeften kaynağa doğru, başa ekle

v = onceki[...] = D: bir adım geri.

Adım 4 / 14

```

1 List<Integer> yol(int[] onceki, int s, int t) {
2   LinkedList<Integer> y = new LinkedList<>();
3   ▶ for (int v = t; v != -1; v = onceki[v])
4     y.addFirst(v);
5   if (y.getFirst() != s) return List.of();
6   return y;           // s ... t
7 }

```

DEĞİŞKENLER

onceki = [-, A, A, E, C, D]

v = D

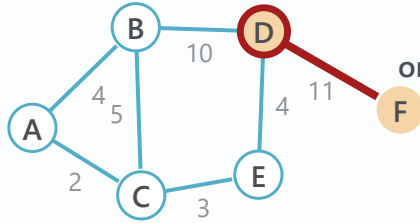
y = [F]

ÇAĞRI YIĞINI

yol(onceki, A, F)

main()

Kod İzleme: önceki[] ile en kısa yolu yazdırma



v	A	B	C	D	E	F
önceki	-	A	A	E	C	D

y	D	F				
---	---	---	--	--	--	--

addFirst → hedeften kaynağa doğru, başa ekle

D listenin **başına** eklenir (sonra gelenler daha önceki düğümler).

Adım 5 / 14

```

1 List<Integer> yol(int[] önceki, int s, int t) {
2   LinkedList<Integer> y = new LinkedList<>();
3   for (int v = t; v != -1; v = önceki[v])
4     y.addFirst(v);
5   if (y.getFirst() != s) return List.of();
6   return y;           // s ... t
7 }

```

DEĞİŞKENLER

önceki = [-, A, A, E, C, D]

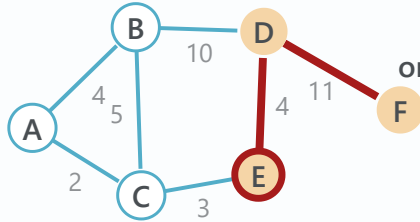
v = D y = [D, F]

ÇAĞRI YIĞINI

yol(önceki, A, F)

main()

Kod İzleme: `onceki[]` ile en kısa yolu yazdırma



v	A	B	C	D	E	F
onceki	-	A	A	E	C	D
y	D	F				

addFirst → hedeften kaynağa doğru, başa ekle

v = onceki[...] = E: bir adım geri.

Adım 6 / 14

```

1 List<Integer> yol(int[] onceki, int s, int t) {
2   LinkedList<Integer> y = new LinkedList<>();
3   for (int v = t; v != -1; v = onceki[v])
4     y.addFirst(v);
5   if (y.getFirst() != s) return List.of();
6   return y;                // s ... t
7 }

```

DEĞİŞKENLER

onceki = [-, A, A, E, C, D]

v = E

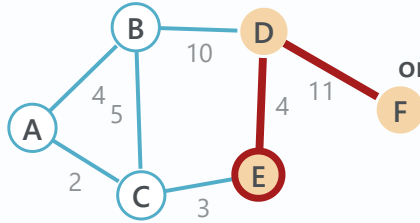
y = [D, F]

ÇAĞRI YIĞINI

yol(onceki, A, F)

main()

Kod İzleme: `onceki[]` ile en kısa yolu yazdırma



v	A	B	C	D	E	F
onceki	-	A	A	E	C	D
y	E	D	F			

addFirst → hedeften kaynağa doğru, başa ekle

E listenin **başına** eklenir (sonra gelenler daha önceki düğümler).

Adım 7 / 14

```

1 List<Integer> yol(int[] onceki, int s, int t) {
2     LinkedList<Integer> y = new LinkedList<>();
3     for (int v = t; v != -1; v = onceki[v])
4         y.addFirst(v);
5     if (y.getFirst() != s) return List.of();
6     return y;           // s ... t
7 }

```

DEĞİŞKENLER

onceki = [-, A, A, E, C, D]

v = E

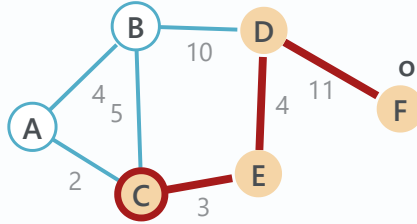
y = [E, D, F]

ÇAĞRI YIĞINI

yol(onceki, A, F)

main()

Kod İzleme: `onceki[]` ile en kısa yolu yazdırma



v	A	B	C	D	E	F
onceki	-	A	A	E	C	D
y	E	D	F			

addFirst → hedeften kaynağa doğru, başa ekle

v = onceki[...] = C: bir adım geri.

Adım 8 / 14

```

1 List<Integer> yol(int[] onceki, int s, int t) {
2   LinkedList<Integer> y = new LinkedList<>();
3   for (int v = t; v != -1; v = onceki[v])
4     y.addFirst(v);
5   if (y.getFirst() != s) return List.of();
6   return y;                // s ... t
7 }
```

DEĞİŞKENLER

onceki = [-, A, A, E, C, D]

v = C

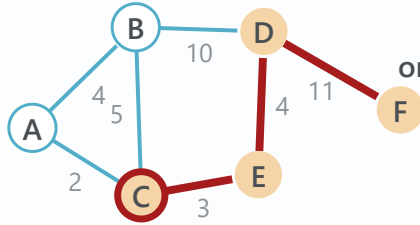
y = [E, D, F]

ÇAĞRI YIĞINI

yol(onceki, A, F)

main()

Kod İzleme: önceki[] ile en kısa yolu yazdırma



v	A	B	C	D	E	F
önceki	-	A	A	E	C	D
y	C	E	D	F		

addFirst → hedeften kaynağa doğru, başa ekle

C listenin **başına** eklenir (sonra gelenler daha önceki düğümler).

Adım 9 / 14

```

1 List<Integer> yol(int[] önceki, int s, int t) {
2   LinkedList<Integer> y = new LinkedList<>();
3   for (int v = t; v != -1; v = önceki[v])
4     y.addFirst(v);
5   if (y.getFirst() != s) return List.of();
6   return y;           // s ... t
7 }

```

DEĞİŞKENLER

önceki = [-, A, A, E, C, D]

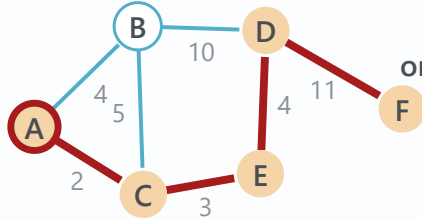
v = C y = [C, E, D, F]

ÇAĞRI YIĞINI

yol(önceki, A, F)

main()

Kod İzleme: `onceki[]` ile en kısa yolu yazdırma



v	A	B	C	D	E	F
onceki	-	A	A	E	C	D

y	C	E	D	F		
---	---	---	---	---	--	--

addFirst → hedeften kaynağa doğru, başa ekle

v = onceki[...] = A: bir adım geri.

Adım 10 / 14

```

1 List<Integer> yol(int[] onceki, int s, int t) {
2   LinkedList<Integer> y = new LinkedList<>();
3   for (int v = t; v != -1; v = onceki[v])
4     y.addFirst(v);
5   if (y.getFirst() != s) return List.of();
6   return y;           // s ... t
7 }

```

DEĞİŞKENLER

onceki = [-, A, A, E, C, D]

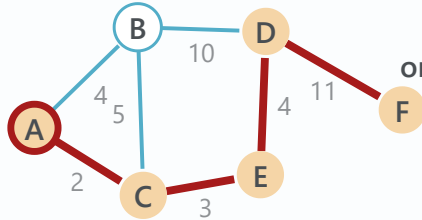
v = A y = [C, E, D, F]

ÇAĞRI YIĞINI

yol(onceki, A, F)

main()

Kod İzleme: `onceki[]` ile en kısa yolu yazdırma



v	A	B	C	D	E	F
onceki	-	A	A	E	C	D
y	A	C	E	D	F	

addFirst → hedeften kaynağa doğru, başa ekle

A listenin **başına** eklenir (sonra gelenler daha önceki düğümler).

Adım 11 / 14

```

1 List<Integer> yol(int[] onceki, int s, int t) {
2     LinkedList<Integer> y = new LinkedList<>();
3     for (int v = t; v != -1; v = onceki[v])
4         y.addFirst(v);
5     if (y.getFirst() != s) return List.of();
6     return y;                // s ... t
7 }

```

DEĞİŞKENLER

onceki = [-, A, A, E, C, D]

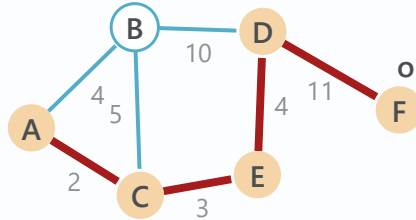
v = A y = [A, C, E, D, F]

ÇAĞRI YIĞINI

yol(onceki, A, F)

main()

Kod İzleme: `onceki[]` ile en kısa yolu yazdırma



v	A	B	C	D	E	F
onceki	-	A	A	E	C	D
y	A	C	E	D	F	
	A	C	E	D	F	

addFirst → hedeften kaynağa doğru, başa ekle

`onceki[A] = -1`: kaynağa varıldı, döngü biter. 5 tur = yoldaki düğüm sayısı → $O(\text{yol uzunluğu})$.

Adım 12 / 14

```

1 List<Integer> yol(int[] onceki, int s, int t) {
2   LinkedList<Integer> y = new LinkedList<>();
3   for (int v = t; v != -1; v = onceki[v])
4     y.addFirst(v);
5   if (y.getFirst() != s) return List.of();
6   return y;                // s ... t
7 }

```

DEĞİŞKENLER

`onceki = [-, A, A, E, C, D]`

`v = -1`

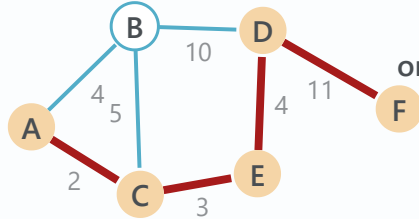
`y = [A, C, E, D, F]`

ÇAĞRI YIĞINI

`yol(onceki, A, F)`

`main()`

Kod İzleme: `onceki[]` ile en kısa yolu yazdırma



v	A	B	C	D	E	F
onceki	-	A	A	E	C	D
y	A	C	E	D	F	

addFirst → hedeften kaynağa doğru, başa ekle

İlk öge kaynak A mı? Evet → t ulaşılabilir. (Ulaşılamayan t için zincir A'ya varmaz; boş liste döner.)

Adım 13 / 14

```

1 List<Integer> yol(int[] onceki, int s, int t) {
2   LinkedList<Integer> y = new LinkedList<>();
3   for (int v = t; v != -1; v = onceki[v])
4     y.addFirst(v);
5   ► if (y.getFirst() != s) return List.of();
6   return y;           // s ... t
7 }

```

DEĞİŞKENLER

onceki = [-, A, A, E, C, D]

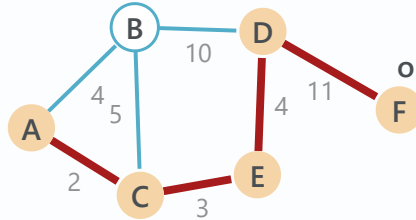
y = [A, C, E, D, F]

ÇAĞRI YIĞINI

yol(onceki, A, F)

main()

Kod İzleme: `onceki[]` ile en kısa yolu yazdırma



v	A	B	C	D	E	F
onceki	-	A	A	E	C	D
y	A	C	E	D	F	

addFirst → hedeften kaynağa doğru, başa ekle

Sonuç **A → C → E → D → F**, uzunluk $d[F] = 2 + 3 + 4 + 11 = 20$.

Adım 14 / 14

```

1 List<Integer> yol(int[] onceki, int s, int t) {
2     LinkedList<Integer> y = new LinkedList<>();
3     for (int v = t; v != -1; v = onceki[v])
4         y.addFirst(v);
5     if (y.getFirst() != s) return List.of();
6     return y; // s ... t
7 }
```

DEĞİŞKENLER

onceki = [-, A, A, E, C, D]

dönüş = [A, C, E, D, F]

ÇAĞRI YIĞINI

yol(onceki, A, F)

main()

Dijkstra Algoritmasının Analizi

Uygulama	Karmaşıklık	Neden?
Öncelikli kuyruk (ikili heap)	$O((n + m) \log n)$	n kez en küçüğü çıkar, en çok m kez güncelle/ekle; her biri $O(\log n)$
Kuyruksuz (dizi taraması)	$O(n^2 + m) = O(n^2)$	Her adımda en küçük mesafeli düğümü bulmak için n düğüm taranır; komşu kontrolleri toplam m

Hangisi ne zaman?

Seyrek çizgede ($m \approx n$) heap: $O(n \log n)$. Çok yoğun çizgede ($m \approx n^2$) dizi taraması $O(n^2)$ daha iyi olabilir.

Negatif ağırlık

Kesinleşen bir mesafe sonradan negatif kenarla kısalabilir → sözde koddaki Dijkstra yanlış sonuç verir. Bayat kayıtlı Java sürümü düğümü yeniden işleyip doğruyu bulabilir ama süre garantisi kaybolur, negatif döngüde bitmez. Bu durumda Bellman-Ford kullanılır.

Neden doğru? Ağırlıklar ≥ 0 iken, kuyruktaki en küçük mesafeli düğüme başka bir düğüm üzerinden daha kısa ulaşılamaz: öbür yollar zaten en az bu kadar uzundur.

Özet

Kavramlar

Düğüm, kenar, derece, yön, ağırlık, yol, döngü ve bağlantılı bileşen.

Gösterimler

Seyrek çizgede komşuluk listesi; hızlı kenar sorgusunda matris.

Algoritma	Yapı	Zaman (liste ile)	Ne verir?
BFS	kuyruk	$O(n + m)$	seviye sırası, ağırlıksız en kısa yol
DFS	yığın / özyineleme	$O(n + m)$	derinlik öncelikli gezinme, döngü, bileşenler
Topolojik	kuyruk / DFS	$O(n + m)$	DAG için bağımlılık sırası
Dijkstra	öncelikli kuyruk	$O((n + m) \log n)$	ağırlıklı en kısa yol (ağırlık ≥ 0)

Veri yapısı algoritmanın davranışını belirler: BFS kuyruk, DFS yığın, Dijkstra öncelikli kuyruk kullanır; topolojik sıralama ise yalnızca DAG'lerde tanımlıdır.

Etkileşimli Simülatörler

Bu bölümün konularını tarayıcıda deneyerek pekiştirin:

Yönsüz Çizge

Düğüm/kenar ekle-sil, BFS ve DFS, bağlantılı bileşenler, komşuluk listesi ve matrisi görünümü.

Yönlü Çizge

Oklu kenarlar, giriş/çıkış dereceleri ve yönlü gezinme.

Dijkstra

Ağırlıklı çizgede en kısa yolları adım adım izleyin.

Uygulama: [Ders Ön Koşul Planlayıcı](#) → Kahn algoritmasıyla dersleri dönem dönem planlayın; ön koşul döngüsünü yakalayın.

Örnek Problemler

1. 10 düğümlü basit yönsüz bir çizgede en fazla kaç kenar olabilir? Derecelerin toplamı 30 ise kaç kenar vardır?
2. $n = 10.000$ düğüm, $m = 30.000$ kenar olan bir yol ağını hangi gösterimle saklarsınız? Neden?
3. Gezinme örneğindeki çizgede BFS ve DFS'yi **H**'den başlatın (komşular alfabetik).
4. DFS'yi yığın yerine kuyrukla yazarsak hangi algoritmayı elde ederiz?
5. Ağırlıklı en kısa yol örneğinde $B \rightarrow C$ kenarı 5 yerine 1 olsaydı A'dan D'ye en kısa yol ne olurdu?

► Cevapları göster

Alıştırmalar

1. Yönsüz kenarlar $(0,1)$, $(0,2)$, $(1,3)$, $(2,3)$, $(3,4)$, $(4,5)$: 0'dan BFS sırası ve mesafeler? (komşular küçükten büyüğe)
2. Aynı çizgenin komşuluk matrisinde kaç hücre 1'dir? Komşuluk listesinde toplam kaç kayıt vardır?
3. Yönlü $0 \rightarrow 1$, $1 \rightarrow 2$, $2 \rightarrow 0$, $2 \rightarrow 3$ için `donguVar(0)` hangi kenarı incelerken true döner?
4. $0 \rightarrow 1$, $0 \rightarrow 2$, $1 \rightarrow 3$, $2 \rightarrow 3$ DAG'ı için kaç farklı topolojik sıra vardır? Yazın.
5. `onceki = [-1, 0, 0, 1, 3]` ise 0'dan 4'e yol nedir? 2'den 4'e?
6. 5 düğümlü bir döngü (C_5) iki parçalı mıdır? Bağlantılı bileşen sayısı kaçtır?

► Cevaplar / ipuçları

Kaynaklar

- Goodrich, Tamassia ve Goldwasser, *Data Structures and Algorithms in Java*, 6. baskı, 2014.
- Cormen, Leiserson, Rivest ve Stein, *Introduction to Algorithms*, 4. baskı, 2022.
- Sedgewick ve Wayne, *Algorithms*, 4. baskı, 2011.

Son

Bölüm 11: Çizgeler

Sonraki sunum: [Bölüm 12: Kümeler](#) · [Çalışma soruları](#) · Simülatörler: [Undirected Graph Simulator](#), [Directed Graph Simulator](#), [Dijkstra Algorithm Simulator](#), [Course Prerequisite Planner](#) · [Terim sözlüğü](#)